

# 算 法 设 计 与 分 析

## 向标混合索引结构与算法

Chao Zhang

Renmin University of China

## Background



## Filtering Strategy

- Pre-filtering
- Post-filtering
- In-filtering

## Equality-Oriented Vector Index

- Filtered DiskANN
- NHQ

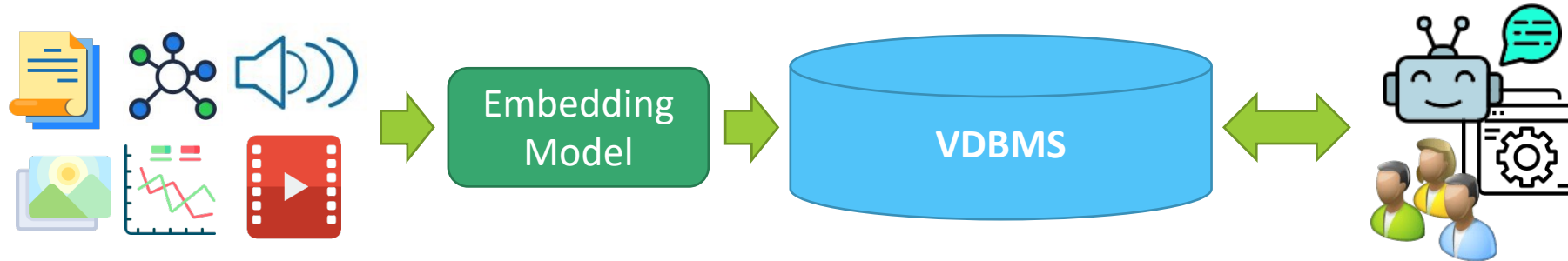
## Predicate-Agnostic Vector Index

- ACORN
- NaviX

## Range-Oriented Vector Index

- SeRF
- DIGRA

# Background

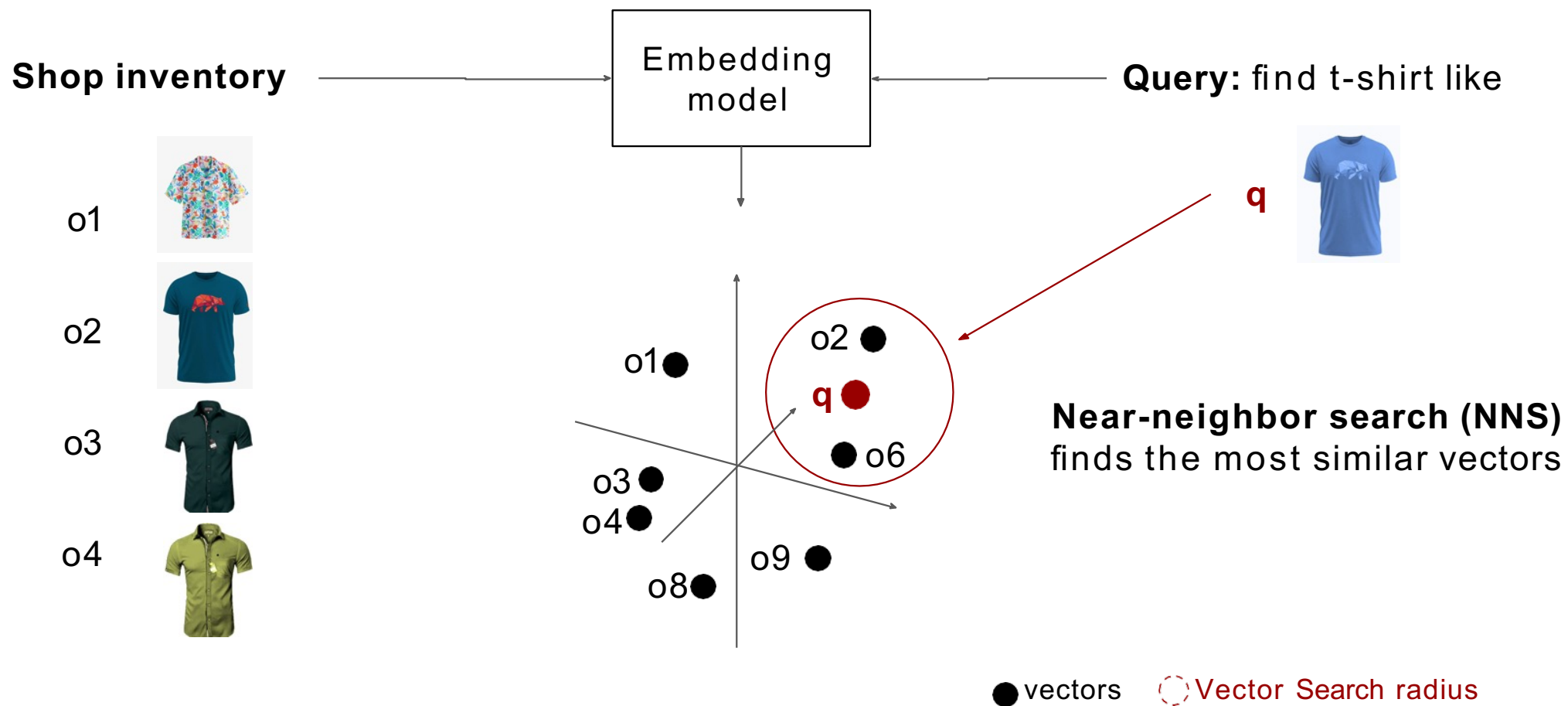


**Store embeddings in a database** and retrieve desired embeddings for whatever downstream task

- Capabilities: similarity-based top-k/range retrieval, hybrid attribute-vector retrieval, multi-modal (multi-vector) retrieval
- Characteristics: read/write latency/throughput, retrieval accuracy, scalability, availability, consistency, fault tolerance, privacy & security, elasticity

# Background

## Vector Search: Searching multi-modal data



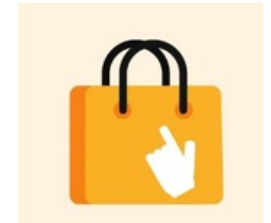


# Background

## Vector search is already a core operator

### Recommendation system

Find the 10 most similar products to my purchase

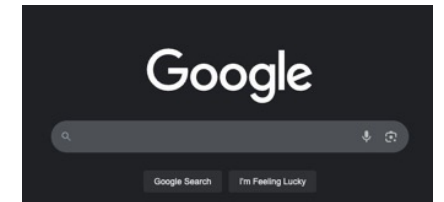


### Semantic search

Find 5 modern and minimal apartments

### Information retrieval

Google search



### RAG

Find relevant data and augment an LLM prompt



# Background

## Embeddings

- Models embed objects in a multidimensional space
- Modality-specific → toward general models
- Dimension sizes: [100s - 1000s dimensions]\* (e.g.: [0.2, 0.1, 0.42, 1.2, ...])
- **Distance captures similarity**

```
SELECT ...  
FROM tableX  
ORDER BY distance(q, vector_col)  
LIMIT BY K
```

sort on the **distance from a query vector**

**top-K nearest neighbors**

**Embeddings make data “structured”**

# Background

**Near-neighbor search (NNS) is not scalable** when it's accurate

**Query:** find top-3 vectors close to vector **q**

## NNS

|                 |
|-----------------|
| [0.2, 0.1, ...] |
| [0.5, 0.1, ...] |
| [0.3, 0.6, ...] |
| [0.5, 0.1, ...] |
| ...             |
| [0.5, 0.1, ...] |

**Calculate distance**  
from **q** and **sort**

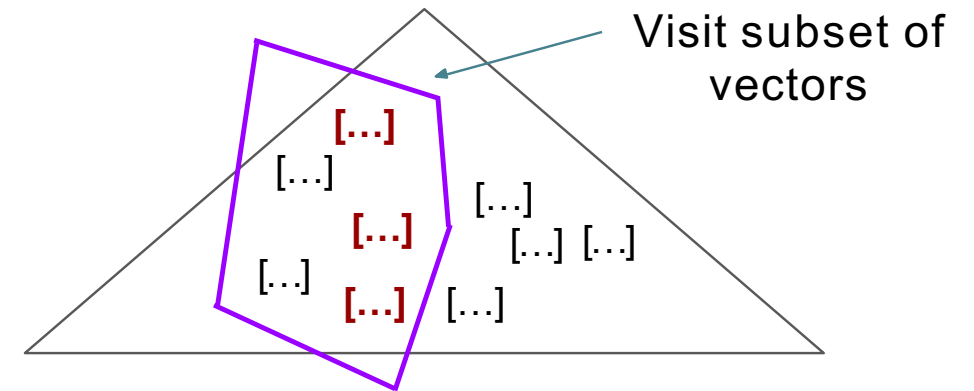


distance

|                 |     |
|-----------------|-----|
| [0.2, 0.1, ...] | 0.5 |
| [0.5, 0.1, ...] | 1.1 |
| [0.3, 0.6, ...] | 2   |
| [0.5, 0.1, ...] | 5   |
| ...             |     |
| [0.5, 0.1, ...] | 50  |

Expensive:  $O(\#vectors)$

## Approximate-NNS via Vector Indices

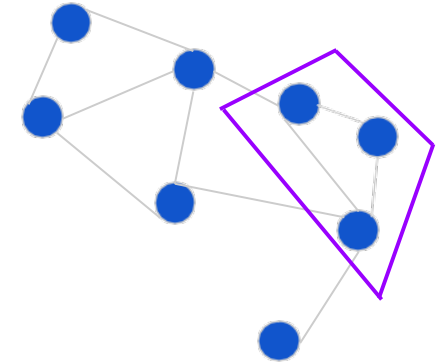


Trade-off  
accuracy for performance

# Background

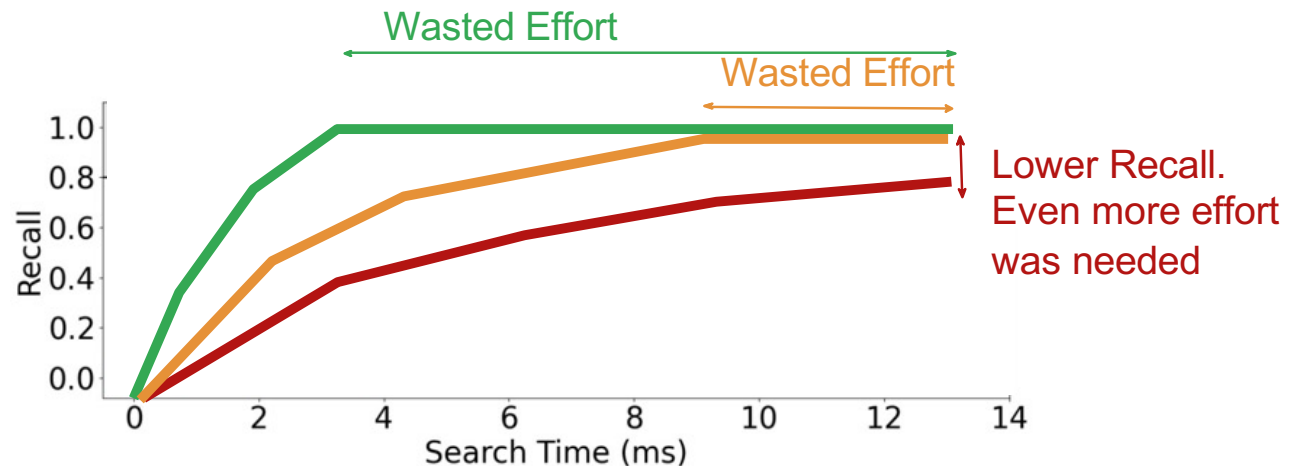
## Performance vs Recall Trade-off in Approximate Nearest Neighbor Search

$\text{ANNS\_search}(\mathbf{q}, K, \text{search\_effort\_params})$



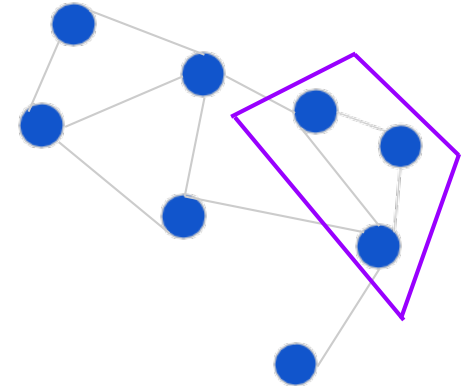
How many vectors to visit to achieve a user specified **target recall** ?

**Challenge**  
Queries have different  
hardness, different search effort  
is needed



# Background

```
ANNS_search(q, K, search_effort_params)
```



**Hard for users and experts to tune**

## **Uniform autotuning for all queries**

Learned offline models (eg Google's CloudSQL VectorAssist)

## **Different for each query**

A model predicts the search effort parameters for each query/index

## **Adaptive**

Decide to continue/stop (early stop) based on the current search state



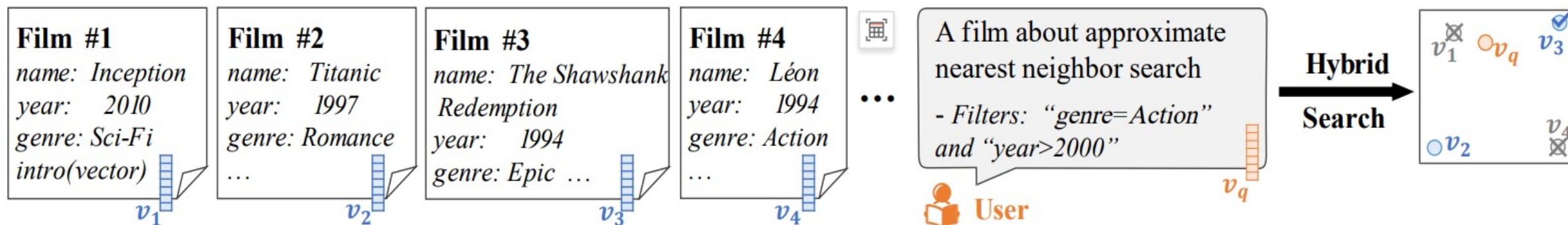
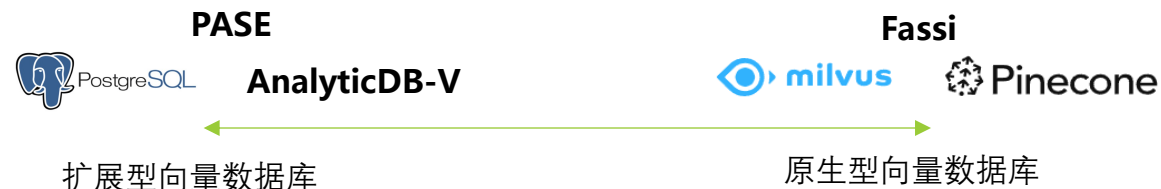
# Filtered Vector Search

## ■ 背景：ANNS 已广泛成为基础能力

- 核心目标：高维向量下的 Top-k 相似搜索
- 应用场景：推荐系统、RAG、多模态搜索

## ■ 动机：现实查询 ≠ 纯向量查询

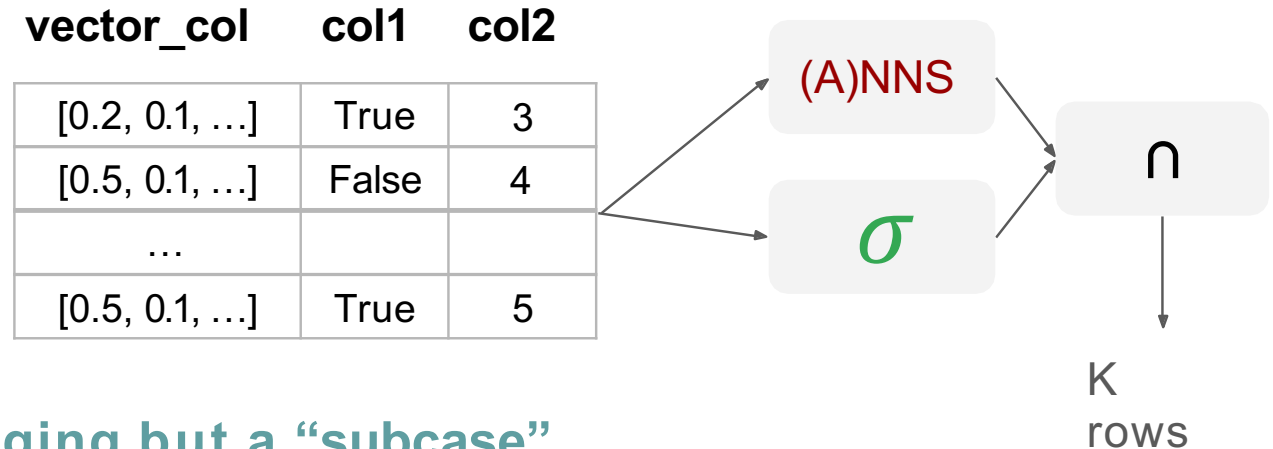
- 真实查询 = 向量 + 过滤条件 (category、time、set)
- 电影海报 + 电影年份 + 电影类型



# Filtered Vector Search

**Filtered Vector Search (FVS):** query structured and unstructured data

```
SELECT ...  
FROM shop_invectorory  
WHERE col1 = True and col2 > 5  
ORDER BY distance(q, vector_col)  
LIMIT BY K
```



Challenging but a “subcase”

**FVS = SQL + Vectors :**

- + Joins
- + Subquery expressions
- + Multiple vectors search (on both sides of a join)
- + Dataset is not vector + tag stored in main memory (common setup for most information retrieval scenarios)

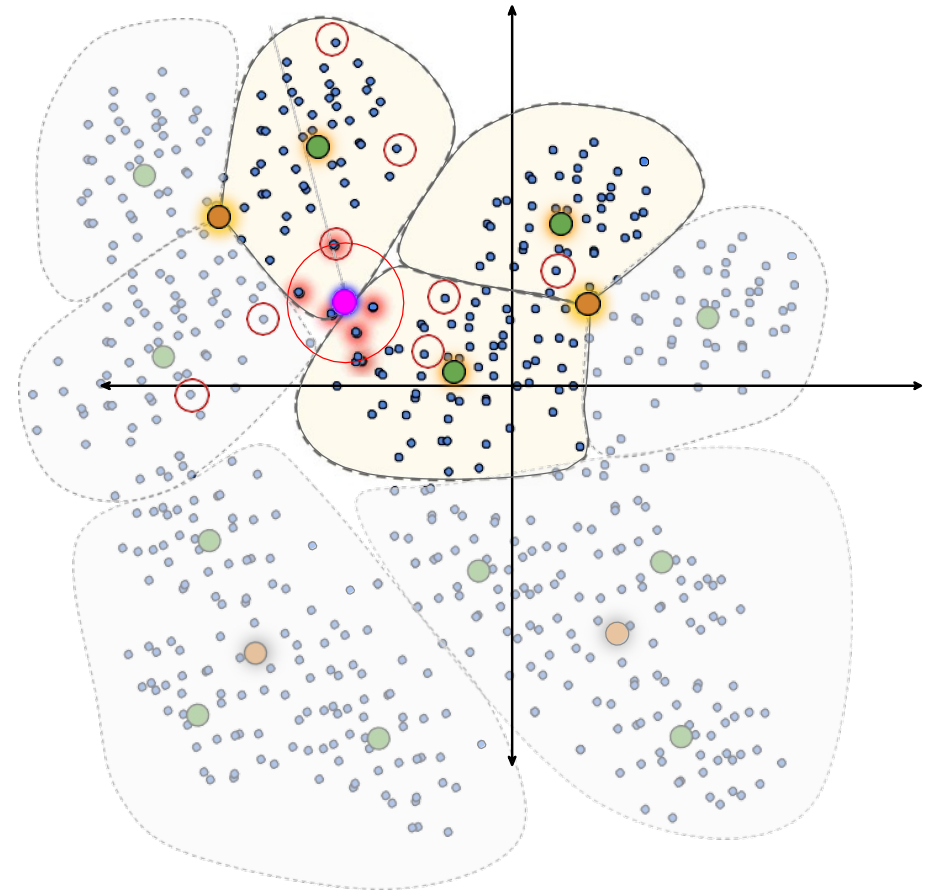
# Filtered Vector Search

## Filtered Vector Search (FVS)

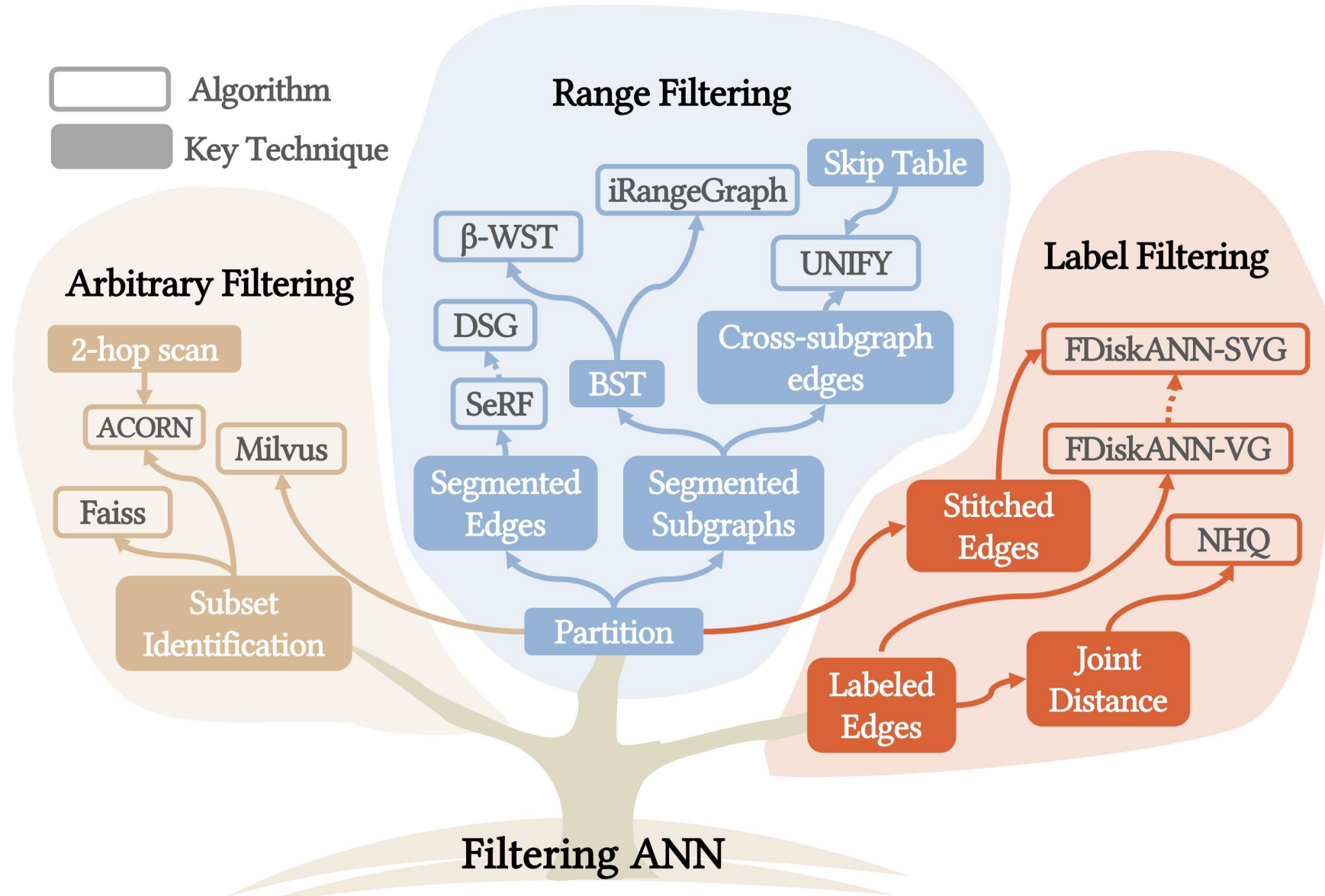
For a **query vector  $q$** ,  
**find the closest centroids/leaves**,  
compute the distances to their  
vectors that satisfy the conditions

For a LIMIT  $k$  query  
-> there may not even be  $k$   
rows/vectors that satisfy the condition  
-> there may be  $k$  but the ones  
furthest away are inferior solutions

**Inspect more centroids/leaves but  
the wasted effort Vs recall tradeoff  
becomes harder**



# Filtered Vector Search



## Taxonomy of Filtering ANN

### Attribute Types

Numerical  
(Range)

$l \leq \text{attr} \leq u$

Categorical  
(Label)

$f \in A$

Arbitrary  
(General)

Any predicate

### Filtering Strategies

#### Pre-filtering

Filter first, then ANN search.  
Effective for IVF; may break graph paths.

#### Post-filtering

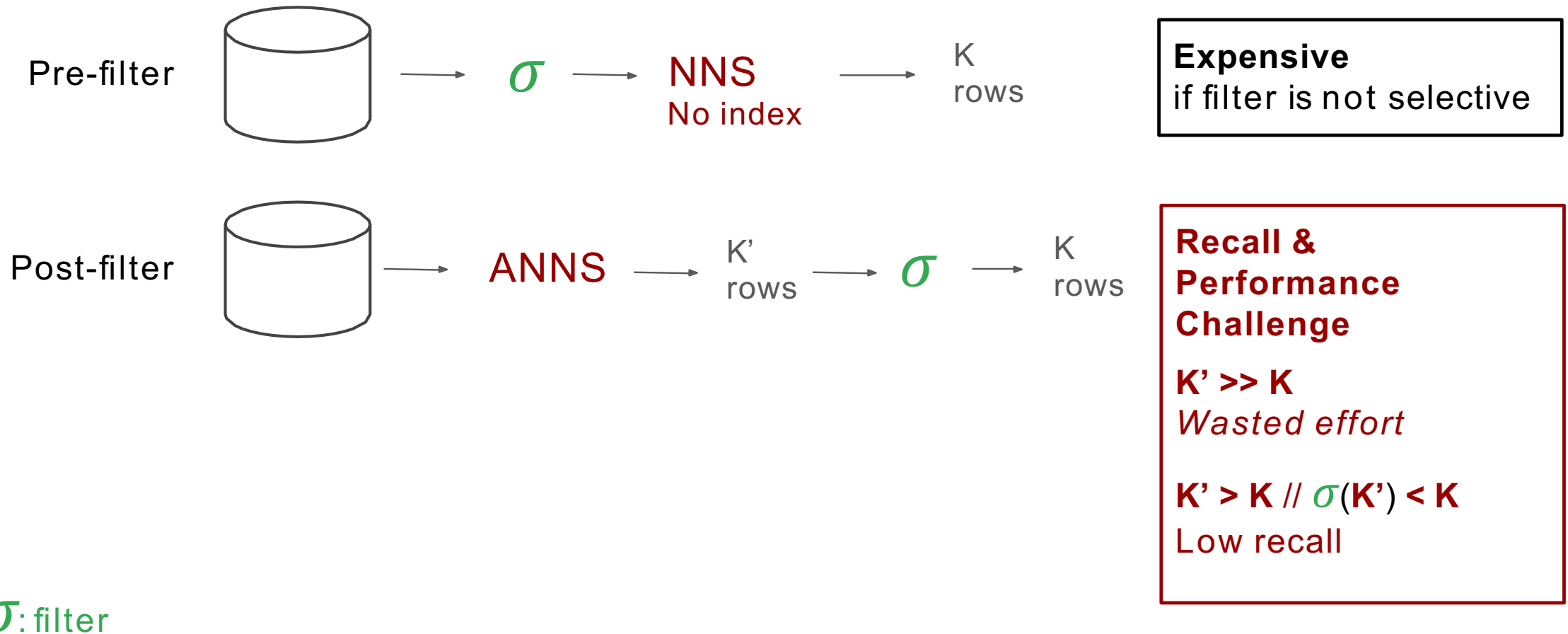
ANN search first, then filter results.  
Good at high selectivity; costly at low.

#### Joint-filtering

Filter during ANN search traversal.  
Best for moderate selectivity levels.

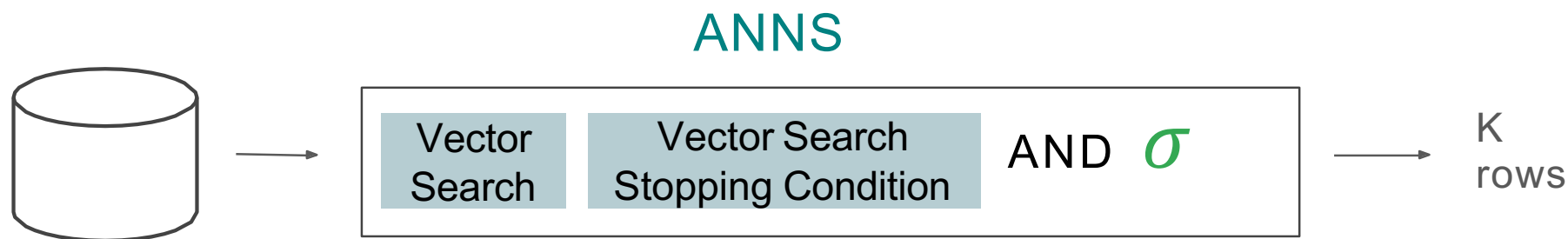
# Filtered Vector Search

## Basic execution methods



# Filtered Vector Search

## Inline-Filter/Joint-Filtering/In-Filtering



**Blur the line between ANNS and **ffiltering** to improve accuracy and performance**

\* multiple implementations of inline filtering

$\sigma$ : filter



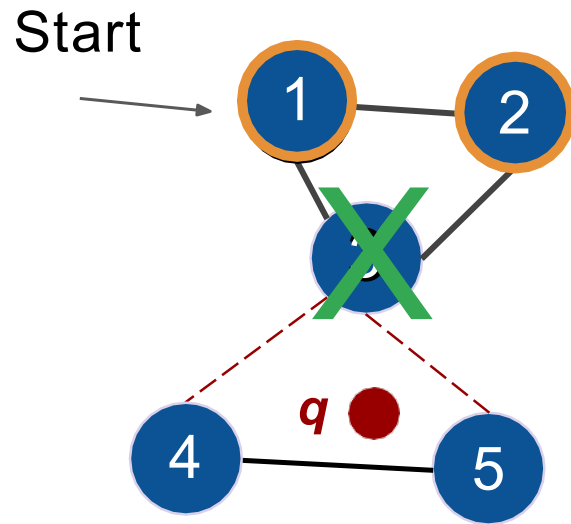
# Filtered Vector Search

## Predicate Subgraph Traversal

Filtered-out nodes

**DO NOT**

participate in navigation



Graph Inline-Filtering #1

|             | Next to Visit | Result |
|-------------|---------------|--------|
| (a) Visit 1 |               |        |
| (b) Visit 2 |               |        |

**Search Stops -> Low Recall**  
Connectivity Breaks

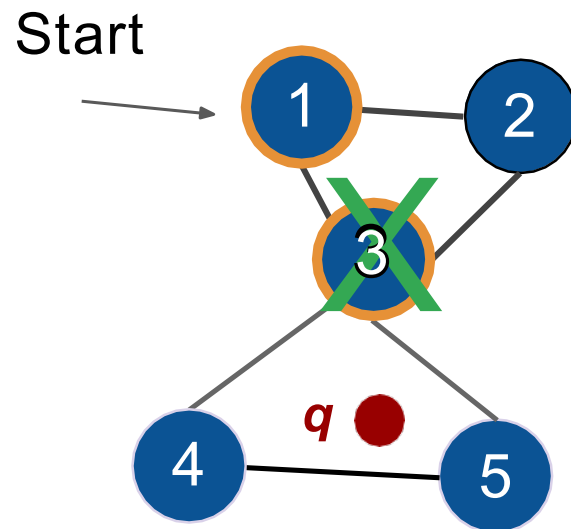
# Filtered Vector Search

## Sweeping

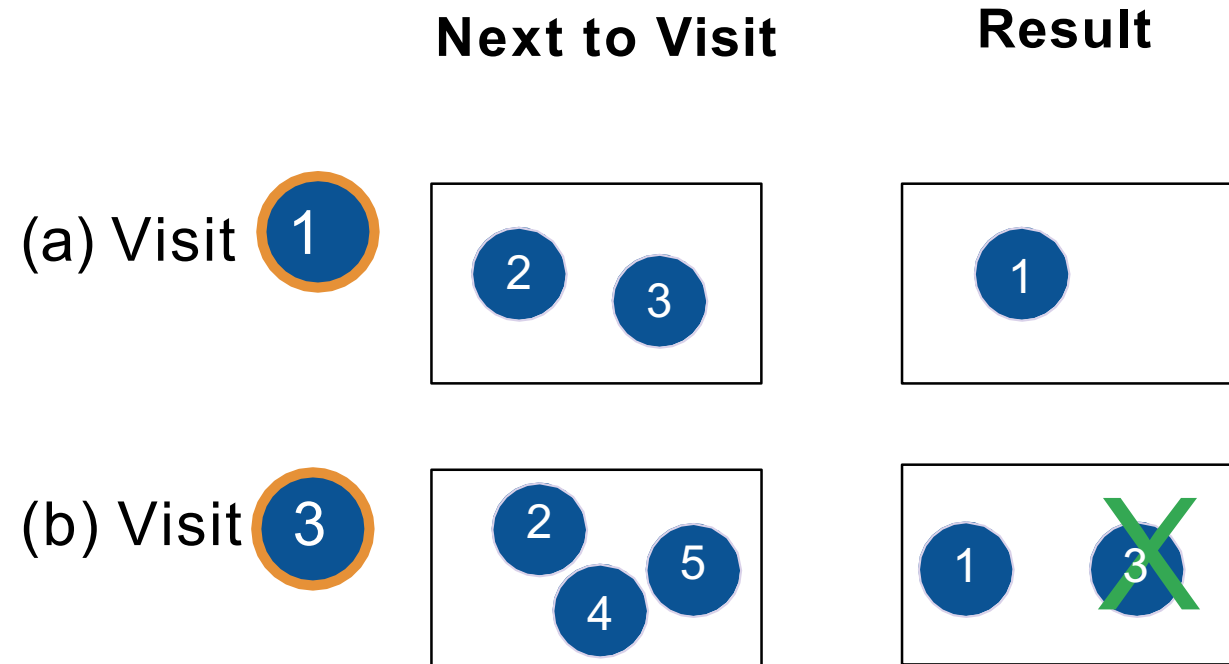
Filtered-out nodes

**DO**

participate in navigation



Graph Inline-Filtering #2

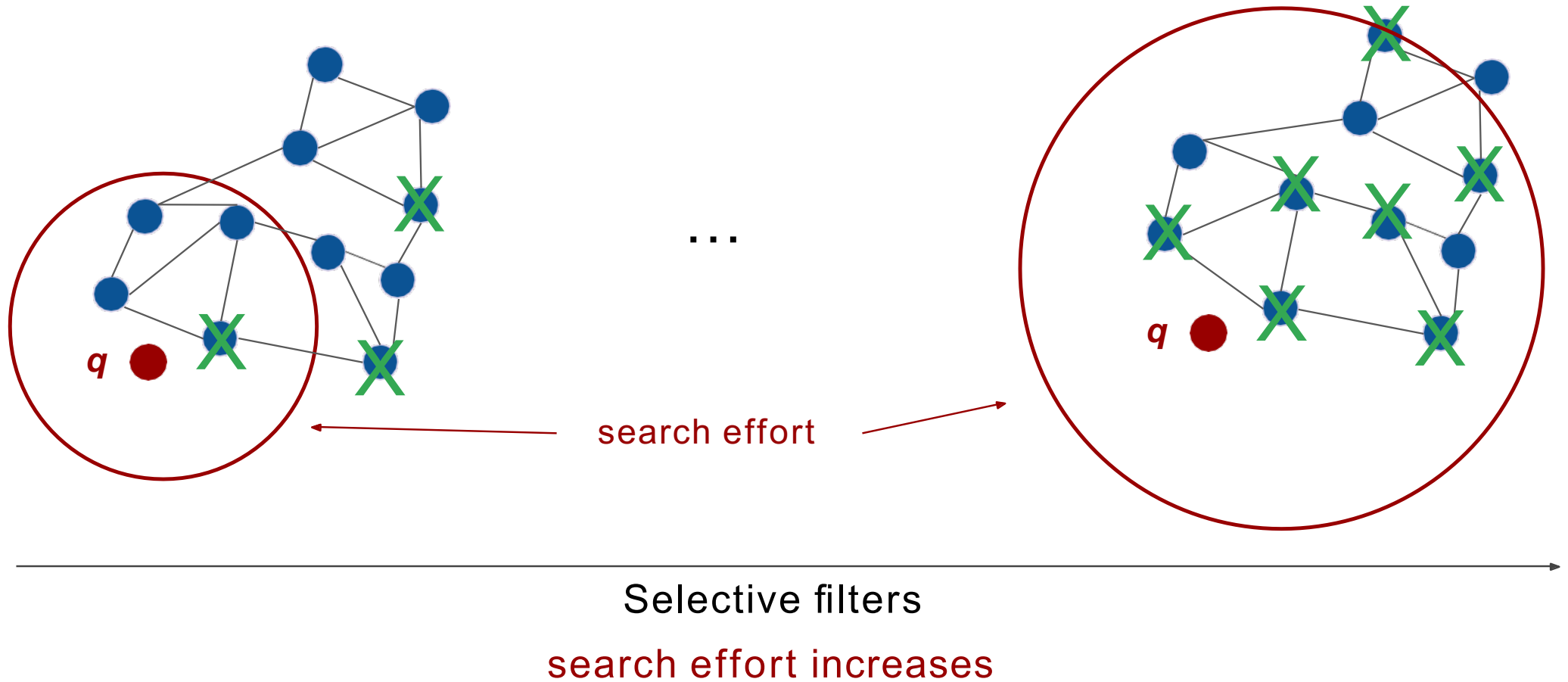


Graph remains connected **at the cost of more distance computations**

# Filtered Vector Search

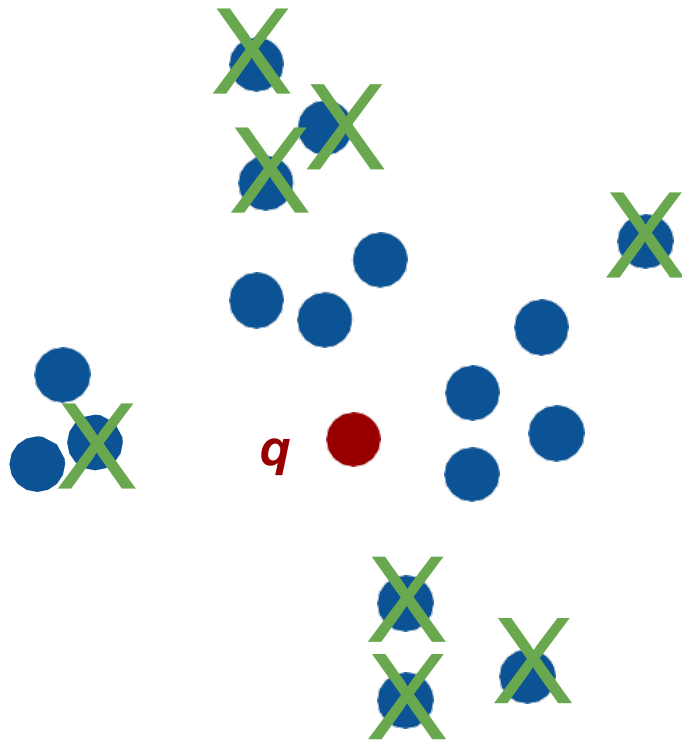
## Filter Selectivity

Selective filters make it harder to find valid nodes



# Filtered Vector Search

## Value-Vector Correlation

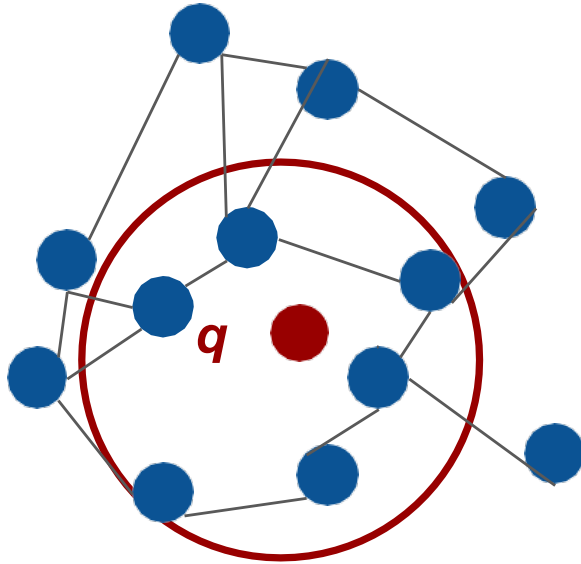


Captures the relationship between the **probability of satisfying the filters** and the **distance from the query vector**.

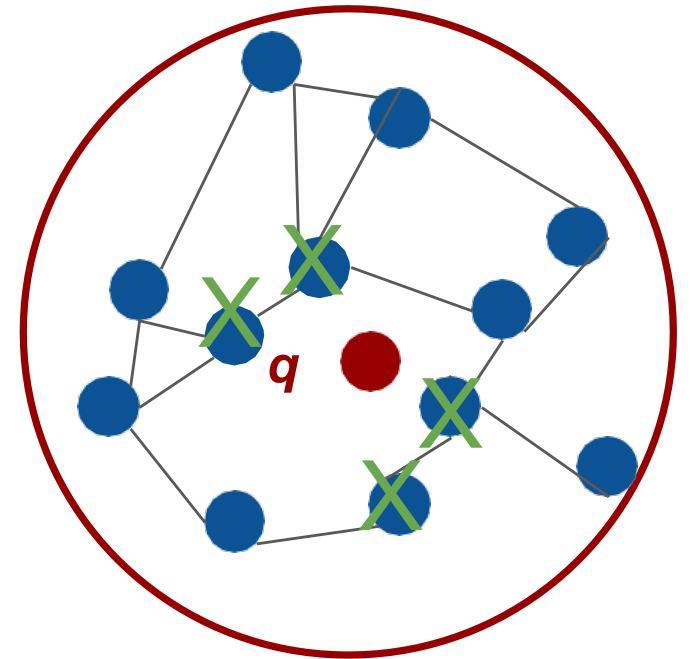
**Positive correlation**

# Filtered Vector Search

**Not all datasets+queries are equally easy**  
and filters change hardness



**Easy Query**



**Filters make it a  
Hard Query**

# Filtered Vector Search

## Background

## Filtering Strategy

- Pre-filtering
- Post-filtering
- In-filtering

## Equality-Oriented Vector Index

- Filtered DiskANN
- NHQ



## Predicate-Agnostic Vector Index

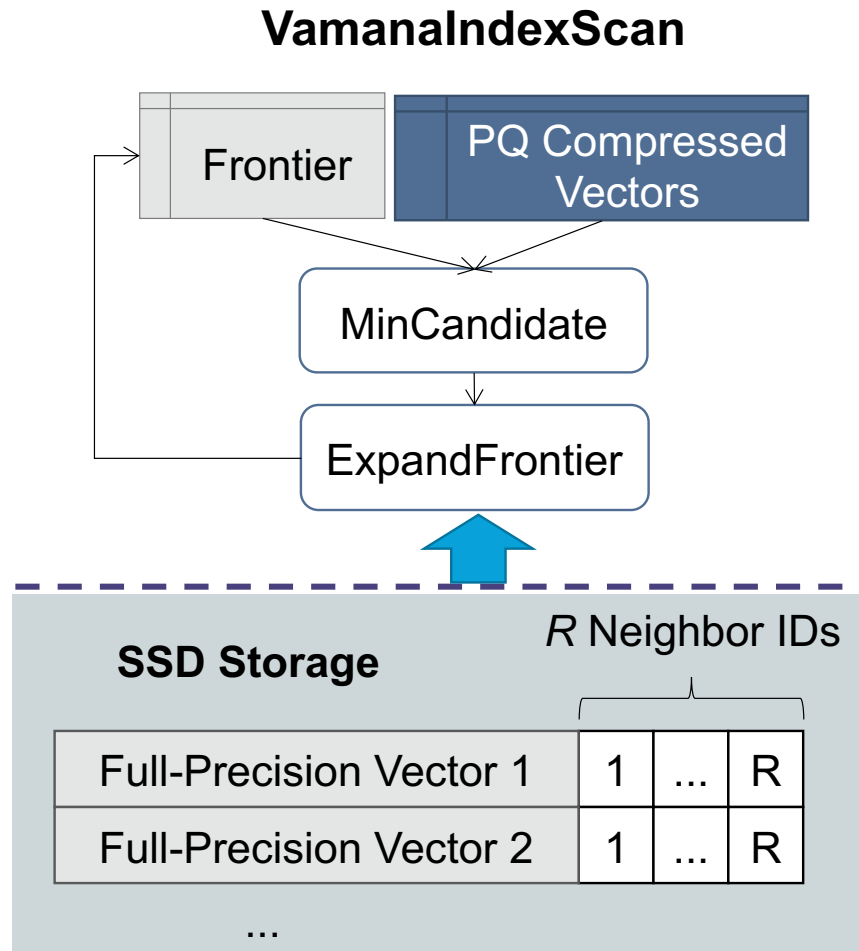
- ACORN
- NaviX

## Range-Oriented Vector Index

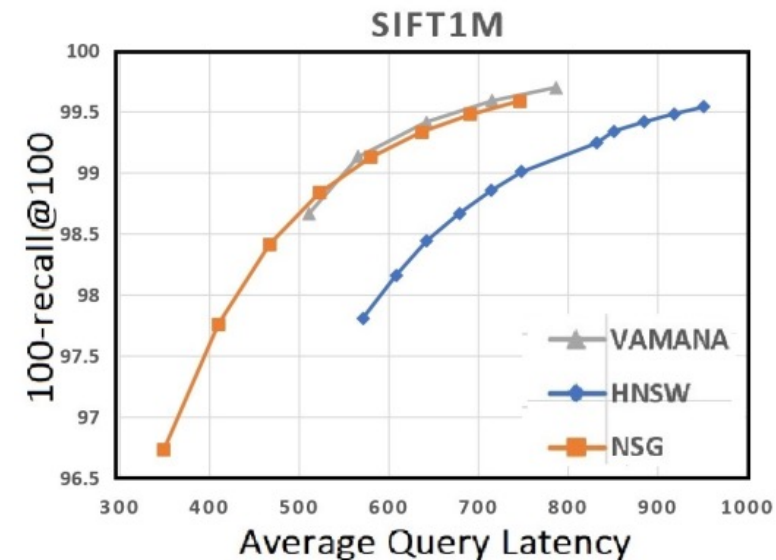
- SeRF
- DIGRA

# Filtered Vector Search

## Vamana/DiskANN



- Similar to NSG
- **On-disk neighborhoods**
- Edge traversal performed in memory using **compressed vectors**



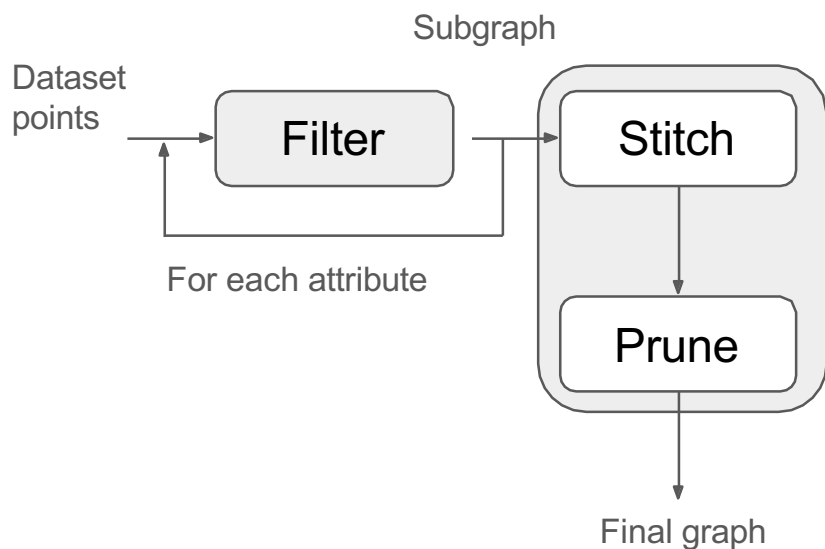


# Filtered Vector Search

## Deep Dive on [Filtered DiskANN](#) ([Gollapudi WWW '23](#))

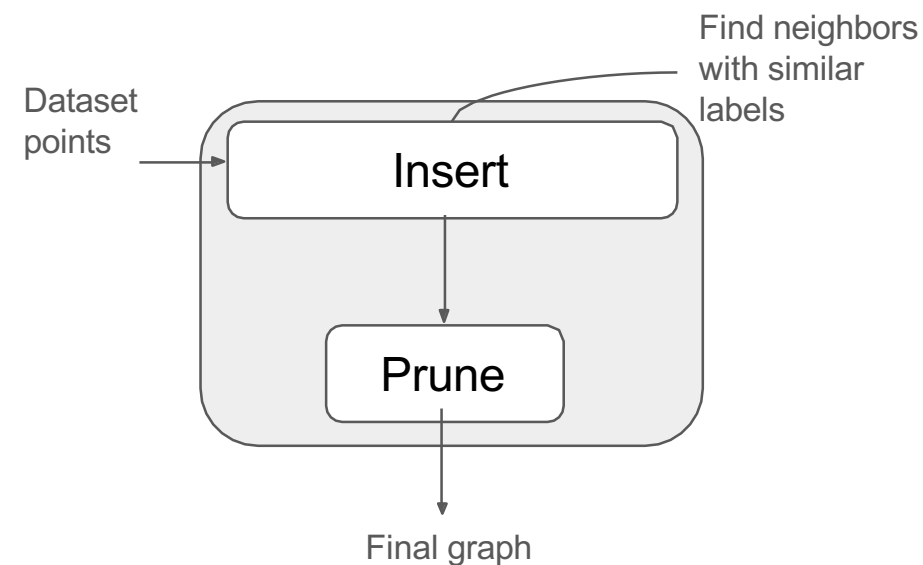
*Based on DiskANN (Vamana).*

### StitchedVamana



**Better QPS @same recall**

### FilteredVamana

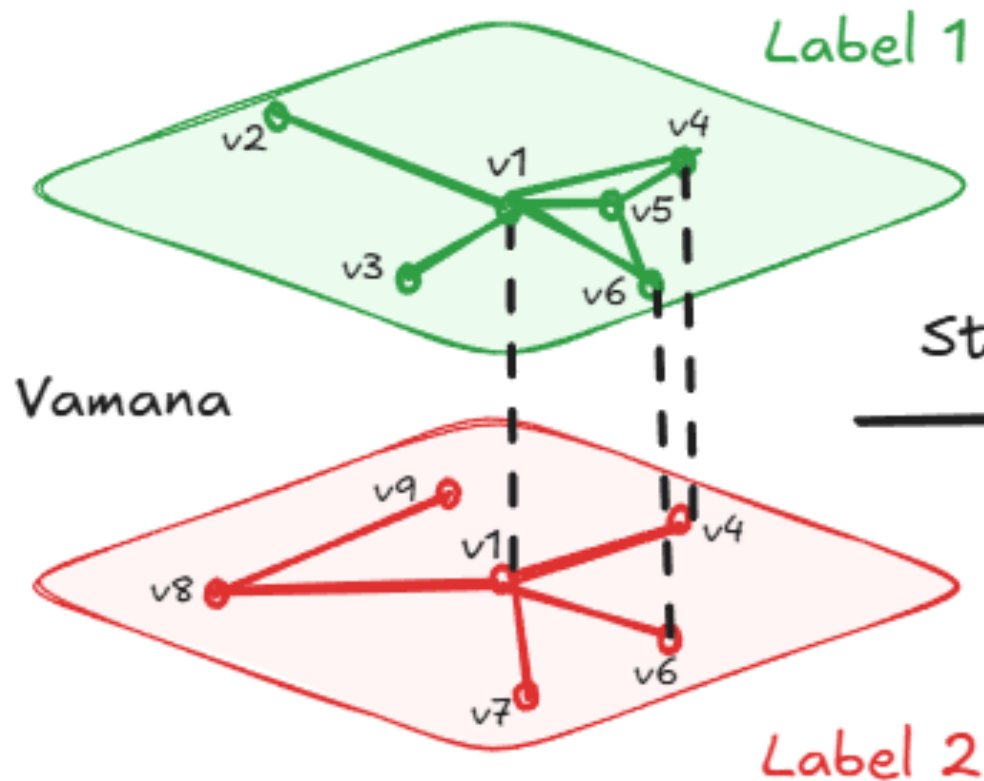


**Faster index building  
&  
More amenable to incremental updates**

# Filtered Vector Search

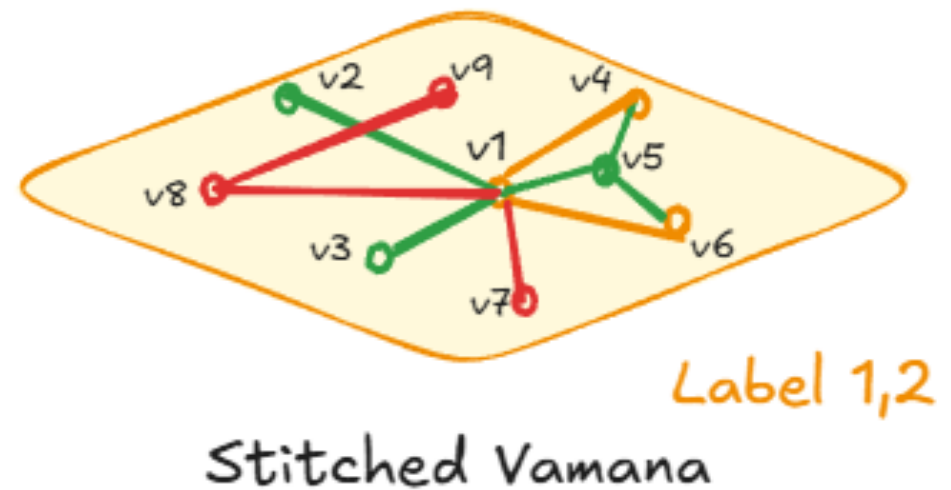
Node with Label 1

Node with Label 2



|    | v1                | v2                | v3                | v4                | v5                | v6                | v7                | v8                | v9                |
|----|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
| L1 | Node with Label 1 | Node with Label 1 | Node with Label 1 | Node with Label 1 | Node with Label 1 | Node with Label 1 | Node with Label 1 |                   |                   |
| L2 | Node with Label 2 |                   |                   | Node with Label 2 |                   | Node with Label 2 | Node with Label 2 | Node with Label 2 | Node with Label 2 |

Stitch



# Filtered Vector Search

---

**Algorithm 1:** FilteredGreedySearch( $S, x_q, k, L, F_q$ )

---

**Data:** Graph  $G$  with initial nodes  $S$ , query vector  $x_q$ , search list size  $\mathcal{L}$ , and query filter(s)  $F_q$ .

**Result:** Result set  $\mathcal{L}$  containing  $k$  approximate nearest neighbors, and a set  $\mathcal{V}$  containing all visited nodes.

**begin**

```
1  Initialize sets  $\mathcal{L} \leftarrow \emptyset$  and  $\mathcal{V} \leftarrow \emptyset$ .
   for  $s \in S$  do
       if  $F_s \cap F_x \neq \emptyset$  then
            $\mathcal{L} \leftarrow \mathcal{L} \cup \{s\}$ 
   while  $\mathcal{L} \setminus \mathcal{V} \neq \emptyset$  do
2       Let  $p^* \leftarrow \arg \min_{p \in \mathcal{L} \setminus \mathcal{V}} \|x_p - x_q\|$ 
3        $\mathcal{V} \leftarrow \mathcal{V} \cup \{p^*\}$ 
4       Let  $N'_{\text{out}}(p^*) \leftarrow \{p' \in N_{\text{out}}(p^*) : F_{p'} \cap F_q \neq \emptyset, p' \notin \mathcal{V}\}$ 
5        $\mathcal{L} \leftarrow \mathcal{L} \cup N'_{\text{out}}(p^*)$ 
       if  $|\mathcal{L}| > L$  then
6           Update  $\mathcal{L}$  with the closest  $L$  nodes to  $x_q$ .
   return  $[k \text{ NNs from } \mathcal{L}; \mathcal{V}]$ 
```

---

**FilteredGreedySearch** takes a query point  $q$  and query attribute  $f$ , a constant  $L$  and start points  $s$ . It first adds all the start points that contain  $f$  to the candidate set. While the candidate set has unvisited points:

1. Identify the closest unvisited point  $p^*$  to  $q$  in the candidate set.
2. Add all neighbours of  $p^*$  that contain  $f$  to the candidate set.
3. Truncate the candidate set by keeping the  $L$  closest points to  $q$ .

# Filtered Vector Search

## Deep Dive on [Filtered DiskANN](#) ([Gollapudi WWW '23](#))

| Dataset | Dim | # Pts.    | # Queries | Source Data | Filters | Filters per Pt. | Unique Filters | 100pc. | 75pc.                 | 50pc.                 | 25pc.                 | 1pc.                 |
|---------|-----|-----------|-----------|-------------|---------|-----------------|----------------|--------|-----------------------|-----------------------|-----------------------|----------------------|
| Turing  | 100 | 2,599,968 | 996       | Text        | Natural | 1.09            | 3070           | 0.127  | $1.56 \times 10^{-4}$ | $4.15 \times 10^{-5}$ | $1.54 \times 10^{-5}$ | $7.7 \times 10^{-6}$ |
| Prep    | 64  | 1,000,000 | 10000     | Text        | Natural | 8.84            | 47             | 0.425  | 0.136                 | 0.130                 | 0.127                 | 0.09                 |
| DANN    | 64  | 3,305,317 | 32926     | Text        | Natural | 3.91            | 47             | 0.735  | 0.361                 | 0.183                 | 0.167                 | 0.150                |
| SIFT    | 128 | 1,000,000 | 10000     | Image       | Random  | 1               | 12             | 0.083  | 0.083                 | 0.083                 | 0.083                 | 0.082                |
| GIST    | 960 | 1,000,000 | 1000      | Image       | Random  | 1               | 12             | 0.083  | 0.083                 | 0.083                 | 0.083                 | 0.082                |
| msong   | 420 | 992,272   | 200       | Audio       | Random  | 1               | 12             | 0.083  | 0.082                 | 0.082                 | 0.082                 | 0.082                |
| audio   | 192 | 53,387    | 200       | Audio       | Random  | 1               | 12             | 0.085  | 0.084                 | 0.083                 | 0.082                 | 0.081                |
| paper   | 200 | 2,029,997 | 10000     | Text        | Random  | 1               | 12             | 0.083  | 0.083                 | 0.083                 | 0.083                 | 0.082                |

Table 1: Datasets used in the evaluation and their statistics. Top 3 rows are real-world datasets; the rest are semi-synthetic.

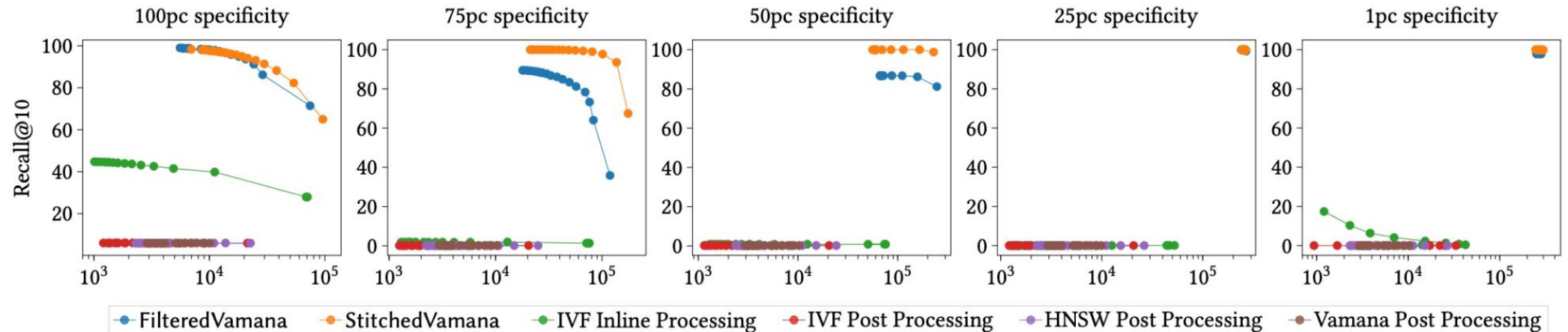


Figure 1: Turing dataset: QPS (x-axis) vs recall@10 for various algorithms with filters of 100, 75, 50, 25 and 1 percentile specificity.

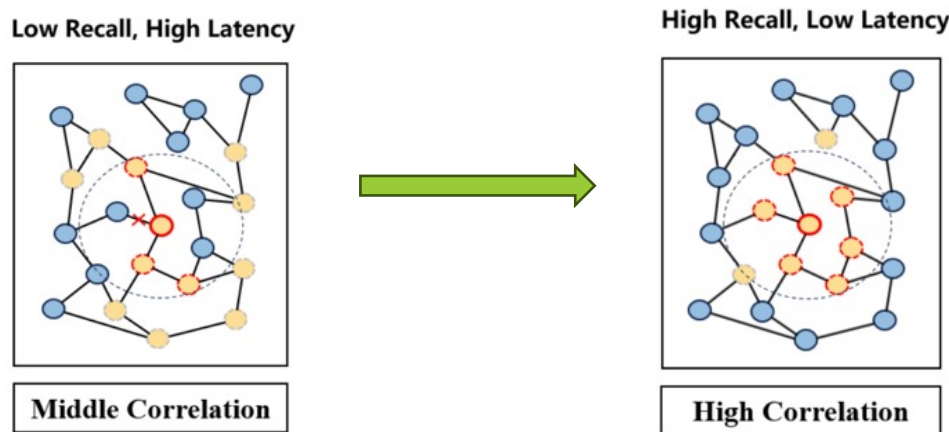
# Filtered Vector Search

## NHQ ( index-aware )

■ **核心思想：将属性深度融入图索引，实现向量搜索与属性过滤的“原生同步”处理。**

- a) 属性感知联合剪枝：将过滤规则嵌入搜索算法内部，消除传统先/后过滤带来的巨大性能损耗。
- b) 高效导航图 (NPG)：优化图拓扑选边与搜索路由机制，极大提升了属性约束下的跳转效率与检索精度。

通过距离融合拉近搜索空间中相同属性节点的距离。

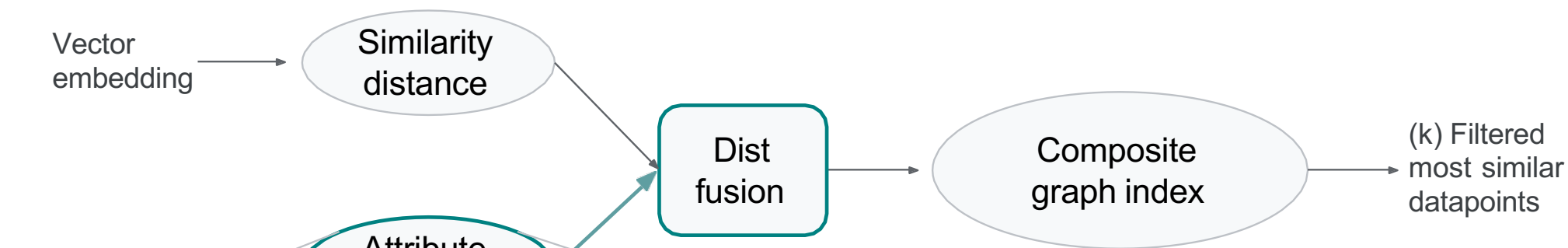


An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint [NeurIPS 2023]



# Filtered Vector Search

## Deep Dive on [NHQ](#) (Wang NIPS'23)



$$\chi(l(e_i), l(e_j)) = \sum_{k=0}^{m-1} \phi(l(e_i)^k, l(e_j)^k),$$

where  $\phi(l(e_i)^k, l(e_j)^k)$  is

$$\phi(l(e_i)^k, l(e_j)^k) = \begin{cases} 0 & l(e_i)^k = l(e_j)^k \\ 1 & l(e_i)^k \neq l(e_j)^k \end{cases}$$

Here  $k$  refers to each filter values ( $m$  is max # of filter values).

The more filter values in common two edges  $e_i, e_j$ , have, the smaller the distance  $X$ .

# Filtered Vector Search

**Joint pruning.** Given a query object  $q$ , we use a composite index  $G$  and a seed set  $P$  (usually chosen randomly from  $V$  [52]) to find the approximate top- $k$  objects based on Algorithm 2. We follow these steps (see Fig. 2, top right): ① *Initialization*. We use a visited vertex set  $C$  to store the vertices for search expansion and a result set  $R$  of size  $k$  to store the current query results. We set both sets to  $P$  at first. ② *Search expansion*. We take out the vertex  $u_i$  with the smallest  $\Gamma(q, u_i)$  from  $C$  as the next visited vertex, and then add  $N(u_i)$  to  $C$ . ③ *Query results update*. We update  $R$  with the better vertices in  $N(u_i)$ . For any vertex  $u_j \in N(u_i)$  and a vertex  $u_r \in R$  that is the farthest from  $q$ , we replace  $u_r$  with  $u_j$  in  $R$  if  $\Gamma(q, u_j) < \Gamma(q, u_r)$ . This means  $u_j$  is more similar to  $q$  in both feature vectors and attributes than  $u_r$ . We repeat ② and ③ until  $R$  does not change, then we return  $R$  as the approximate top- $k$  objects.

---

**Algorithm 2:** Joint Pruning ( $G, q, P$ )

---

**Input:** Composite index  $G = (V, E)$ ,  
query object  $q$ , seed set  $P$

**Output:** Result set  $R$

candidate set  $C \leftarrow P$ ; result set  $R \leftarrow P$

**while**  $R$  is updated **do**

$u_i \leftarrow \arg \min_{u_i \in C} \Gamma(q, u_i)$ ;

$C = C \setminus \{u_i\}$

$N(u_i) \leftarrow$  the neighbors of  $u_i$ ;

$C = C \cup N(u_i)$

**forall**  $u_j \in N(u_i)$  **do**

$u_r \leftarrow \arg \max_{u_r \in R} \Gamma(q, u_r)$

**if**  $\Gamma(q, u_j) < \Gamma(q, u_r)$  **then**

$R = R \setminus \{u_r\}$ ;

$R = R \cup \{u_j\}$

**return**  $R$

---

# Filtered Vector Search

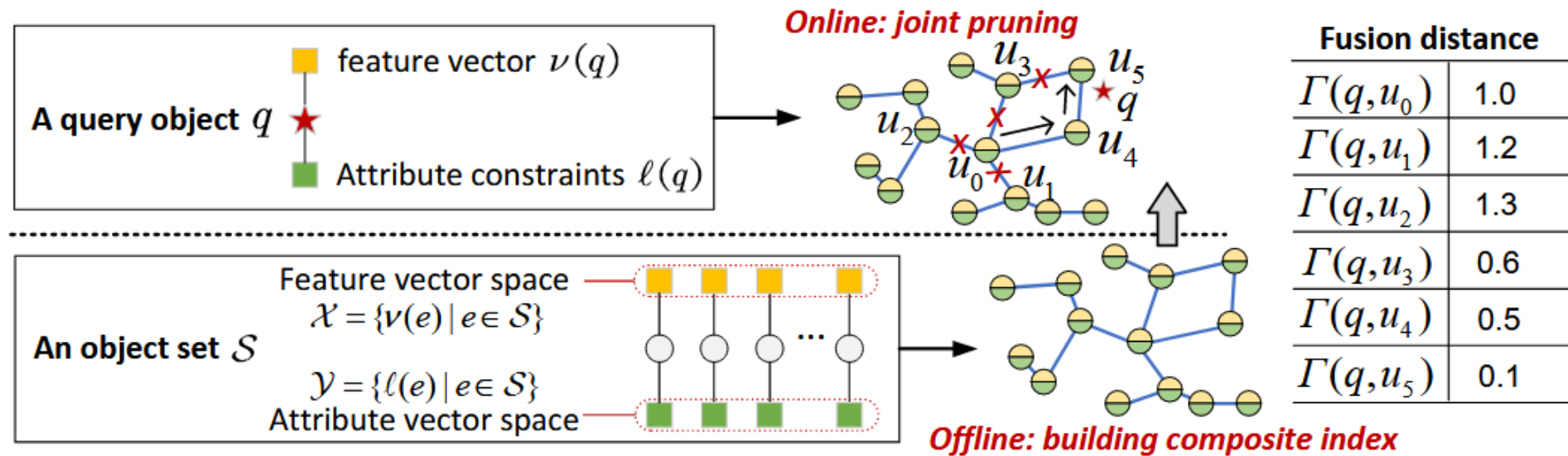


Figure 2: NHQ framework overview.

We show an example of joint pruning for finding the nearest object (i.e., top-1) to the query object  $q$  in Fig. 2. We start with a random vertex as the seed, i.e.,  $P = \{u_0\}$  and set  $C = \{u_0\}$  and  $R = \{u_0\}$ . Then, we expand the search space with  $u_0$ 's neighbors and get  $C = \{u_1, u_2, u_3, u_4\}$ . We update  $R$  with  $u_4$  because  $\Gamma(q, u_4) = 0.5$  is smaller than  $\Gamma(q, u_0) = 1.0$ . We continue to expand the search space with  $u_4$ 's and  $u_5$ 's neighbors and get  $R = \{u_5\}$ . Since no vertex  $u_j \in N(u_5)$  has a smaller distance than  $\Gamma(q, u_5)$ , we return  $u_5$  as the approximate top-1 nearest object. We prune the vertices  $u_1, u_2$ , and  $u_3$  from the search space.



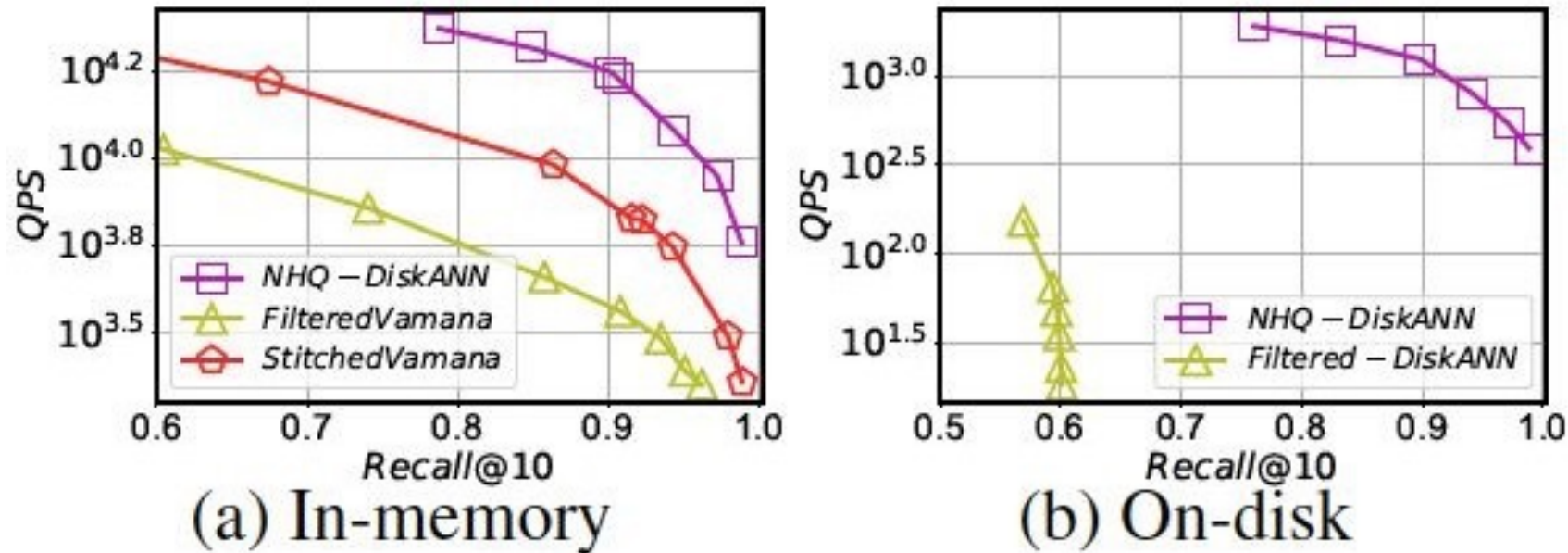
# Filtered Vector Search

Lemma 1. The composite index and the ordinary PG have the same index size for an object set  $S$ .

Theorem 2. Given a query object  $q$ , the composite index guarantees at least the same  $\text{Recall}@k$  of HQ as the ordinary PG.

# Filtered Vector Search

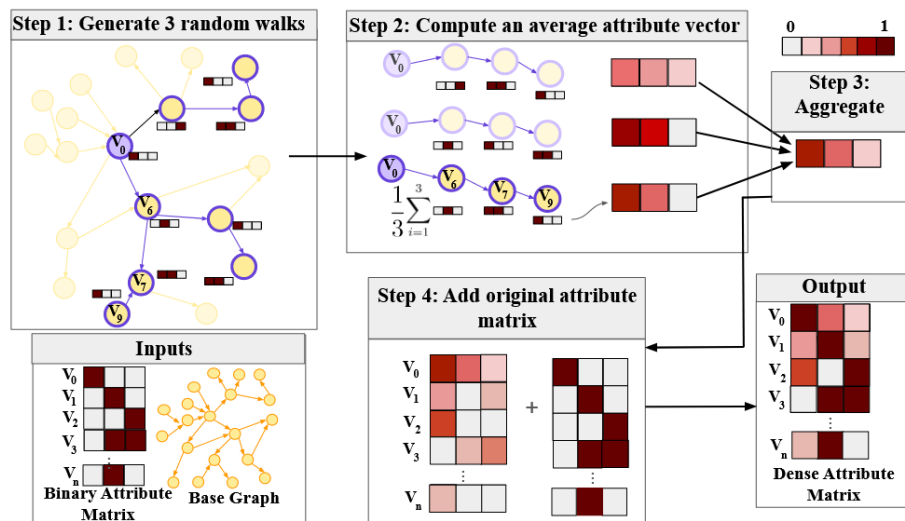
## NHQ vs. FilteredDiskANN



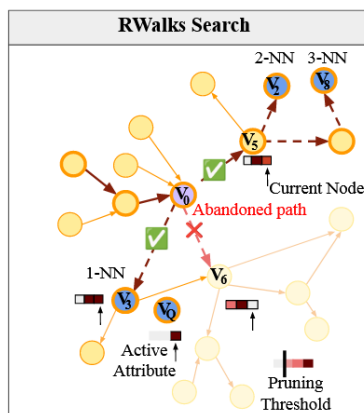
NHQ-DiskANN >> Filtered-DiskANN (in-memory or disk)

Note: **Equality filter conditions for non-intersecting sets** may benefit from Stitched/FilteredVamana over NHQ.

# Filtered Vector Search



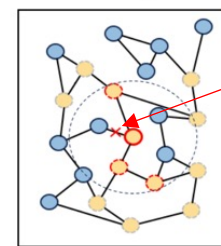
(a) RWalks attribute enrichment workflow



(b) RWalks search with pruning

Fig. 2. RWalks workflow

Low Recall, High Latency



Middle Correlation

在搜索时基于混合距离判断是否遍历下一个节点。

## Rwalks ( search-aware )

### ■ 核心思想：随机游走学习属性分布，用混合距离引导遍历

- a) 随机游走特征扩散：通过多轮定长游走聚合邻域属性，使每个节点具备周边属性分布的信息。
- b) 启发式导航剪枝：基于属性分布预测动态筛选路径，协同优化约束满足率与搜索跳转效率。

## Background

## Filtering Strategy

- Pre-filtering
- Post-filtering
- In-filtering

## Equality-Oriented Vector Index

- Filtered DiskANN
- NHQ

## Predicate-Agnostic Vector Index

- ACORN
- NaviX

## Range-Oriented Vector Index

- SeRF
- DIGRA

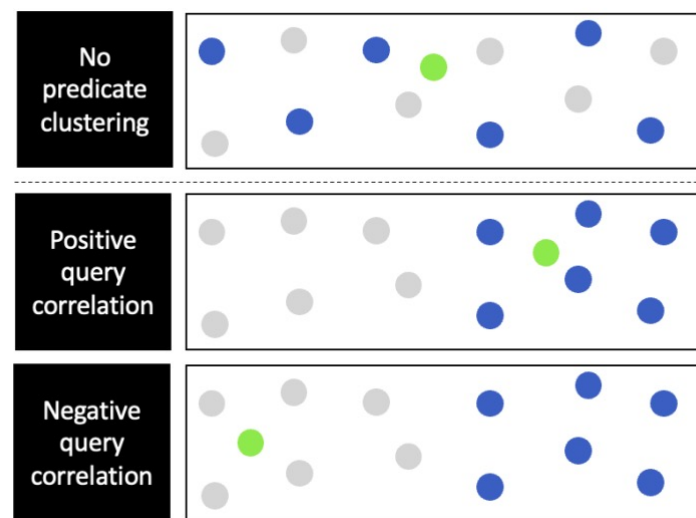


# Filtered Vector Search

## ACORN ( Index-aware )

■ **核心思想**: 在单一的、全局的HNSW索引上, 通过“谓词子图”模拟Oracle Partition Index的行为。构建**密集HNSW避免过滤导致图不可用**

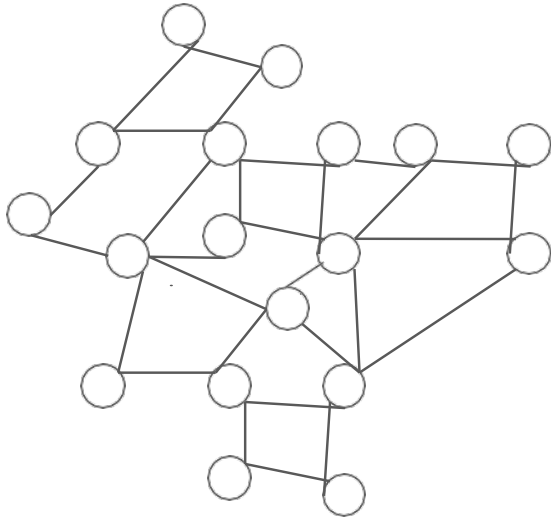
- a) 多倍邻居冗余扩展: 扩展邻居节点( $M \rightarrow M \cdot \gamma, \gamma > 1$ ), 同时结合两跳覆盖剪枝, 利用冗余连接避免属性过滤造成的路径断裂。
- b) 两跳补偿搜索策略: 在底层搜索中引入  $M\beta$  预取与两跳路径搜索, 确保在稀疏条件下依然能维持高召回率。



# Filtered Vector Search

## Deep Dive on [ACORN](#) (Patel SIGMOD'24)

Reuse unfiltered indexes

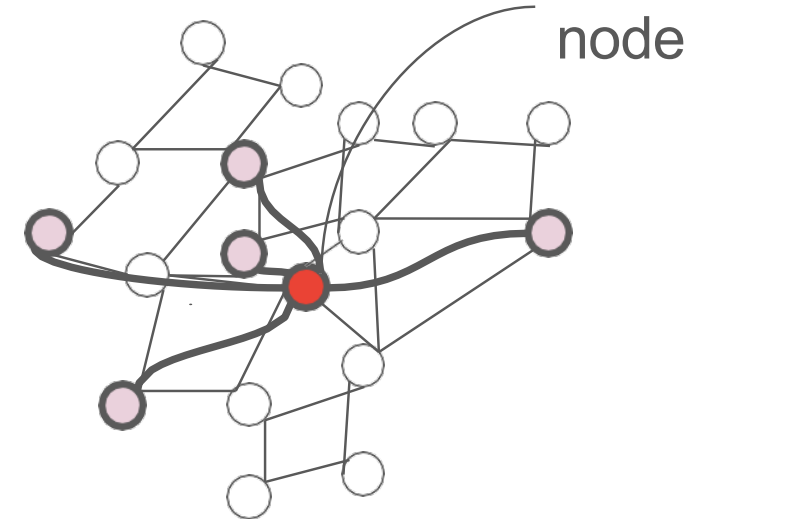


Use inline filtering to discover filter-passing nodes



Add edges to alleviate connectivity issue

Then, **light up the right neighborhood** at search time!



Densified Predicate traversal



# Filtered Vector Search

Constructing the ACORN- $\gamma$  index introduces a new parameter:  $\gamma$ : neighbour expansion factor

The construction is the same as for the HNSW algorithm except for **two modifications**:

- 1、Candidate sets are of size  $M \cdot \gamma$  and are created by searching the index using the 2-hop neighbour selection strategy
- 2、Pruning works as follows: keep the first  $M\beta$  candidates in the sorted candidates list. For the remaining points we prune them if they occur in the temporary list  $H$ . If they are not in  $H$  we keep them and add their neighbours to  $H$ . Once we run out of candidates or  $|H|$  plus the surviving points exceeds  $M \cdot \gamma$  we stop and discard  $H$  and the remaining candidates.

# Filtered Vector Search

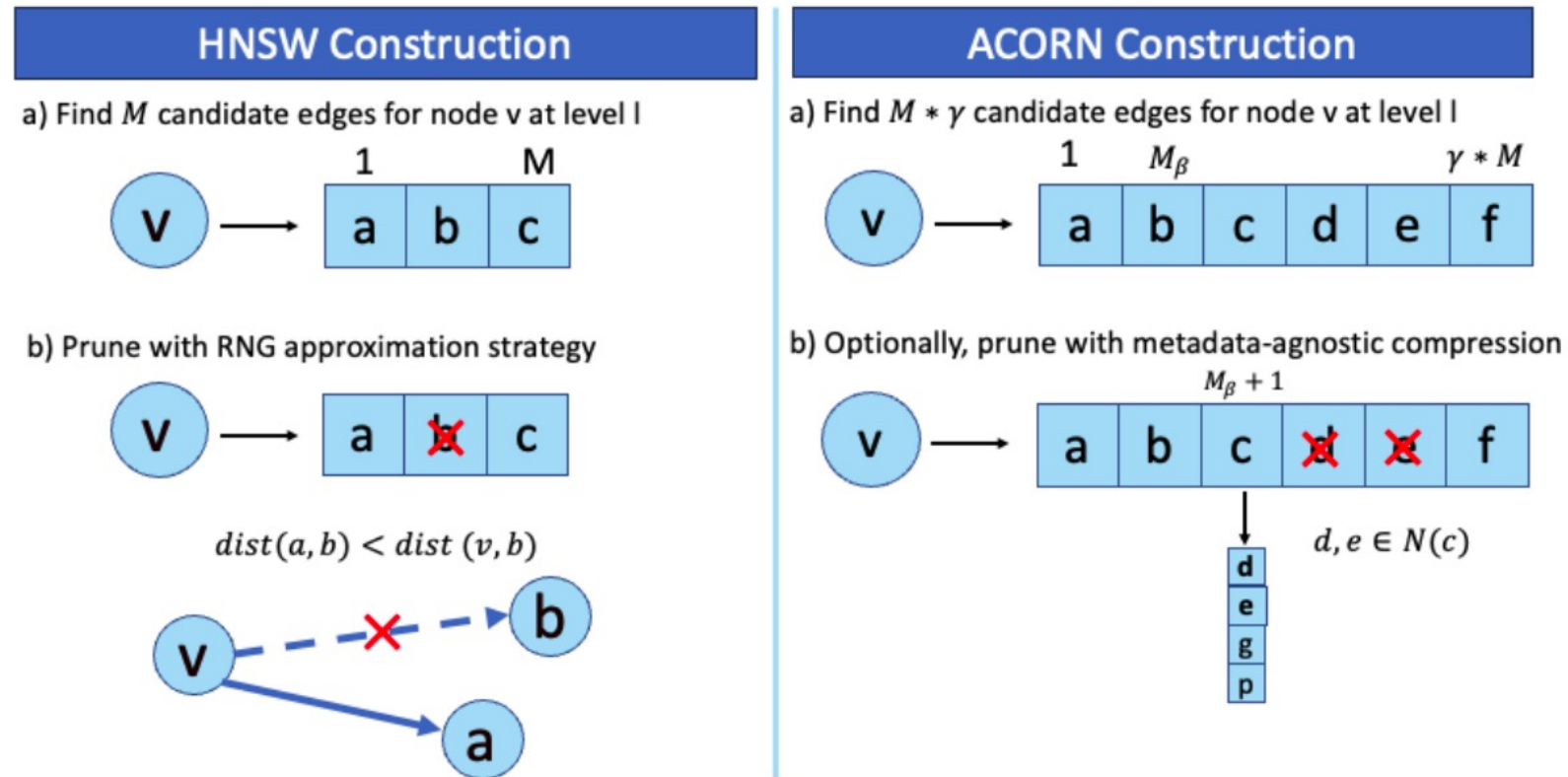


Fig. 5. A comparison of HNSW and ACORN- $\gamma$ 's strategies for (a) selecting candidate edges, shown for  $M=3$ , and (b) pruning candidate edges for each inserted node  $v$ , shown for  $M=3$ ,  $M_\beta=2$ ,  $\gamma=2$ .

# Filtered Vector Search

---

**Algorithm 2:** ACORN-SEARCH-LAYER( $x_q, p_q, e, ef, l$ )

---

**Input:** query vector  $x_q$ , query predicate  $p_q$ , entry-point  $e$ , number of nearest neighbors to return  $ef$ , level to search  $l$

**Output:**  $ef$  nearest elements to  $x_q$

```
1  $T \leftarrow e$  // visited set
2  $C \leftarrow e$  // candidate set
3  $W \leftarrow e$  // dynamic list of found nearest neighbors
4 while  $|C| > 0$  do
5    $c \leftarrow \text{extract arg min}_{x \in C} \|x_q - x\|$ 
6    $f \leftarrow \text{get arg max}_{x \in W} \|x_q - x\|$ 
7   if  $\text{dist}(c, x_q) > \text{dist}(f, x_q)$  and  $|W| \geq efc$ 
8     break
9    $\text{neighborhood} \leftarrow \text{GET-NEIGHBORS}(c, l, p_q)$ 
10  for each  $v \in \text{neighborhood}[1:M]$ 
11    if  $v \notin T$ 
12       $T \leftarrow T \cup v$ 
13       $f \leftarrow \text{arg max}_{x \in W} \|x_q - x\|$ 
14      if  $\text{dist}(v, x_q) < \text{dist}(f, x_q)$  or  $|W| < ef$ 
15         $C \leftarrow C \cup v$ 
16         $W \leftarrow W \cup v$ 
17        if  $|W| > ef$ 
18          remove furthest element from  $W$  to  $x_q$ 
19
20 end
21 return  $W$ 
```

---

$M$ : HNSW max degree,  $M_\beta$ : compression factor

1. **Filter:** Scan the whole neighbour list and keep only the first  $M$  points passing the filter.
2. **Compression-based Heuristic:** Filter the first  $M_\beta$  points. Expand the rest to include their neighbours and apply the filter on this extended list. Of all points passing the filter, we keep the closest  $M$ .
3. **Neighbour Expansion** Expand the whole neighbour list to also include their neighbours. Filter all the points in the extended list and only keep the closest  $M$ .

# Filtered Vector Search

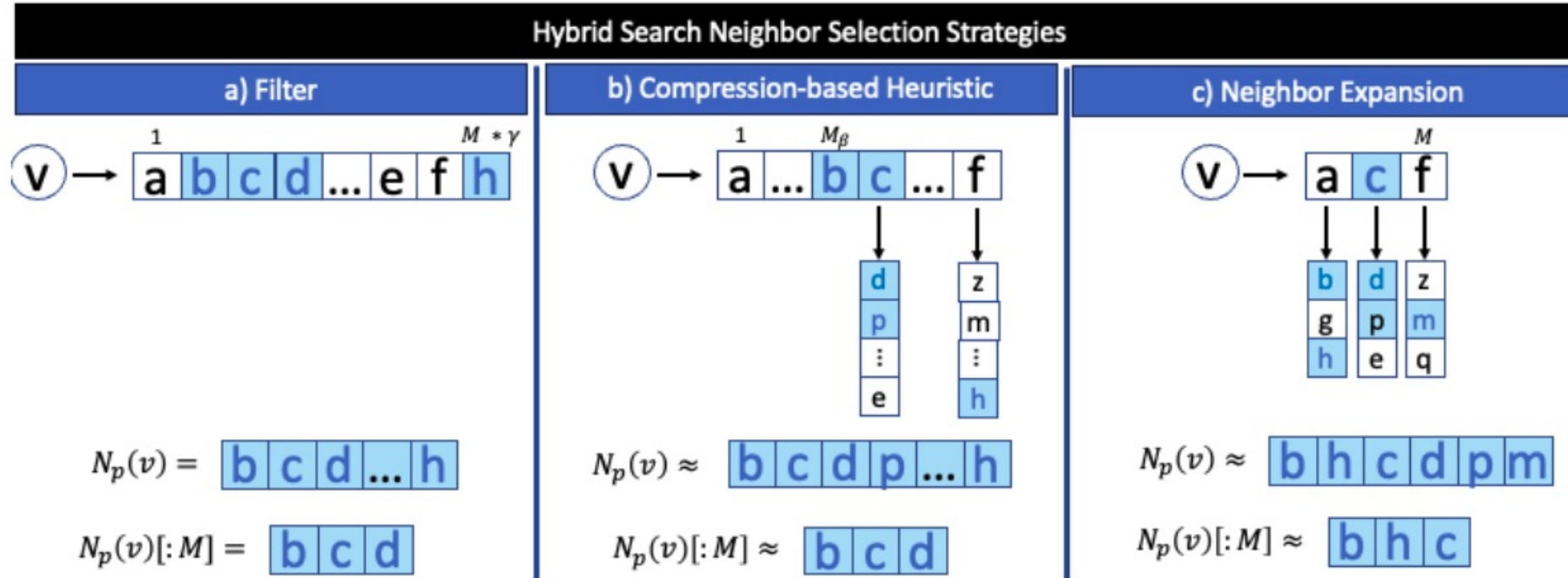


Fig. 4. Diagram of ACORN's neighbor selection strategies. Blue nodes represent neighbors that pass the query predicate. Sub-figure (a) shows the simple predicate-based filter applied to uncompressed edge lists of size  $M \cdot \gamma$ , followed by truncation to size  $M = 3$ . Sub-figure (b) shows the compression-based heuristic. Sub-figure (c) shows the neighbor expansion strategy used in ACORN-1.



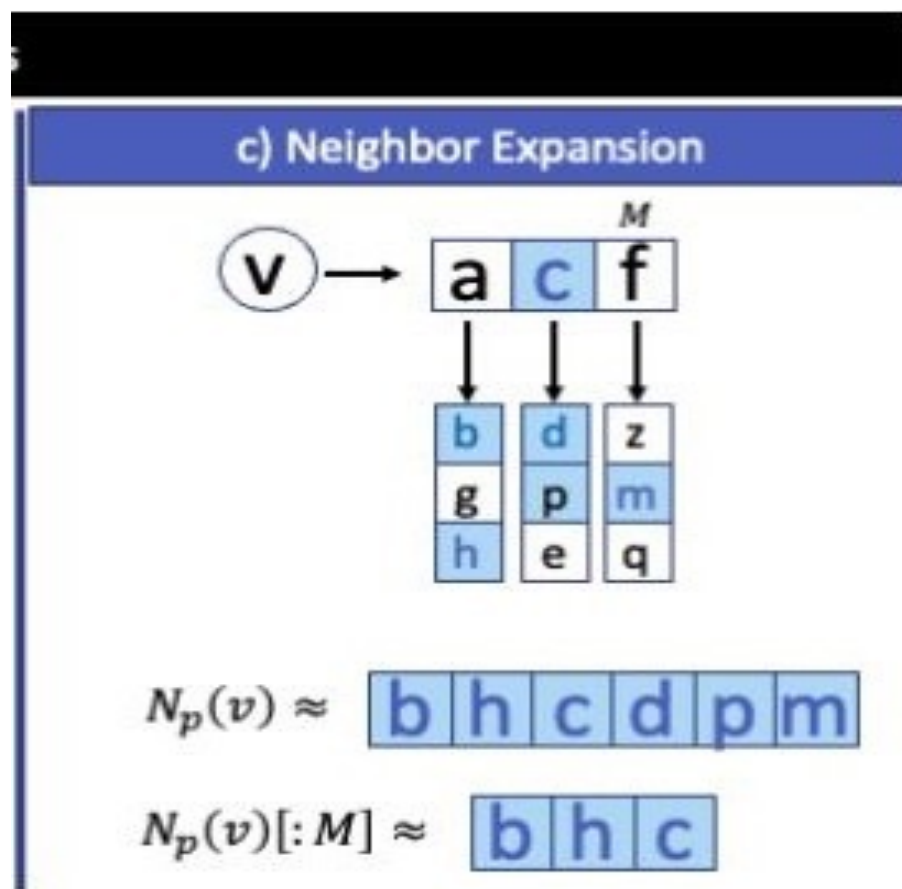
# Filtered Vector Search

## Deep Dive on ACORN (Patel SIGMOD'24)

Two variants:

- 1) ACORN-1: faster to build
- 2) ACORN- $\gamma$ : faster to search

| 变体              | $\gamma$ 值 | 特点            | 适用场景        |
|-----------------|------------|---------------|-------------|
| ACORN- $\gamma$ | $>1$       | 构建时扩展邻居，搜索时高效 | 追求极致查询性能    |
| ACORN-1         | 1          | 搜索时动态扩展邻居，构建快 | 资源受限或构建时间敏感 |



# Filtered Vector Search

## Construction Complexity

For fixed parameters  $M$ ,  $M\beta$  and  $\text{efc}$ , ACORN- $\gamma$ 's overall expected construction complexity scales

$O(n \cdot \gamma \cdot \log(n) \cdot \log(\gamma))$ . Compared to HNSW, which has  $O(n \cdot \log(n))$  expected construction complexity

## Search Complexity

ACORN- $\gamma$ 's expected search complexity scales:  $O((d + \gamma) \cdot \log(s \cdot n) + \log(1/s))$



# Filtered Vector Search

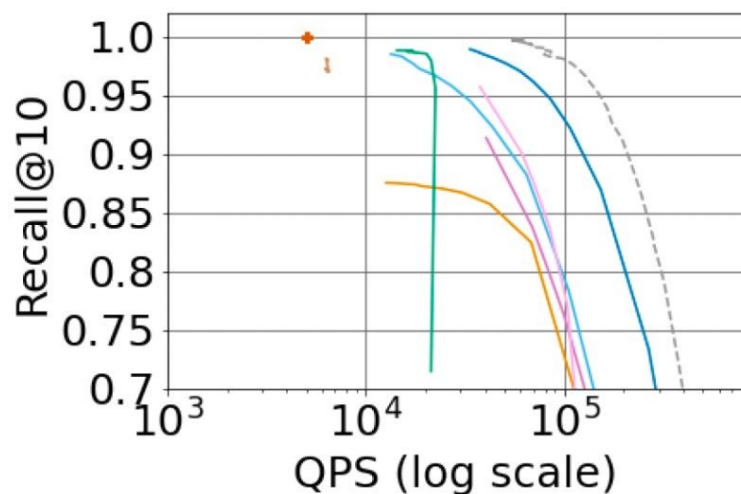
Experimental setup of [ACORN](#) (Patel SIGMOD'24)

**Table 2: Datasets**

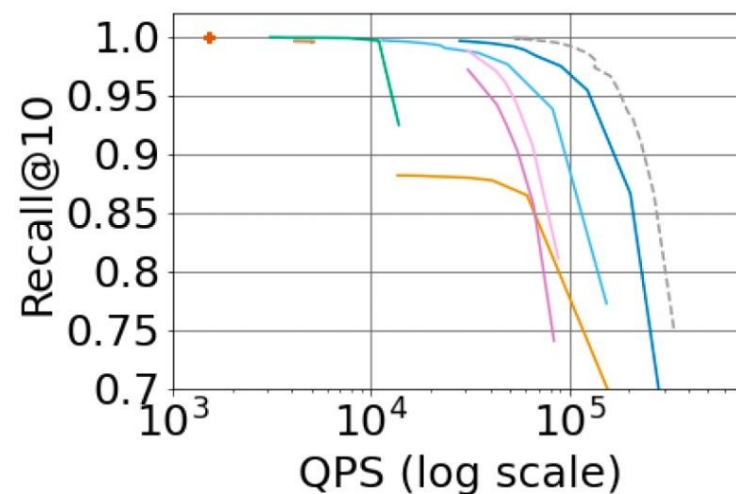
|             | Base Data  |            |                    |                                       | Query Workload                                                |                           |                       |
|-------------|------------|------------|--------------------|---------------------------------------|---------------------------------------------------------------|---------------------------|-----------------------|
|             | # Vectors  | Vector Dim | Vector Source Data | Structured Data                       | Predicate Operators                                           | Avg. Query Selectivity    | Predicate Cardinality |
| SIFT1M      | 1,000,000  | 128        | images             | random int.                           | equals( $y$ )                                                 | 0.083                     | 12                    |
| Paper       | 2,029,997  | 200        | passages           | random int.                           | equals( $y$ )                                                 | 0.083                     | 12                    |
| TripClick   | 1,055,976  | 768        | passages           | clinical area list & publication date | contains( $y_1 \vee y_2 \vee \dots$ ) & between( $y_1, y_2$ ) | 0.17, 0.36 <sup>2</sup>   | $> 10^8$              |
| LAION (1M)  | 1,000,448  | 512        | images             | text captions & keyword list          | regex-match( $y$ ) & contains( $y_1 \vee y_2 \vee \dots$ )    | 0.056 - 0.13 <sup>3</sup> | $> 10^{11}$           |
| LAION (25M) | 24,653,427 | 512        | same as above      | same as above                         | same as above                                                 | same as above             | same as above         |

# Filtered Vector Search

## Experiments of ACORN (Patel SIGMOD'24)



(a) SIFT1M Dataset

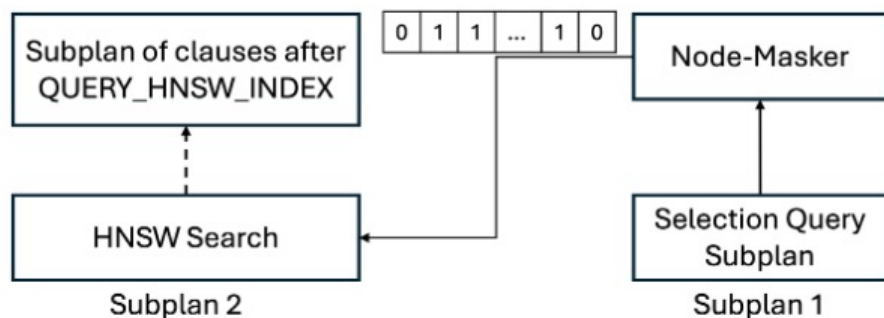


(b) Paper Dataset

ACORN outperforms both Filtered DiskANN and NHQ for a fixed recall, while maintaining generality (not specialized to a single filter value).

Oracle Partitions = Ideal Filtered HNSW (upper bound for performance)

# Filtered Vector Search



$$\sigma_l = \frac{|S(\text{neighbors of } c_{\min})|}{|\text{neighbors of } c_{\min}|}$$

selectivity: high med low  
|-----|-----|  
best: onehop-s directed blind

针对局部不同的情况动态调整搜索策略

## NaviX ( search-aware )

### ■ 核心思想：引入搜索感知的动态路由机制

- a) 多模态路径探索：根据局部属性分布，在 “一跳精确探索 (One-hop) ” 与 “二跳启发式加速 (Directed/Blind 2-hop) ” 间灵活切换。
- b) 局部选择性自适应：在搜索每一步动态计算属性过滤率，通过代价模型自动选择最优的邻居访问广度，确保存量稀疏时的搜索连通性。

## Background

## Filtering Strategy

- Pre-filtering
- Post-filtering
- In-filtering

## Equality-Oriented Vector Index

- Filtered DiskANN
- NHQ

## Predicate-Agnostic Vector Index

- ACORN
- NaviX

## Range-Oriented Vector Index

- SeRF
- DIGRA



# Filtered Vector Search

- ANNS + predicates on attributes = Multi-Modal ANNS
  - Examples:



+ price range: < \$200 =



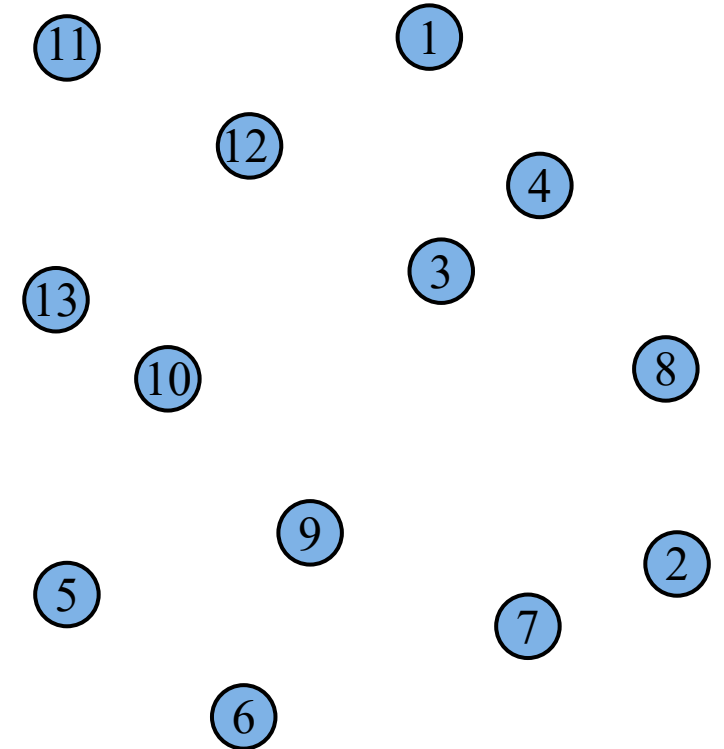
+ timestamp: 05:00:00 to 20:00:00 =



# Range-Filtering ANNS

---

- Input
  - A set of data vectors, each associated with a numerical attribute value.

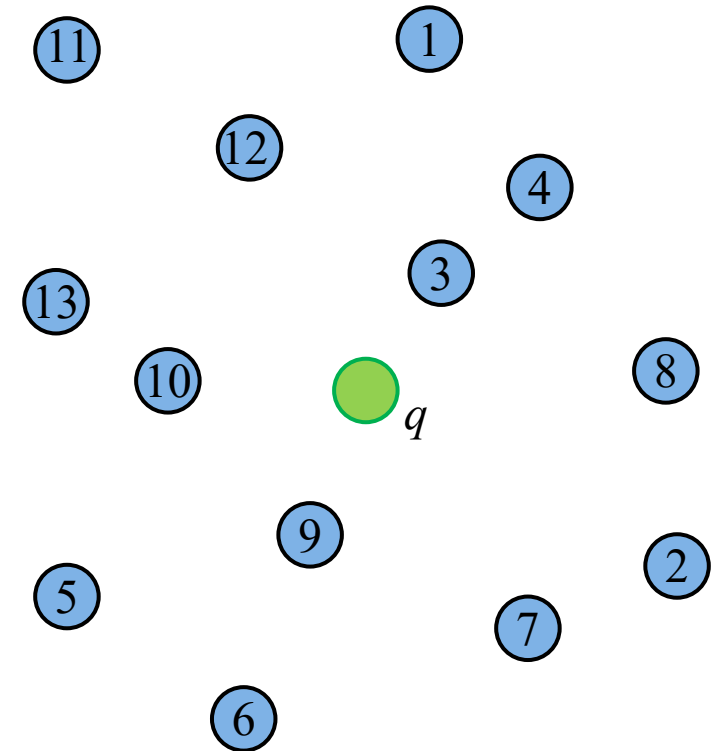




# Range-Filtering ANNS

---

- Input
  - A set of data vectors, each associated with a numerical attribute value.
- Range-filtering ANNS query  $(q, r, k)$ 
  - $q$ : query vector
  - $r$ : query range
  - $k$ : num. of results

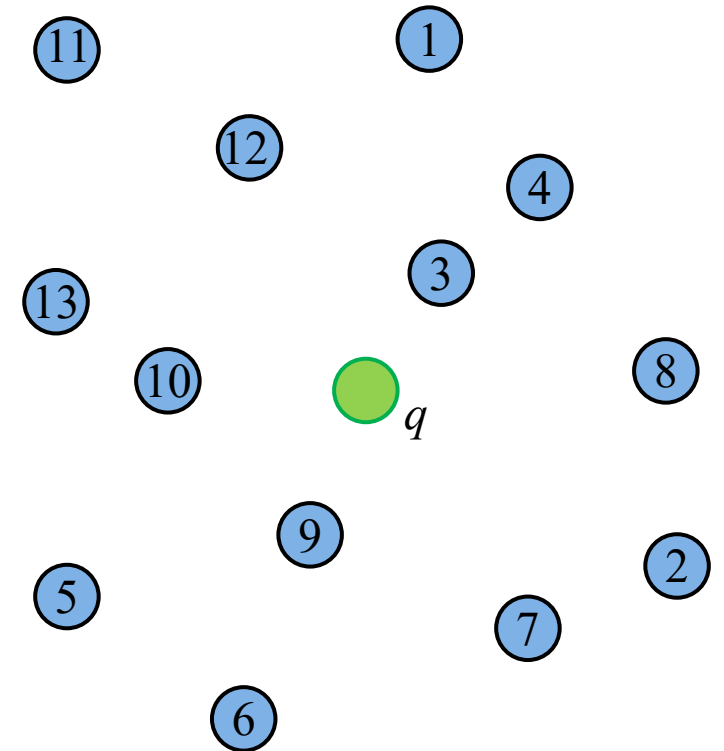


$$r = [1, 4] \quad k = 2$$

# Range-Filtering ANNS

---

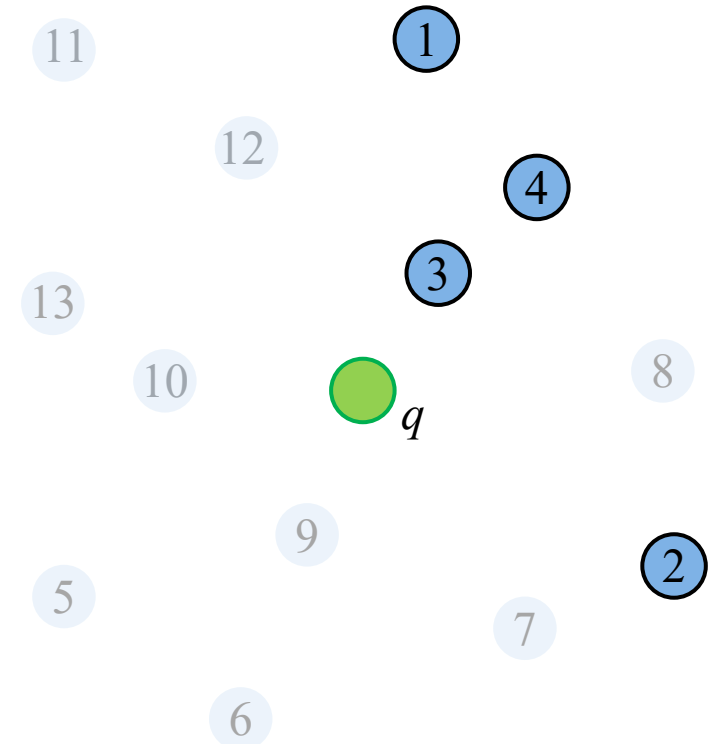
- Input
  - A set of data vectors, each associated with a numerical attribute value.
- Range-filtering ANNS query  $(q, r, k)$ 
  - $q$ : query vector
  - $r$ : query range
  - $k$ : num. of results
- Output: approximate  $k$ -nearest-neighbors of  $q$  among all the data vectors whose attribute values are within the query range  $r$



# Range-Filtering ANNS

---

- Input
  - A set of data vectors, each associated with a numerical attribute value.
- Range-filtering ANNS query  $(q, r, k)$ 
  - $q$ : query vector
  - $r$ : query range
  - $k$ : num. of results
- Output: approximate  $k$ -nearest-neighbors of  $q$  among all the data vectors whose attribute values are within the query range  $r$

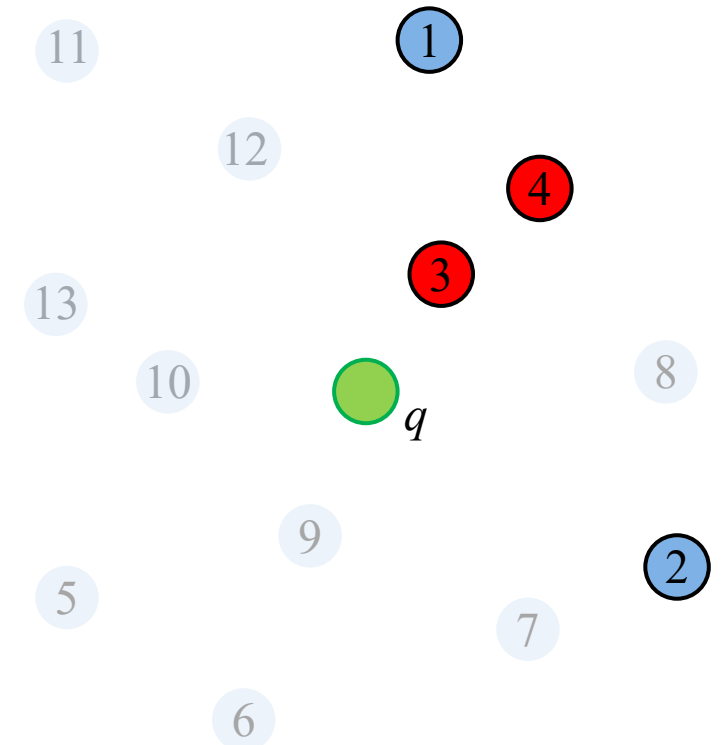


$$r = [1, 4] \quad k = 2$$

# Range-Filtering ANNS

---

- Input
  - A set of data vectors, each associated with a numerical attribute value.
- Range-filtering ANNS query  $(q, r, k)$ 
  - $q$ : query vector
  - $r$ : query range
  - $k$ : num. of results
- Output: approximate  $k$ -nearest-neighbors of  $q$  among all the data vectors whose attribute values are within the query range  $r$

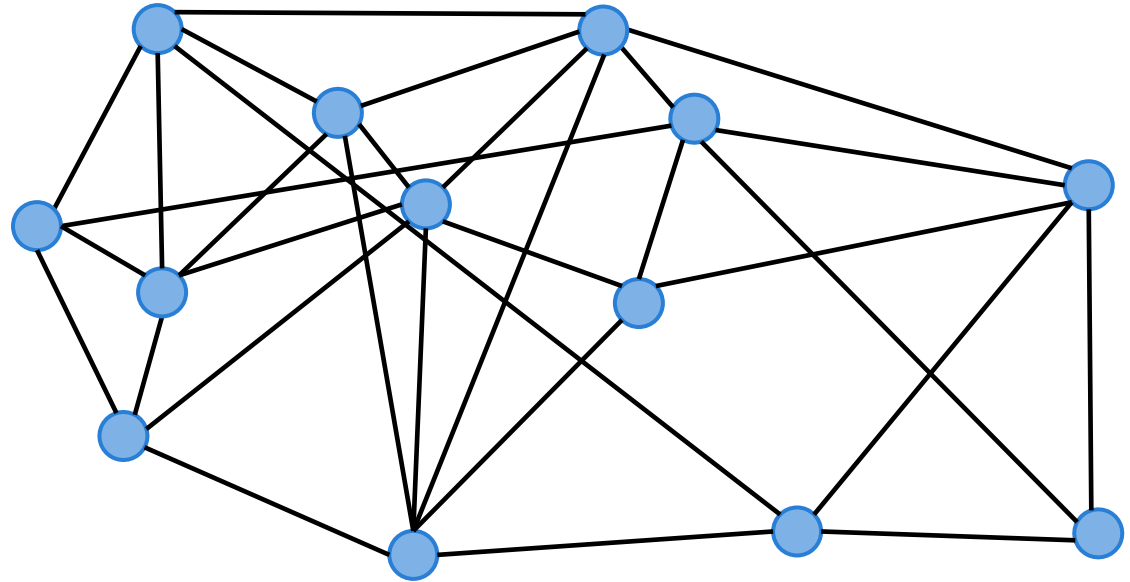


$$r = [1, 4] \quad k = 2$$

# HNSW Search: 10,000 foot view

---

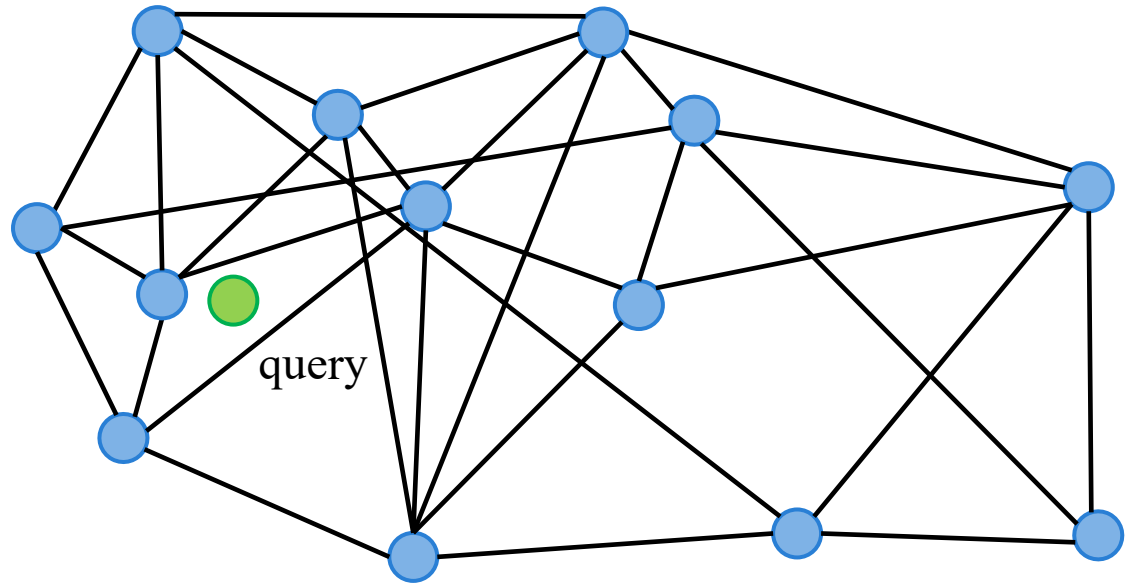
- Each node is a data vector.



# HNSW Search: 10,000 foot view

---

- Each node is a data vector.
- Query processing:
  - a query vector arrives.

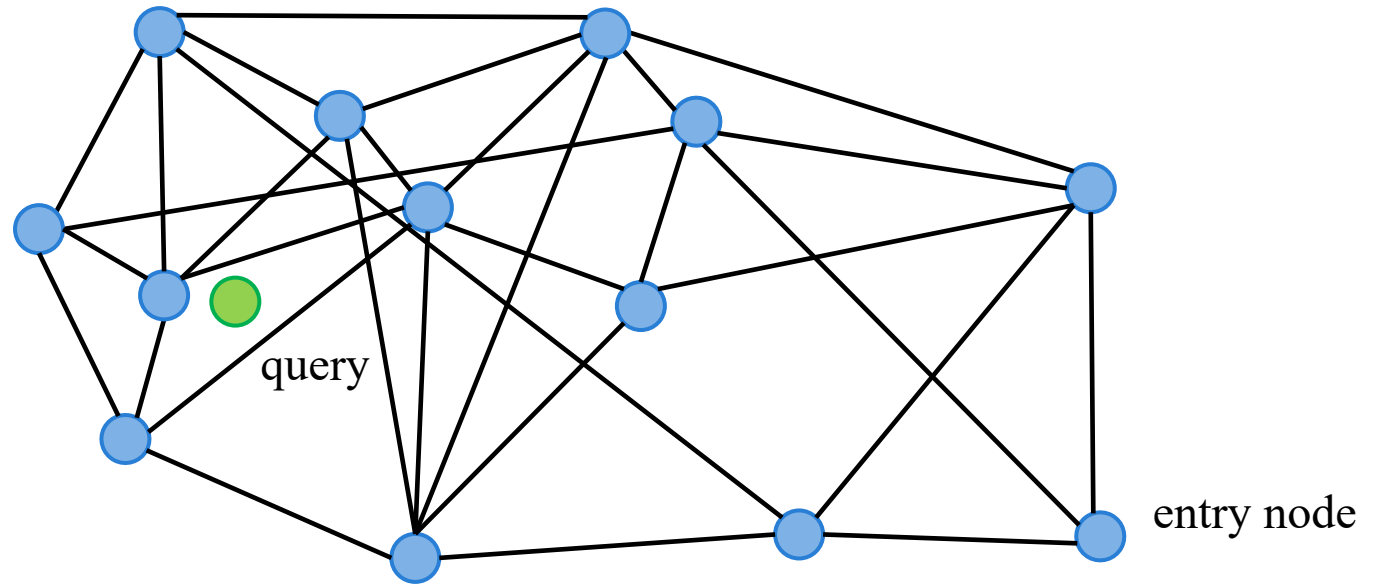




# HNSW Search: 10,000 foot view

---

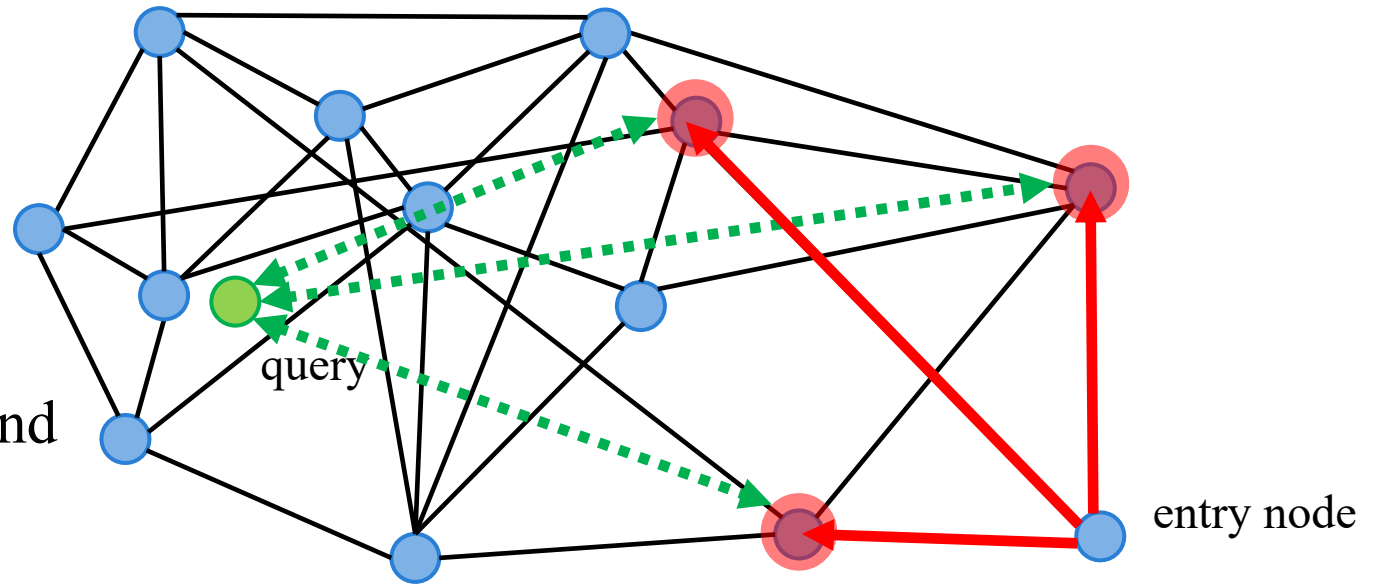
- Each node is a data vector.
- Query processing:
  - a query vector arrives.
  - start from a random entry node.



# HNSW Search: 10,000 foot view

---

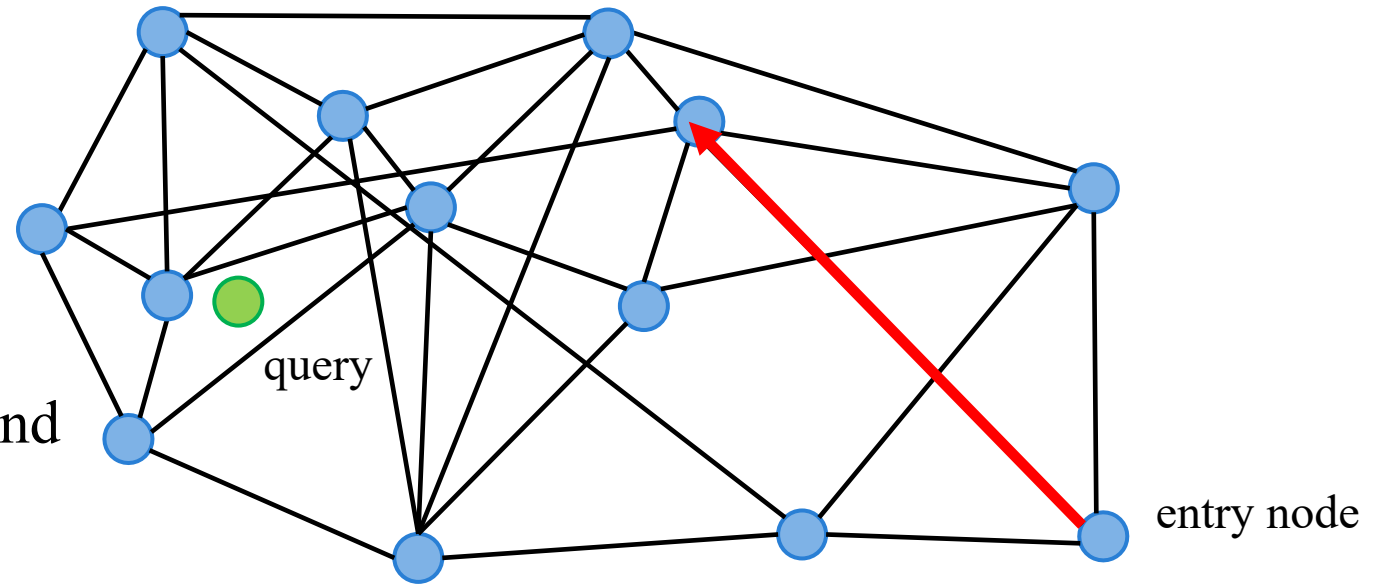
- Each node is a data vector.
- Query processing:
  - a query vector arrives.
  - start from a random entry node.
  - from the connected nodes, find the closet one to the query



# HNSW Search: 10,000 foot view

---

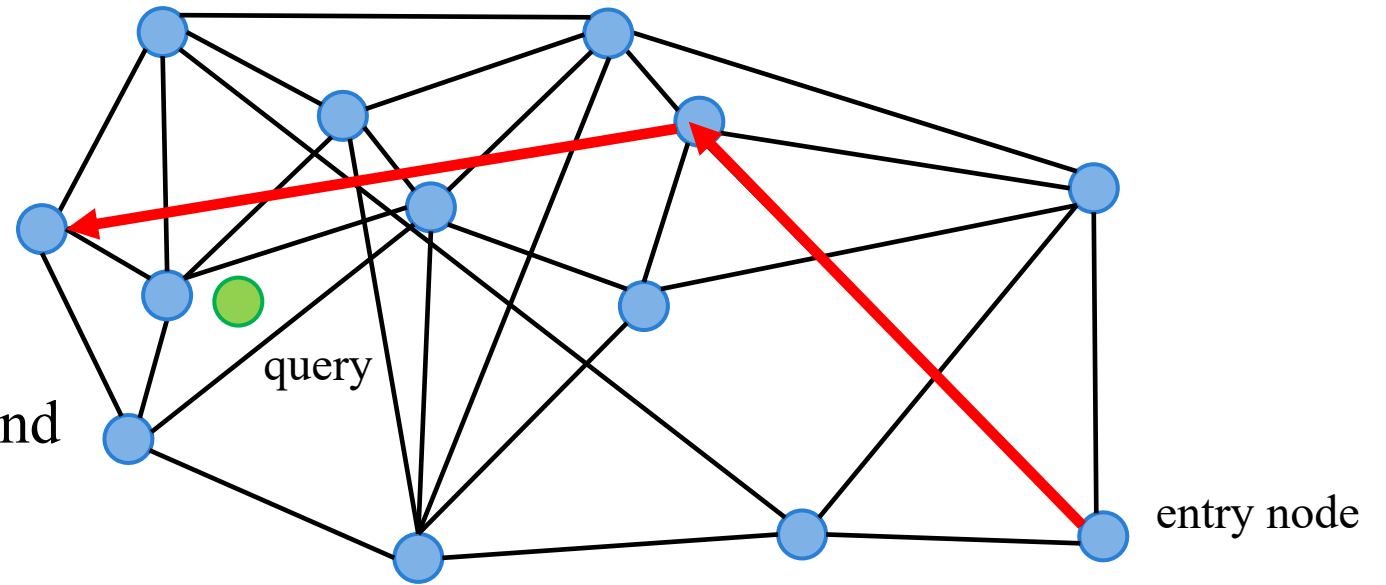
- Each node is a data vector.
- Query processing:
  - a query vector arrives.
  - start from a random entry node.
  - from the connected nodes, find the closet one to the query



# HNSW Search: 10,000 foot view

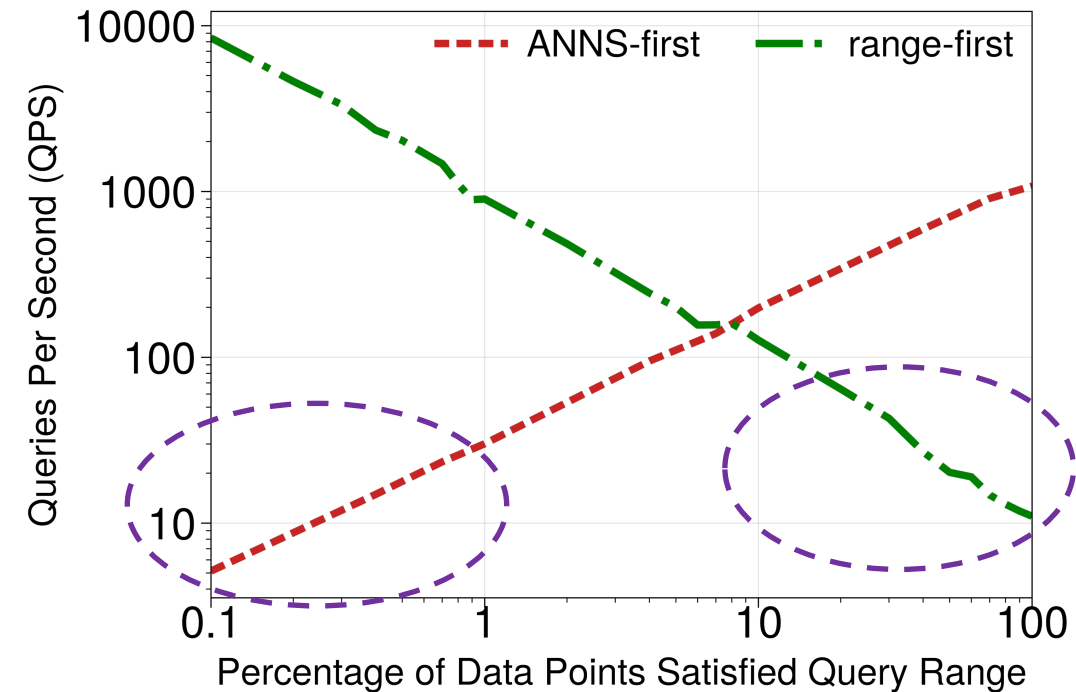
---

- Each node is a data vector.
- Query processing:
  - a query vector arrives.
  - start from a random entry node.
  - from the connected nodes, find the closet one to the query
  - traverse in a greedy manner until converges, return the KNN among visited nodes



# Limitation of Existing Methods

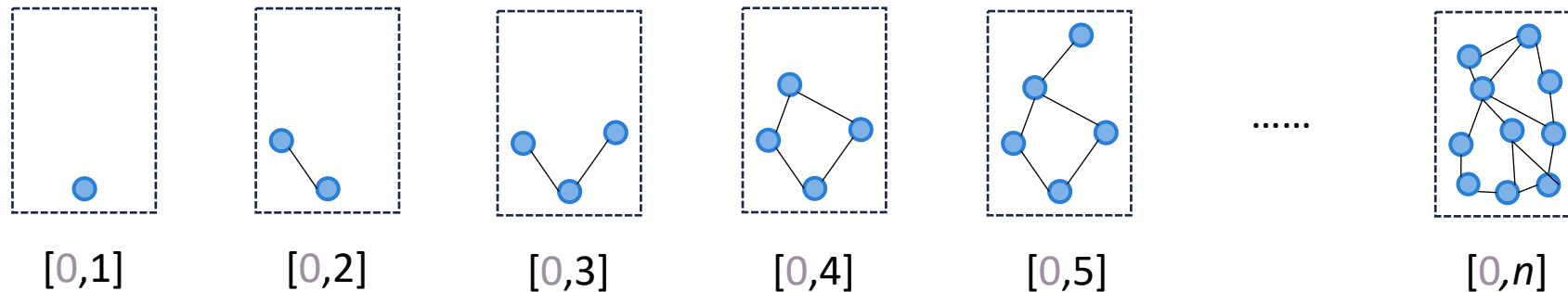
- ANNS-first (a.k.a. post-filtering):
  - perform HNSW search until  $k$  data vectors satisfying the range condition are found.
  - slow when query range is small as data vectors in the query range are rare and ANNS takes long to stop.
- range-first (a.k.a. pre-filtering):
  - scan all the data vectors in the query range
  - slow when the query range is large.



# Half-bounded Query Range

---

- Half-bounded query range:  $\leq m$  or  $[0, m]$ 
  - For simplicity, suppose attribute values are integers starting from 1 to  $n$ .
- Examples: find relevant documents *before 2019*; similar pants *less than \$50*
- Solution: build one HNSW for each possible half-bounded range  $[0, m]$



**Pros:** excellent query performance

**Cons:** high index cost → **Our idea:** one graph to encode all the  $n$  HNSW graphs losslessly

# HNSW Graph Construction: 10,000 foot view

---

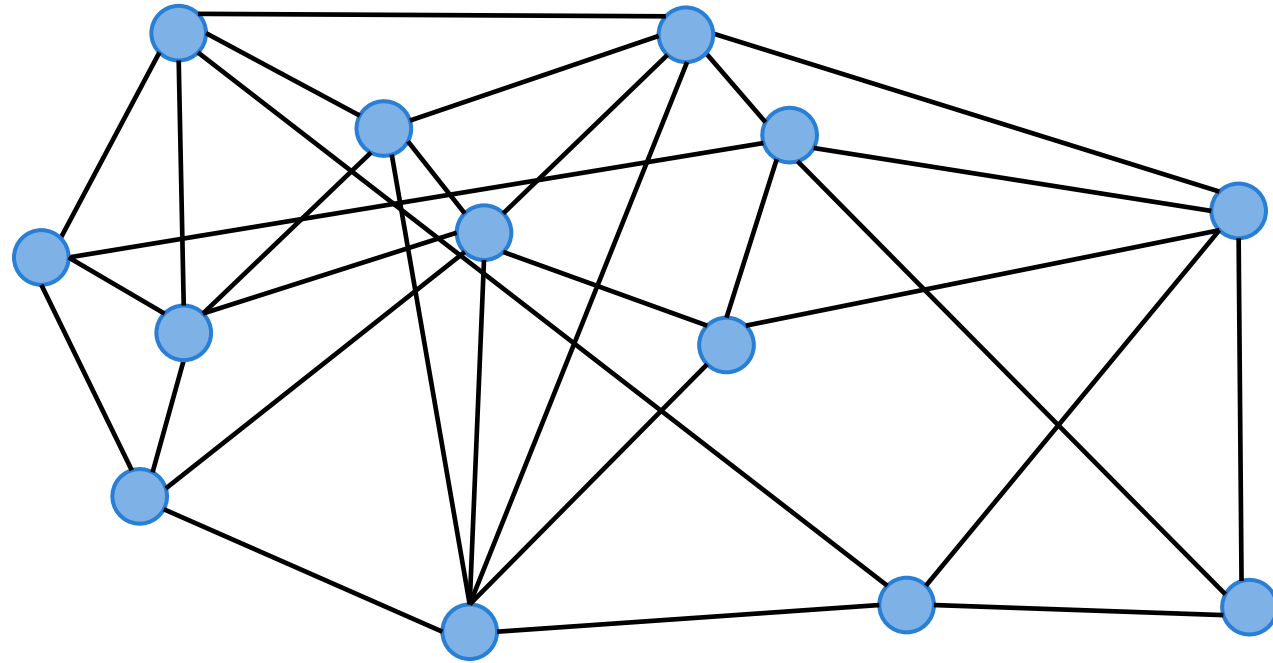
- Insert node into the graph one by one



# HNSW Graph Construction: 10,000 foot view

---

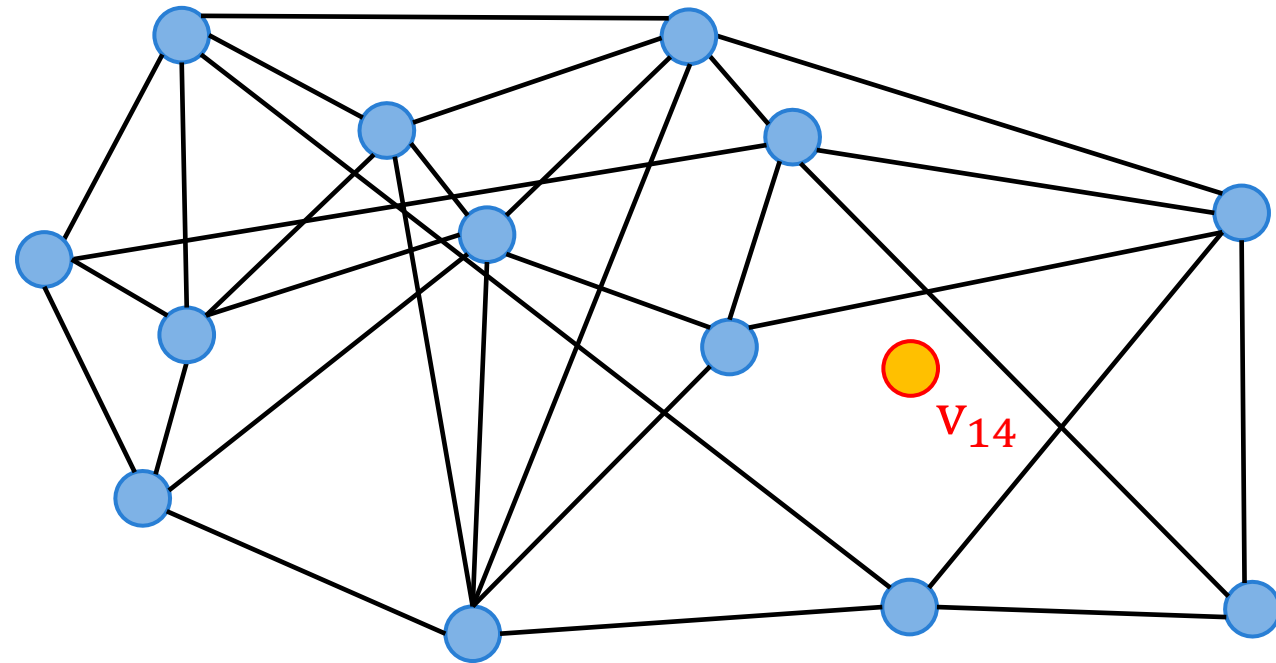
- Insert node into the graph one by one
- Example: HNSW graph of  $v_1 v_2 \dots v_{13}$



# HNSW Graph Construction: 10,000 foot view

---

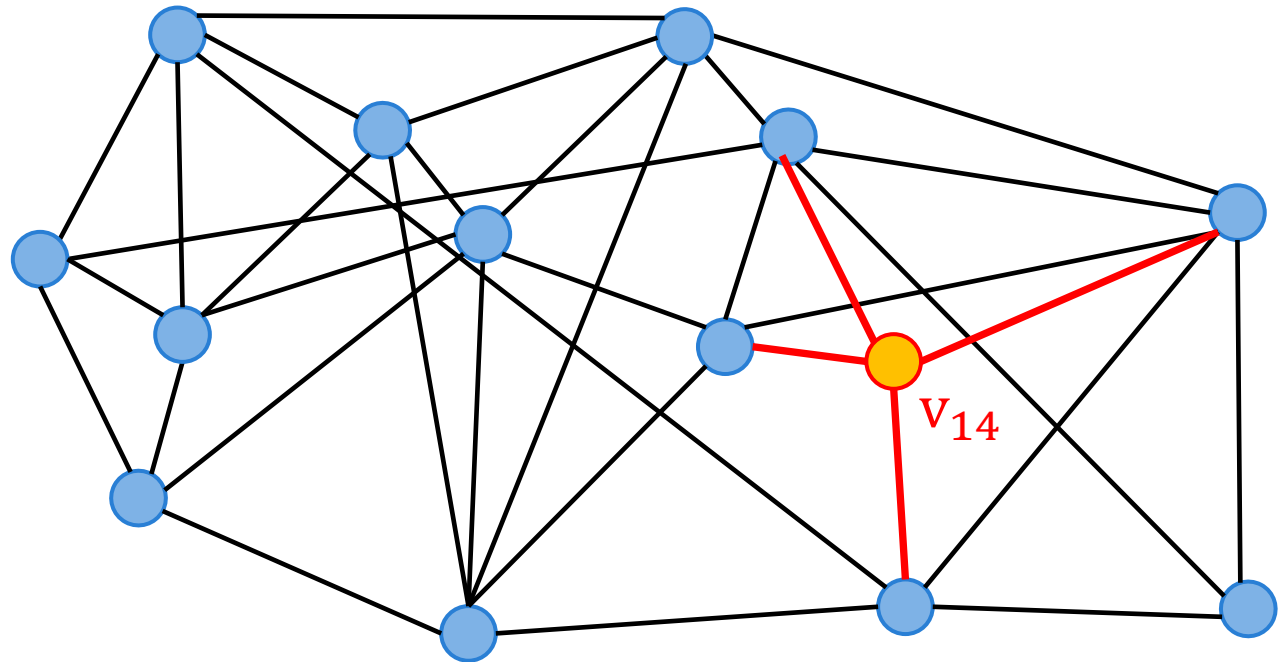
- Insert node into the graph one by one
- Example: HNSW graph of  $v_1 v_2 \dots v_{13}$
- To insert a node  $v_{14}$



# HNSW Graph Construction: 10,000 foot view

---

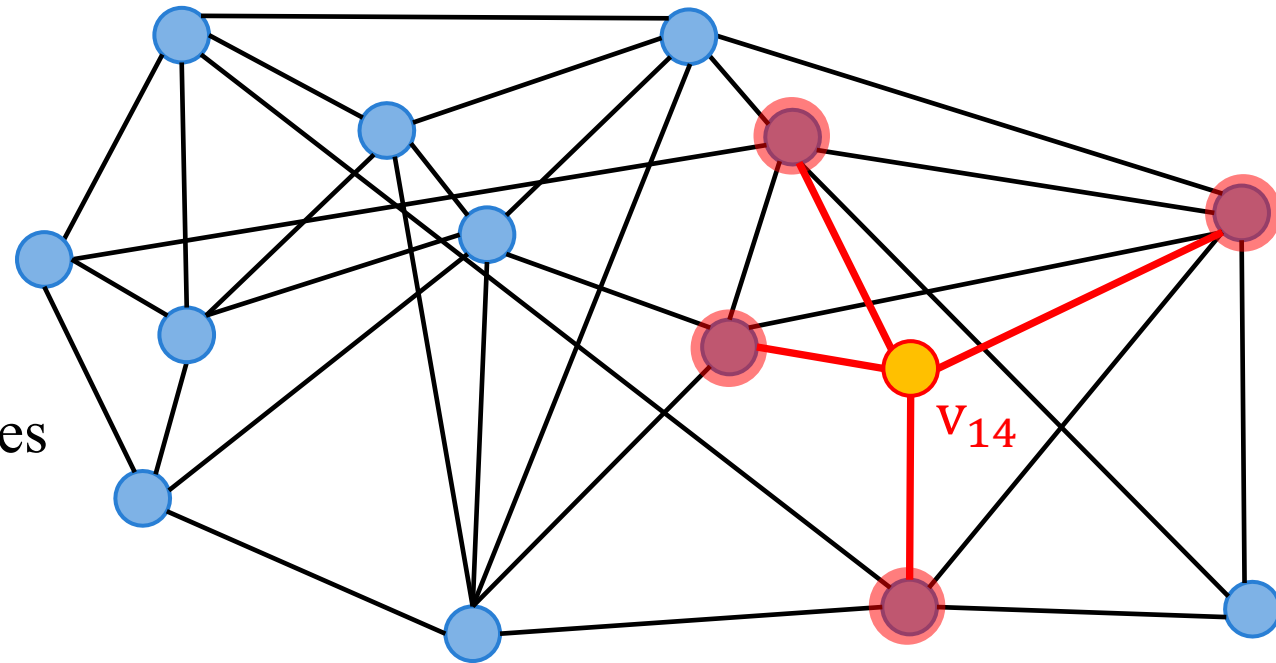
- Insert node into the graph one by one
- Example: HNSW graph of  $v_1 v_2 \dots v_{13}$
- To insert a node  $v_{14}$ 
  1. create  $\leq M$  edges from  $v_{14}$



# HNSW Graph Construction: 10,000 foot view

---

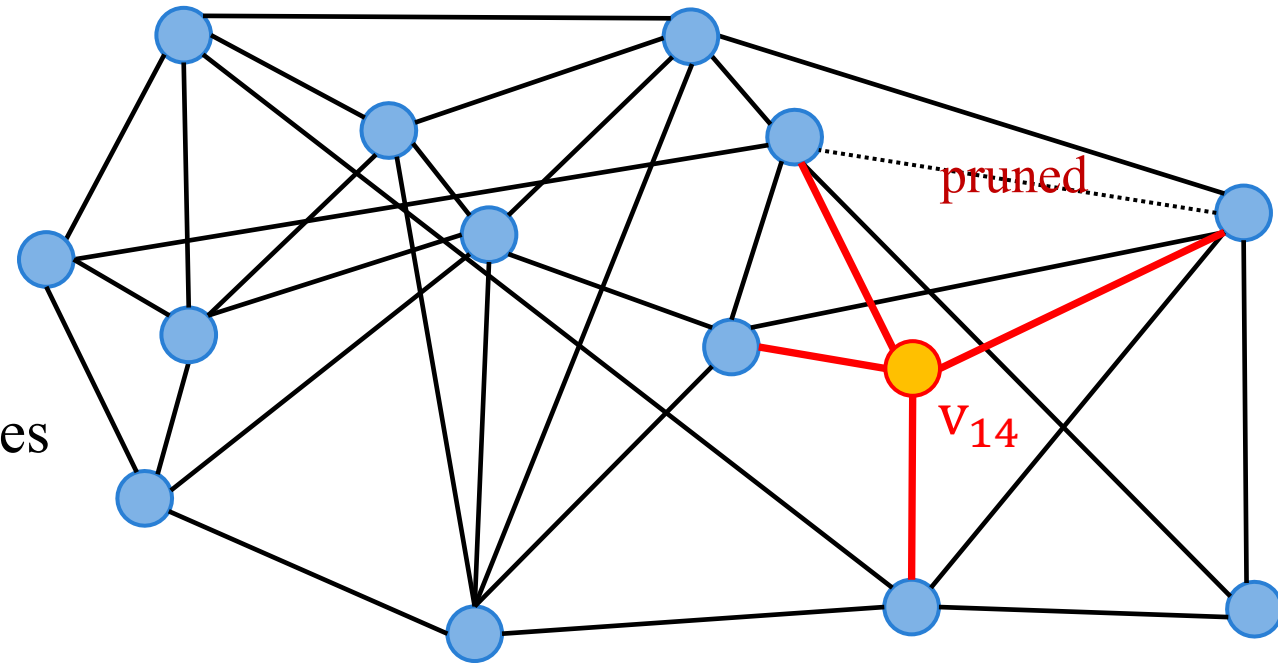
- Insert node into the graph one by one
- Example: HNSW graph of  $v_1 v_2 \dots v_{13}$
- To insert a node  $v_{14}$ 
  1. create  $\leq M$  edges from  $v_{14}$
  2. check connected node, if it has more than  $M$  edges, prune its edges



# HNSW Graph Construction: 10,000 foot view

---

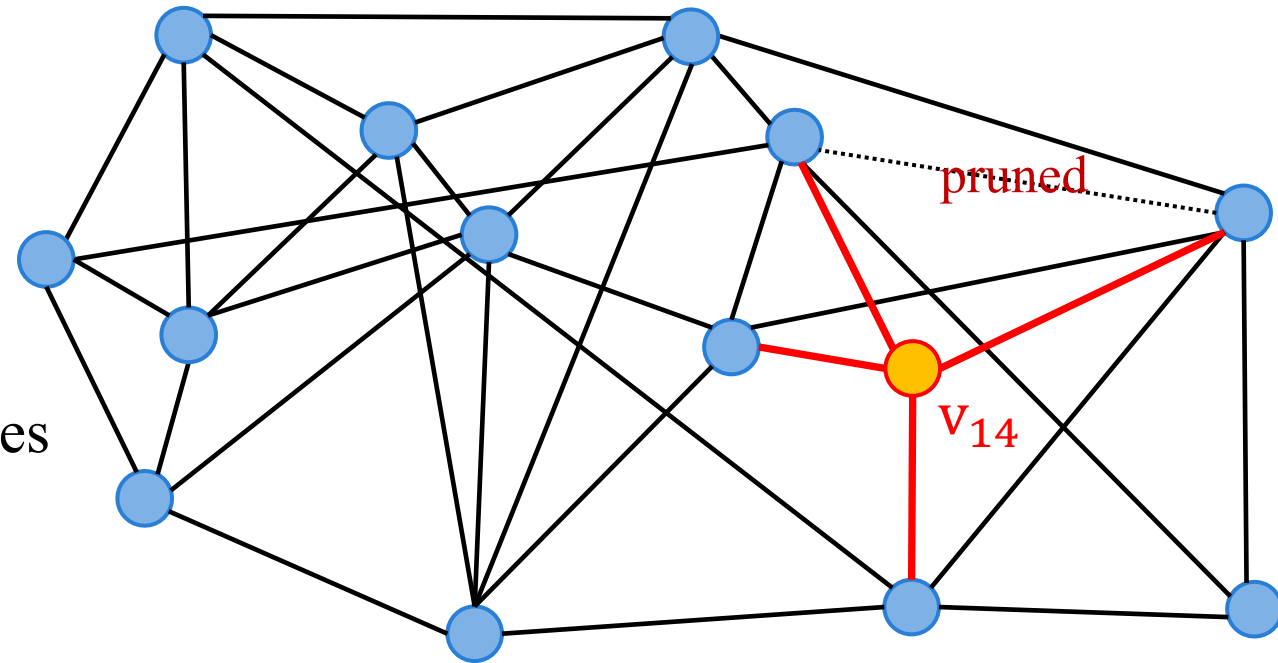
- Insert node into the graph one by one
- Example: HNSW graph of  $v_1 v_2 \dots v_{13}$
- To insert a node  $v_{14}$ 
  1. create  $\leq M$  edges from  $v_{14}$
  2. check connected node, if it has more than  $M$  edges, prune its edges



# HNSW Graph Construction: 10,000 foot view

---

- Insert node into the graph one by one
- Example: HNSW graph of  $v_1 v_2 \dots v_{13}$
- To insert a node  $v_{14}$ 
  1. create  $\leq M$  edges from  $v_{14}$
  2. check connected node, if it has more than  $M$  edges, prune its edges



HNSW graph of  $v_1 v_2 \dots v_{13} v_{14}$



# SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search

CHAOJI ZUO<sup>\*</sup>, Rutgers University, USA

MIAO QIAO, The University of Auckland, New Zealand

WENCHAO ZHOU, Alibaba Group, China

FEIFEI LI, Alibaba Group, China

DONG DENG<sup>†</sup>, Rutgers University, USA

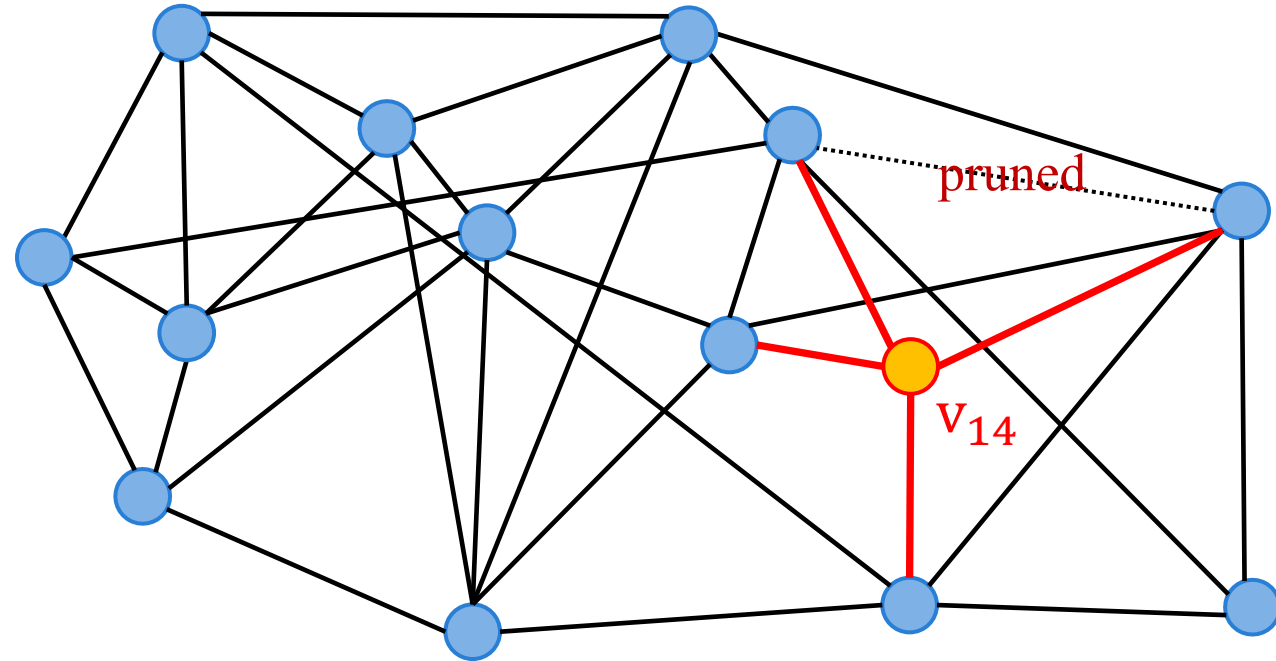
Effective vector representation models, e.g., word2vec and node2vec, embed real-world objects such as images and documents in high dimensional vector space. In the meanwhile, the objects are often associated with attributes such as timestamps and prices. Many scenarios need to jointly query the vector representations of the objects together with their attributes. These queries can be formalized as range-filtering approximate nearest neighbor search (ANNS) queries. Specifically, given a collection of data vectors, each associated with an attribute value whose domain has a total order. The range-filtering ANNS consists of a query range and a query vector. It finds the approximate nearest neighbors of the query vector among all the data vectors whose attribute values fall in the query range. Existing approaches suffer from a rapidly degrading query performance when the query range width shifts. The query performance can be optimized by a solution that builds an ANNS index for every possible query range; however, the index time and index size become prohibitive – the number of query ranges is quadratic to the number  $n$  of data vectors. To overcome these challenges, for the query range contains all attribute values smaller than a user-provided threshold, we design a structure called the segment graph whose index time and size are the same as a single ANNS index, yet can losslessly compress the  $n$  ANNS indexes, reducing the indexing cost by a factor of  $\Omega(n)$ . To handle general range queries, we propose a 2D segment graph with average-case index size  $O(n \log n)$  to compress  $n$  segment graphs, breaking the quadratic barrier. Extensive experiments conducted on real-world datasets show that our proposed structures outperformed existing methods significantly; our index also exhibits superior scalability.



# Range-Filtering ANNS: Segment Graph

---

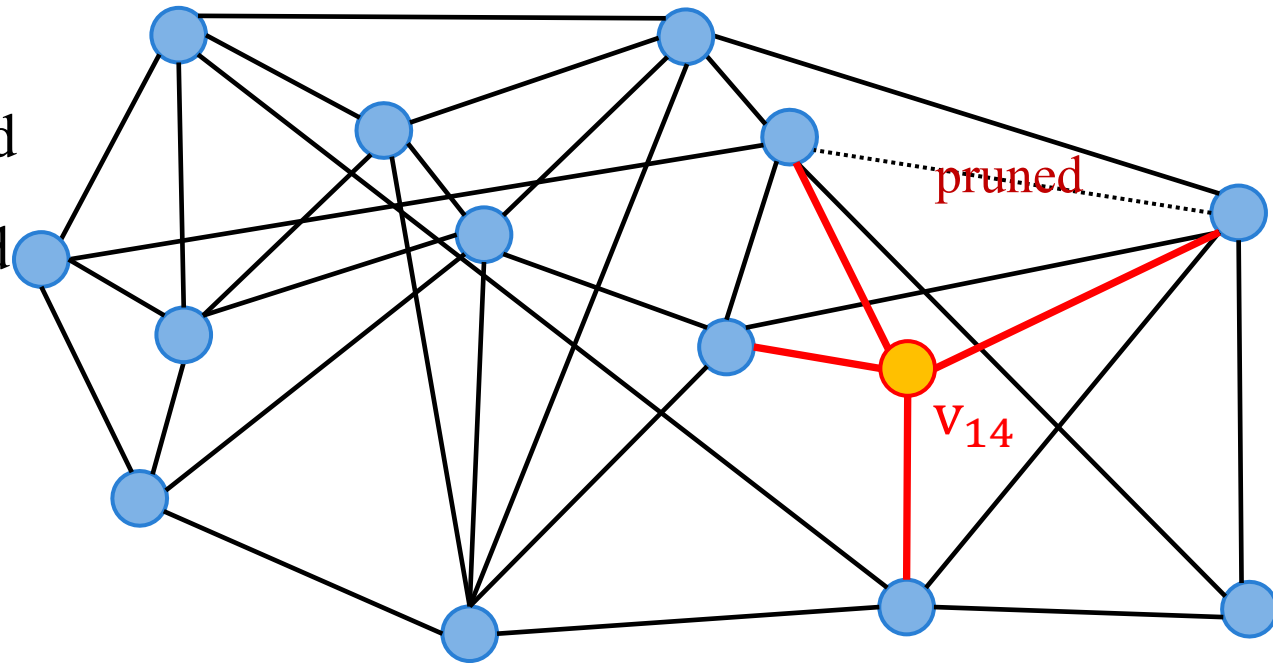
- Insert the nodes in the order of their attribute values.



# Range-Filtering ANNS: Segment Graph

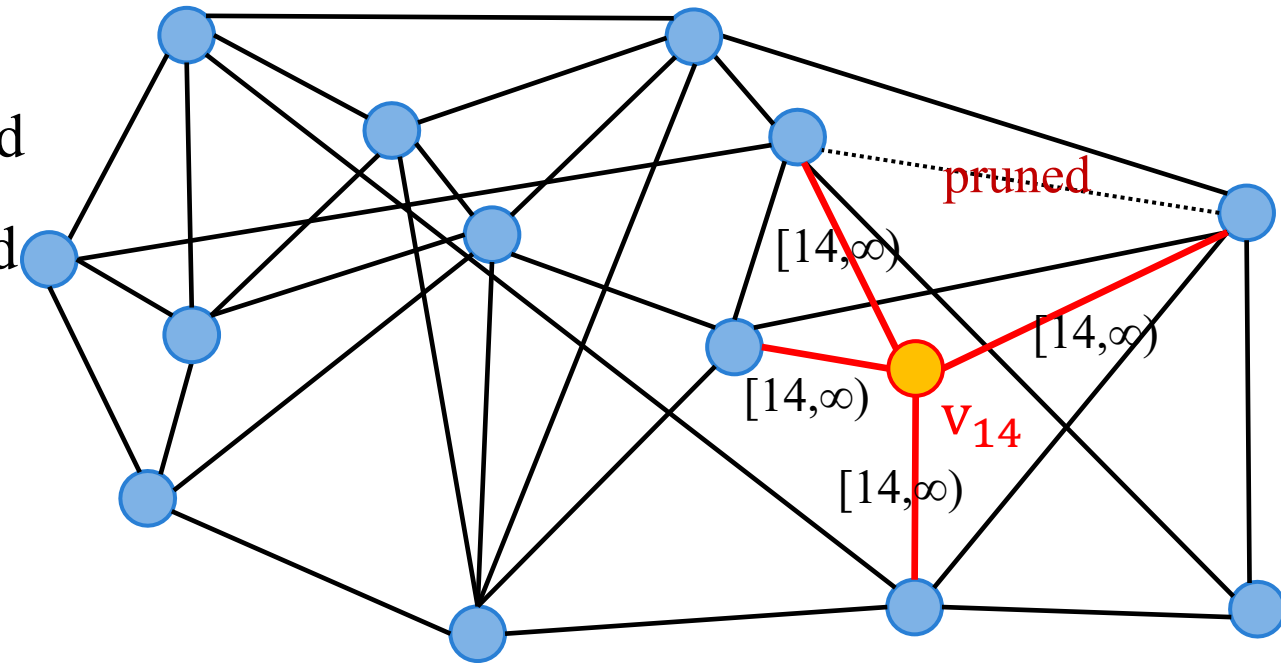
---

- Insert the nodes in the order of their attribute values.
- Each edge is labeled a segment  $[x, y)$ 
  - edge created when  $x^{\text{th}}$  node inserted
  - edge pruned when  $y^{\text{th}}$  node inserted



# Range-Filtering ANNS: Segment Graph

- Insert the nodes in the order of their attribute values.
- Each edge is labeled a segment  $[x, y)$ 
  - edge created when  $x^{\text{th}}$  node inserted
  - edge pruned when  $y^{\text{th}}$  node inserted

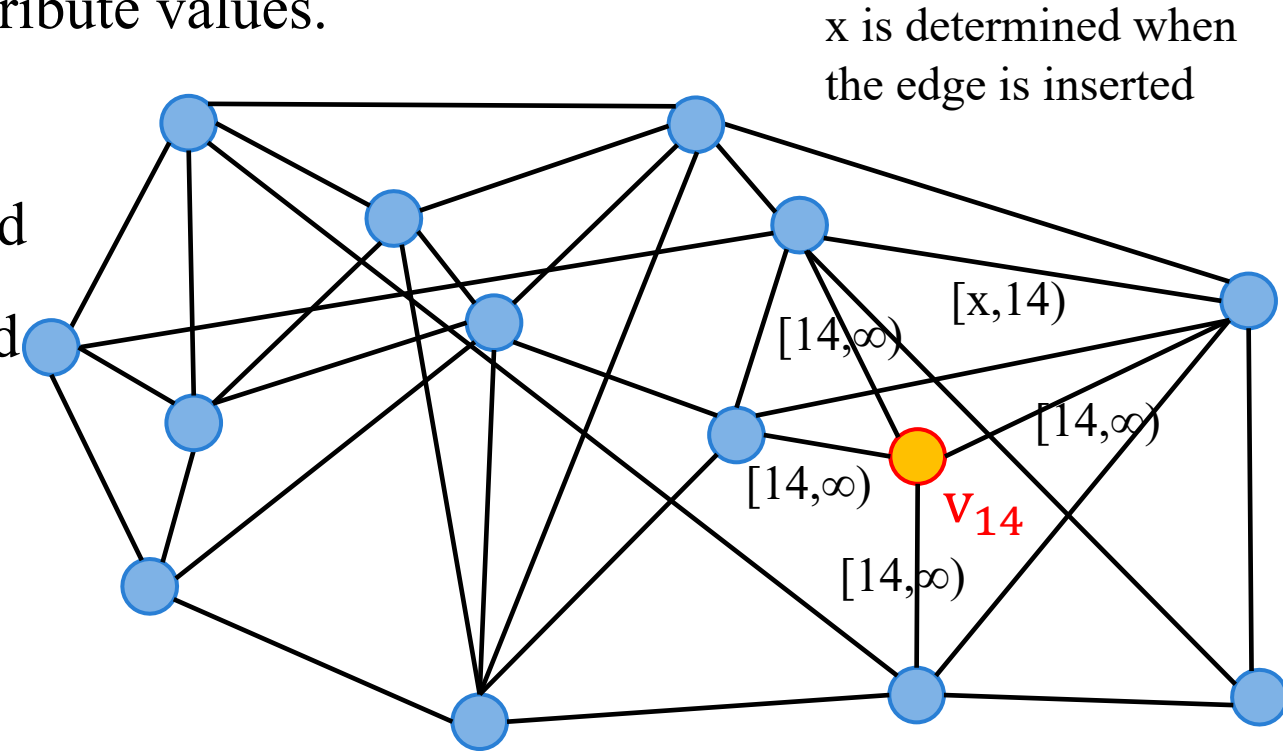


the  $\infty$  will be updated  
when the edge is pruned

# Range-Filtering ANNS: Segment Graph

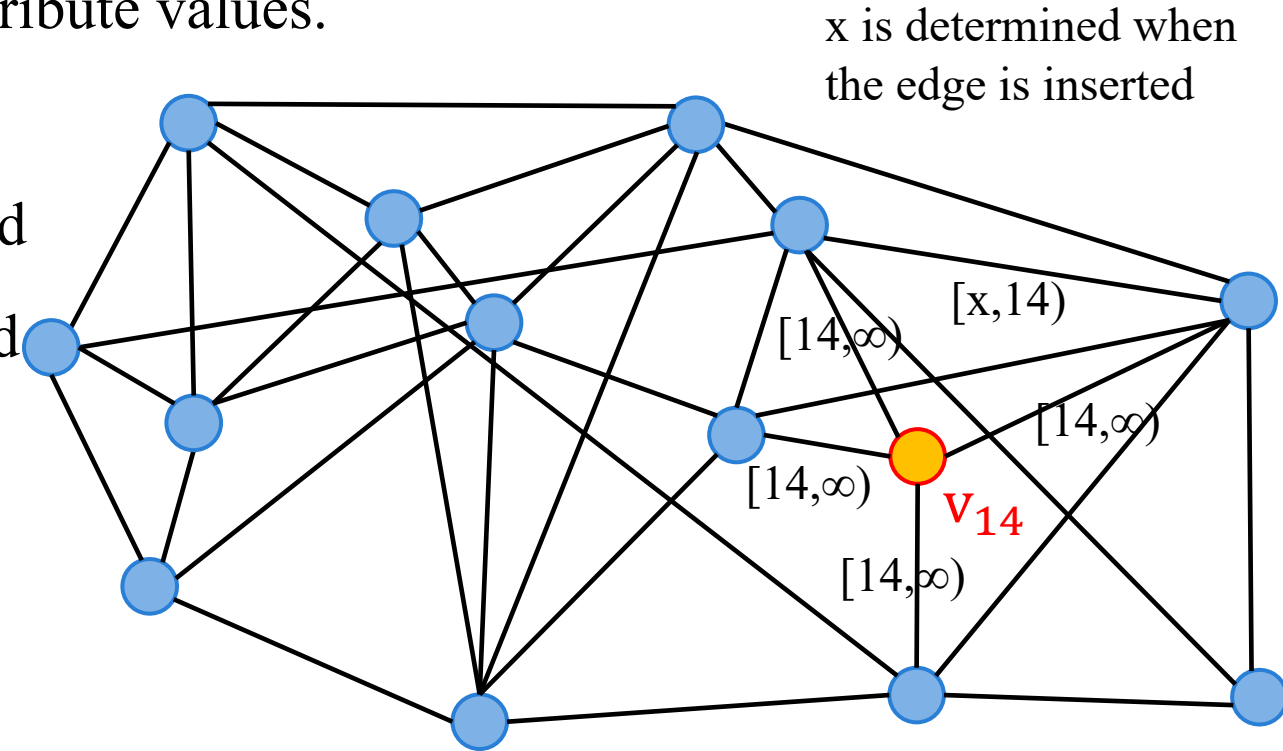
---

- Insert the nodes in the order of their attribute values.
- Each edge is labeled a segment  $[x, y)$ 
  - edge created when  $x^{\text{th}}$  node inserted
  - edge pruned when  $y^{\text{th}}$  node inserted



# Range-Filtering ANNS: Segment Graph

- Insert the nodes in the order of their attribute values.
- Each edge is labeled a segment  $[x, y)$ 
  - edge created when  $x^{\text{th}}$  node inserted
  - edge pruned when  $y^{\text{th}}$  node inserted

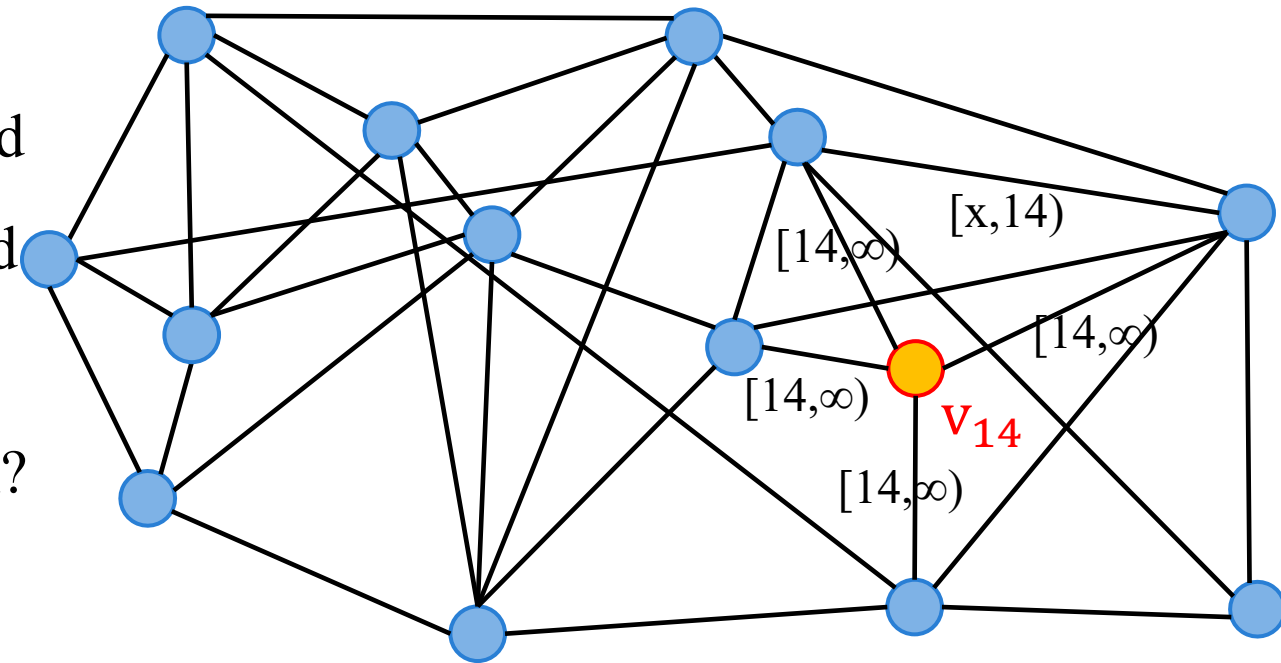


Segment Graph, a lossless  
compression of  $n$  HNSW graphs

# Range-Filtering ANNS: Segment Graph

---

- Insert the nodes in the order of their attribute values.
- Each edge is labeled a segment  $[x, y)$ 
  - edge created when  $x^{\text{th}}$  node inserted
  - edge pruned when  $y^{\text{th}}$  node inserted
- The HNSW graph for query range  $\leq m$ ?

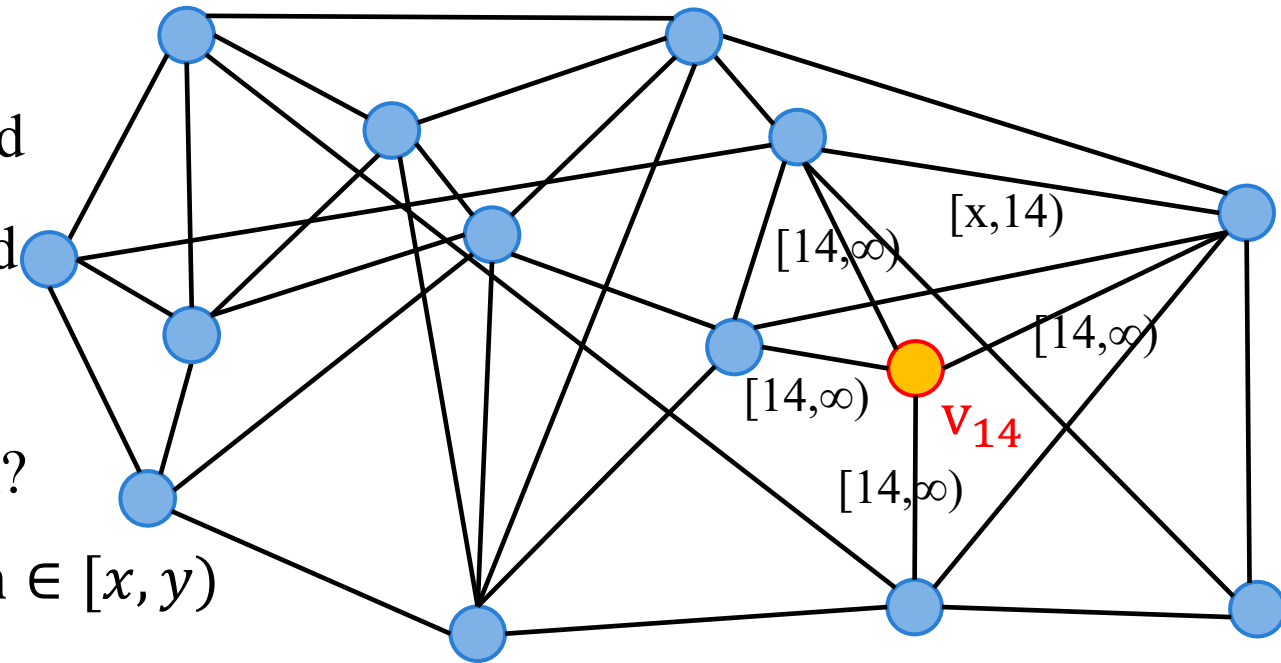


Segment Graph, a lossless  
compression of  $n$  HNSW graphs

# Range-Filtering ANNS: Segment Graph

---

- Insert the nodes in the order of their attribute values.
- Each edge is labeled a segment  $[x, y)$ 
  - edge created when  $x^{\text{th}}$  node inserted
  - edge pruned when  $y^{\text{th}}$  node inserted
- The HNSW graph for query range  $\leq m$ ?
  - edges with segment  $[x, y)$  where  $m \in [x, y)$



Segment Graph, a lossless  
compression of  $n$  HNSW graphs

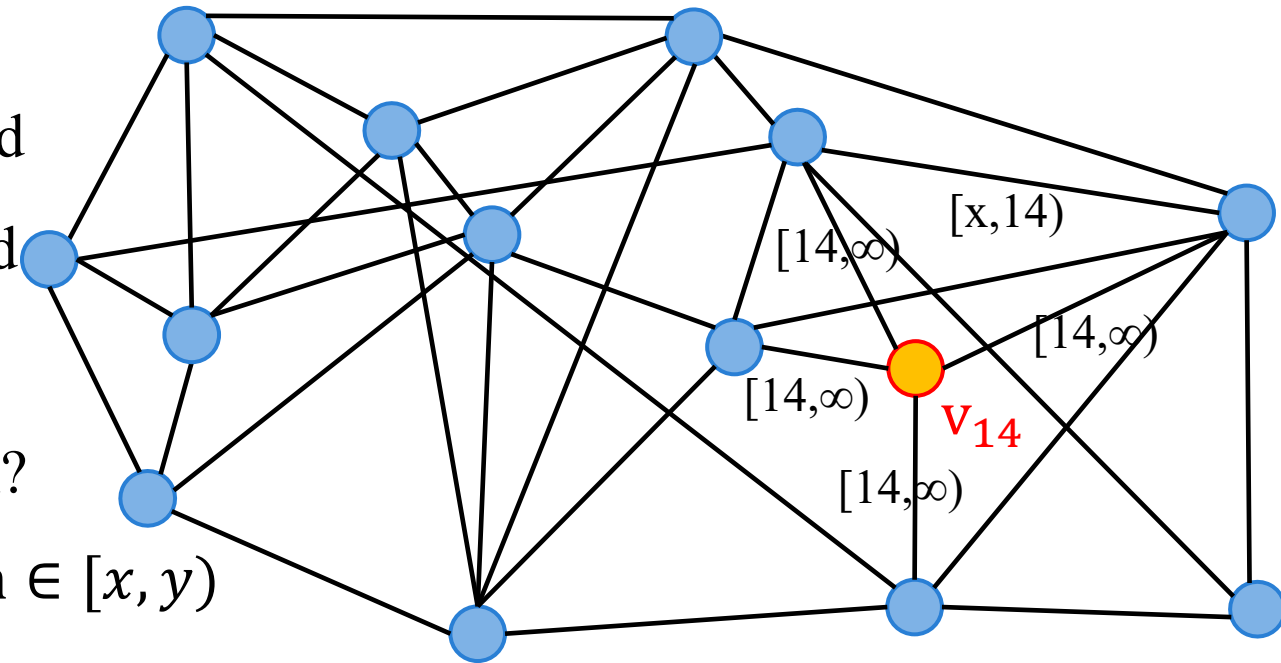


# Range-Filtering ANNS: Segment Graph

- Insert the nodes in the order of their attribute values.

- Each edge is labeled a segment  $[x, y)$ 
  - edge created when  $x^{\text{th}}$  node inserted
  - edge pruned when  $y^{\text{th}}$  node inserted

- The HNSW graph for query range  $\leq m$ ?
  - edges with segment  $[x, y)$  where  $m \in [x, y)$ 
    - if  $m < x$ , the edge has not created yet



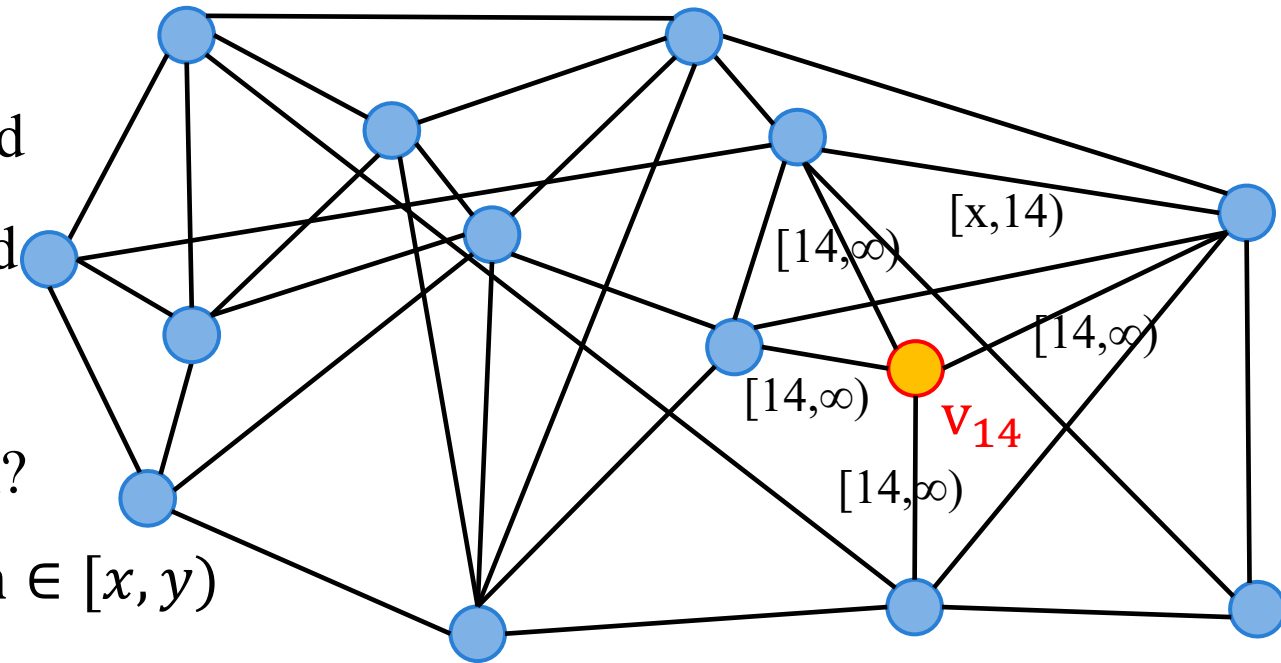
Segment Graph, a lossless  
compression of  $n$  HNSW graphs

# Range-Filtering ANNS: Segment Graph

- Insert the nodes in the order of their attribute values.

- Each edge is labeled a segment  $[x, y)$ 
  - edge created when  $x^{\text{th}}$  node inserted
  - edge pruned when  $y^{\text{th}}$  node inserted

- The HNSW graph for query range  $\leq m$ ?
  - edges with segment  $[x, y)$  where  $m \in [x, y)$ 
    - if  $m < x$ , the edge has not created yet
    - if  $y \leq m$ , the edge has been pruned

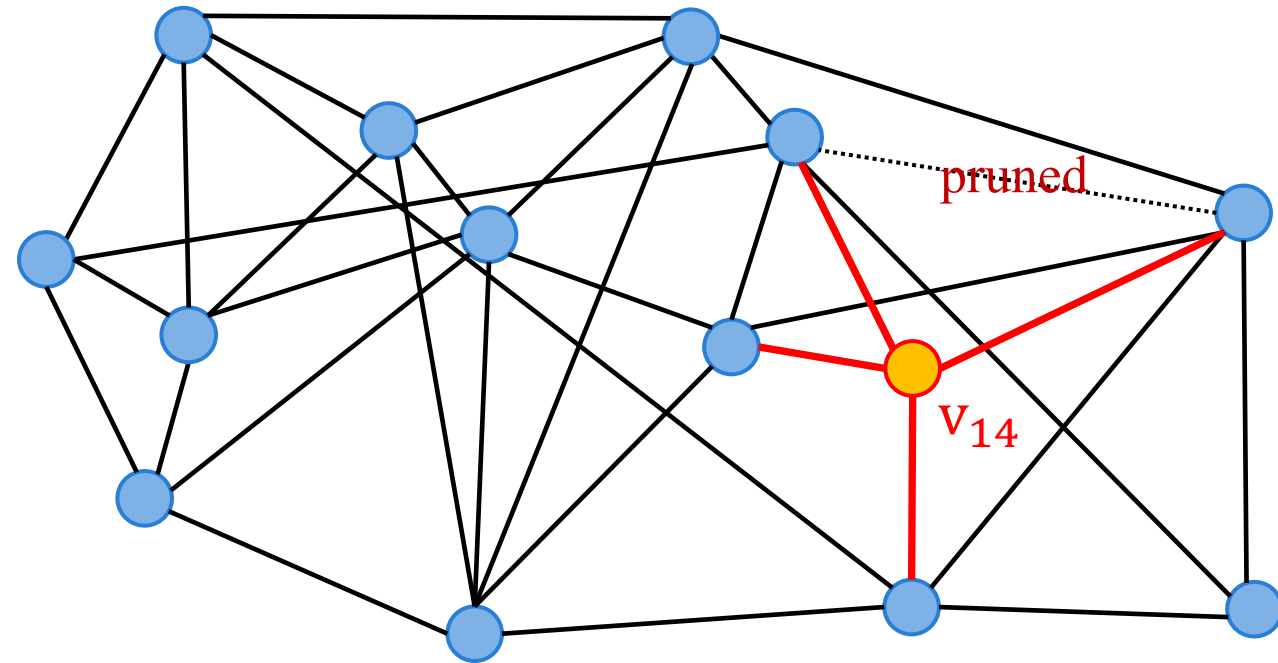


Segment Graph, a lossless  
compression of  $n$  HNSW graphs

# Range-Filtering ANNS: Analysis

---

- **Observation:** in HNSW the new edge must from the new node, while pruned edges are between old nodes

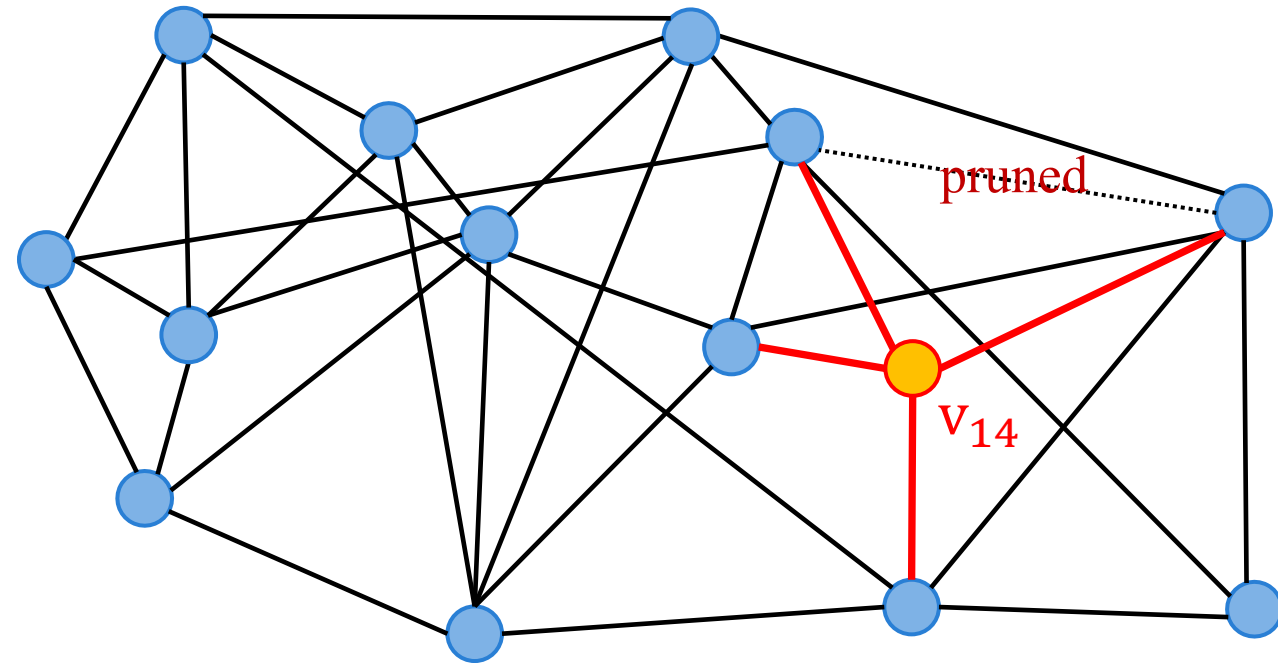


HNSW graph of  $v_1 v_2 \dots v_{13} v_{14}$

# Range-Filtering ANNS: Analysis

---

- **Observation:** in HNSW the new edge must from the new node, while pruned edges are between old nodes
  - thus a pruned edge in HNSW will never be added back.

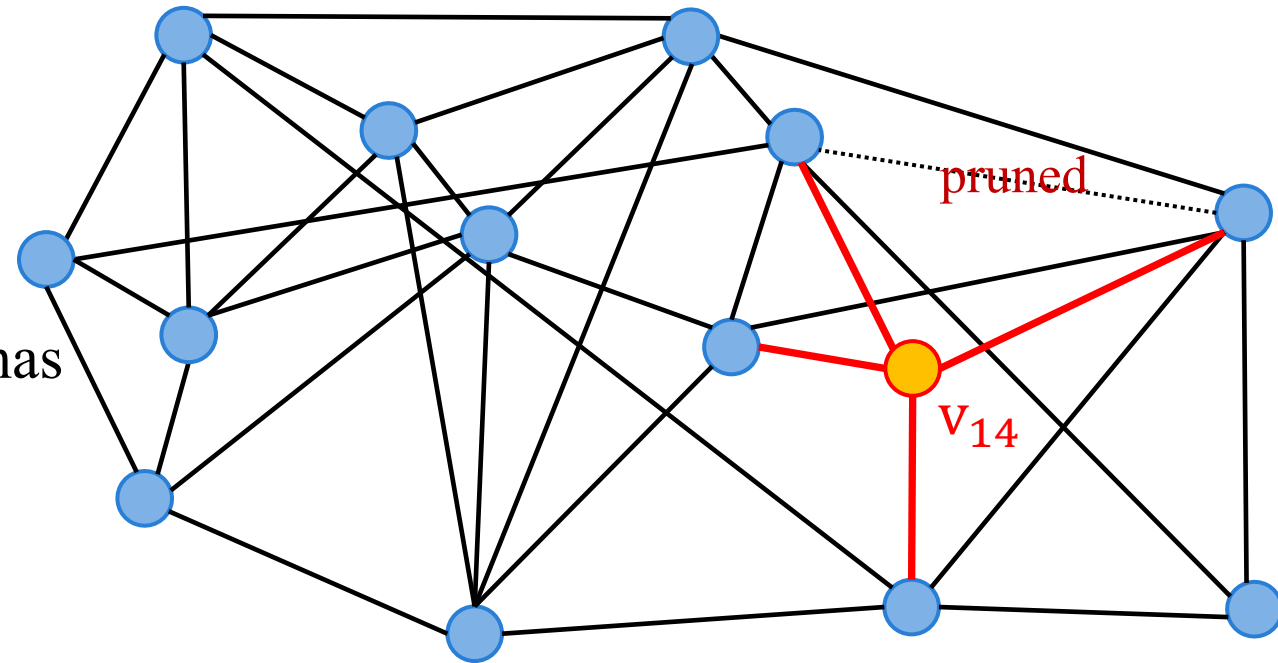


HNSW graph of  $v_1 v_2 \dots v_{13} v_{14}$

# Segment Graph: Analysis

---

- **Observation:** in HNSW the new edge must from the new node, while pruned edges are between old nodes
  - thus a pruned edge in HNSW will never be added back.
  - thus each edge in Segment Graph has a unique segment.

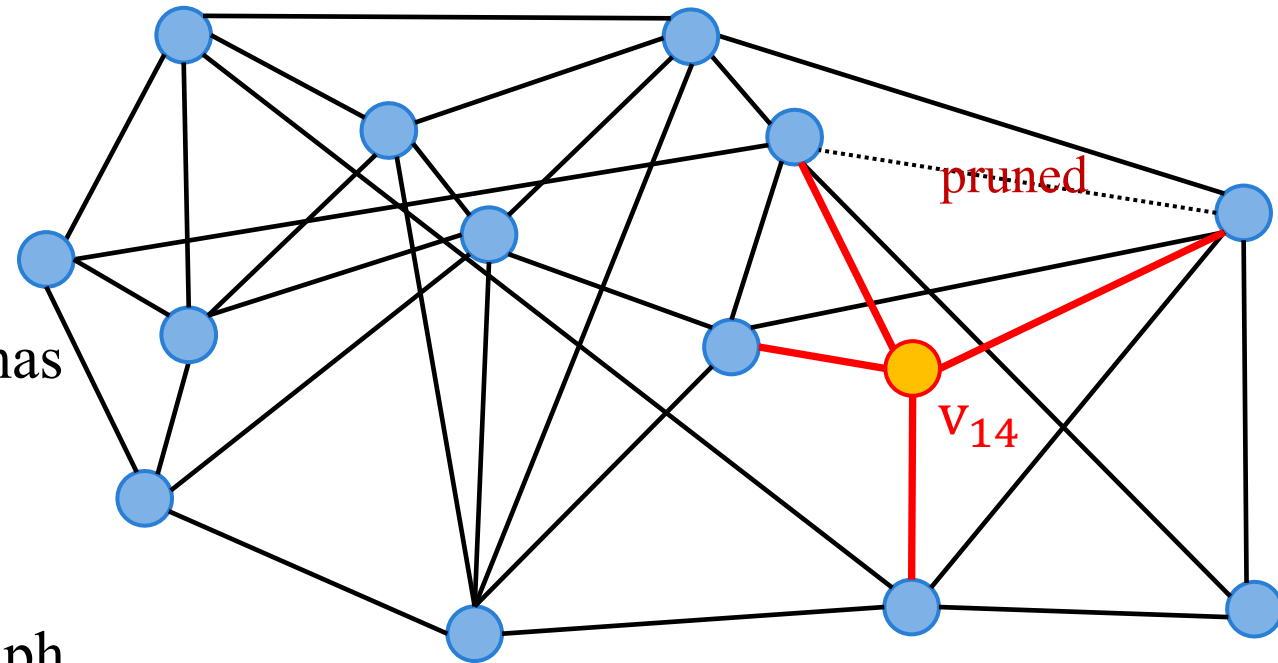


HNSW graph of  $v_1 v_2 \dots v_{13} v_{14}$

# Segment Graph: Analysis

---

- **Observation:** in HNSW the new edge must from the new node, while pruned edges are between old nodes
  - thus a pruned edge in HNSW will never be added back.
  - thus each edge in Segment Graph has a unique segment.
- For  $n$  nodes,
  - at most  $nM$  edges in an HNSW graph

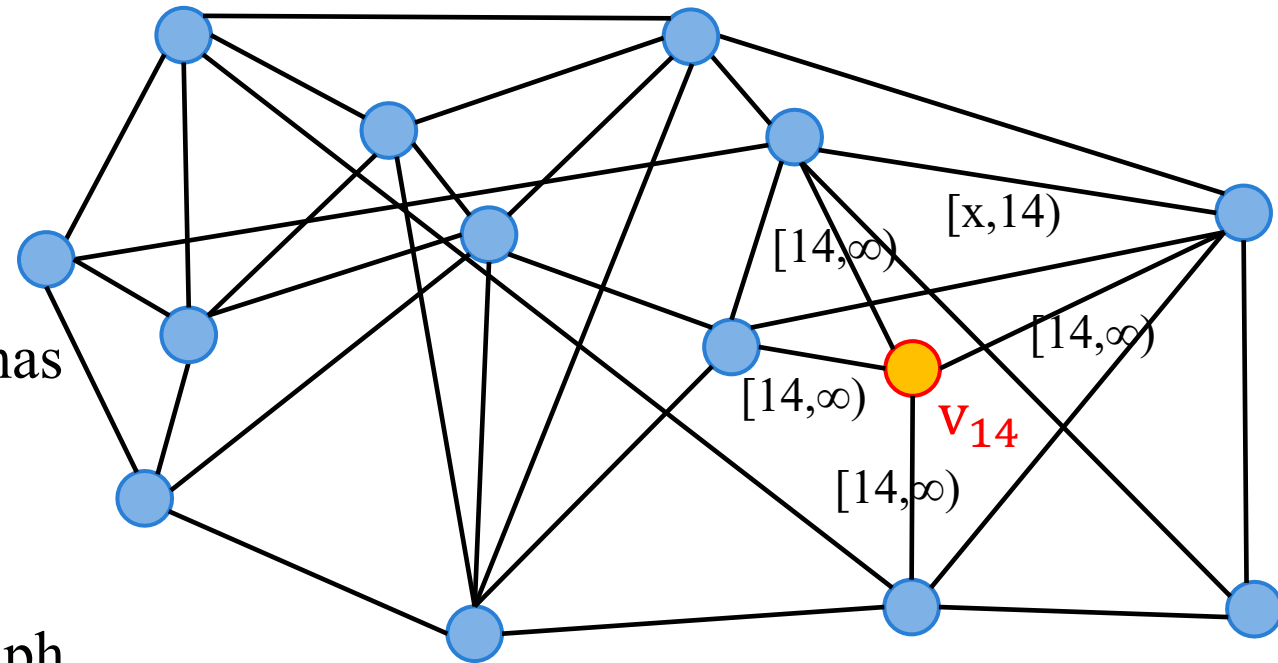


HNSW graph of  $v_1 v_2 \dots v_{13} v_{14}$

# Segment Graph: Analysis

---

- **Observation:** in HNSW the new edge must from the new node, while pruned edges are between old nodes
  - thus a pruned edge in HNSW will never be added back.
  - thus each edge in Segment Graph has a unique segment.
- For  $n$  nodes,
  - at most  $nM$  edges in an HNSW graph
  - at most  $2nM$  edges in Segment Graph (each edge is stored twice & no pruning).

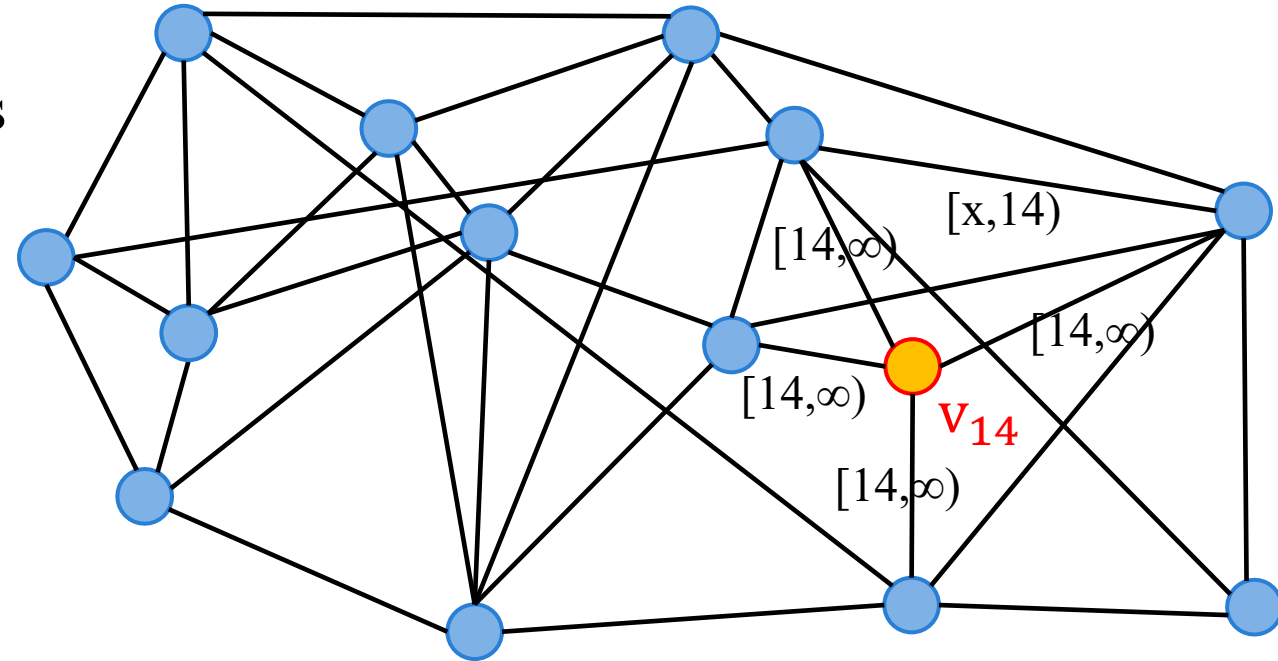




# Segment Graph: Conclusion

---

- A segment graph is a lossless compression of  $n$  HNSW graphs.
  - same index time and size complexity as building a HNSW graph over all nodes
  - minimal query overhead.
  - same query accuracy/results as HNSW
  - best of both worlds: excellent query performance and low index cost.



# Arbitrary Query Ranges

---

- A **2D segment graph** to compress the  $n$  segment graphs
  - a segment graph for all query ranges starting from each  $x$  in  $[1, n]$
  - $n$  segment graphs is enough for all possible query ranges

In the 2D Segment Graph, we assign a **tuple  $(l, r, b, e)$**  to each edge, where  $l$  and  $r$  represent the Segment Graphs the edge belongs to, and  $b$  and  $e$  represent when the edge is added and pruned in those Segment Graphs.

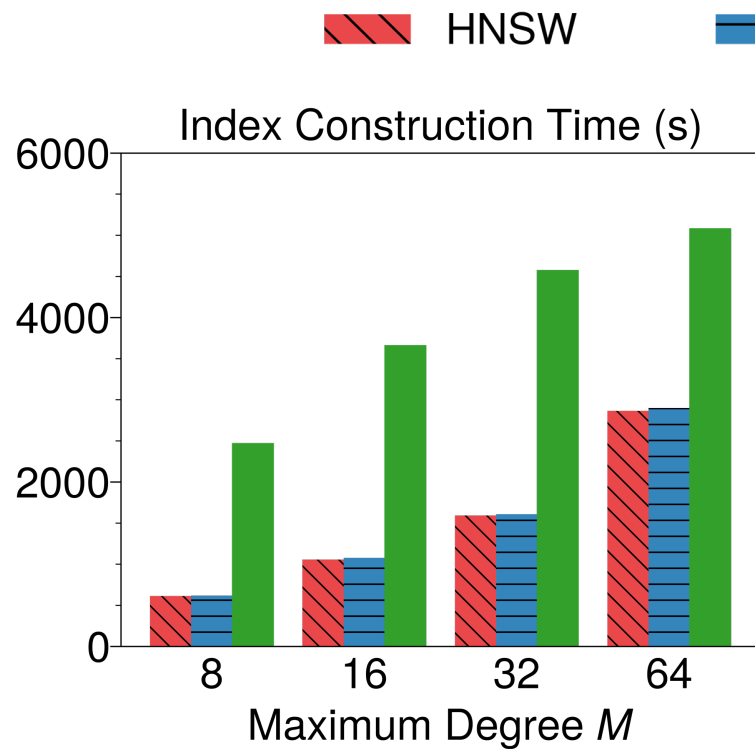
For example, the edge from point 9 to point 15 has a tuple indicating it belongs to Segment Graphs from 0 to  $n$ , 1 to  $n$  and 2 to  $n$ . And this edge is added at 9 and pruned at 15 in all of these segment graph.

# Experiment Setup

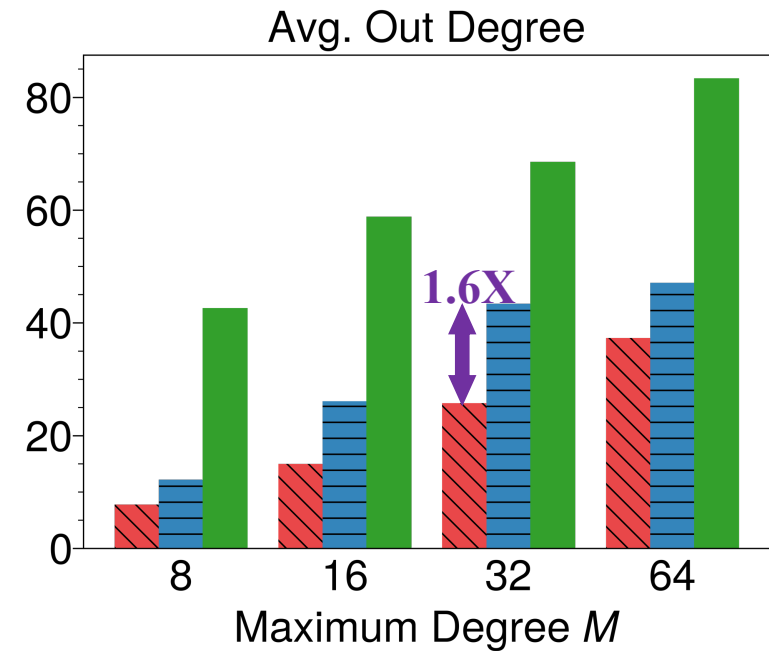
---

- Datasets
  - **DEEP**: 96-D image descriptor embedding of ImageNet, random number as the synthetic attribute value
  - **YouTube-Audio**: 128-D audio feature vector of YouTube videos. Video released time as attribute value
  - **WIT-Images**: 2048-D ResNet-50 embedding of images from Wikipedia. The size of the image as the attribute value
- Environment
  - Intel® Xeon® Platinum 8358 CPU@2.6GHz
  - 256GB RAM

# Index Construction

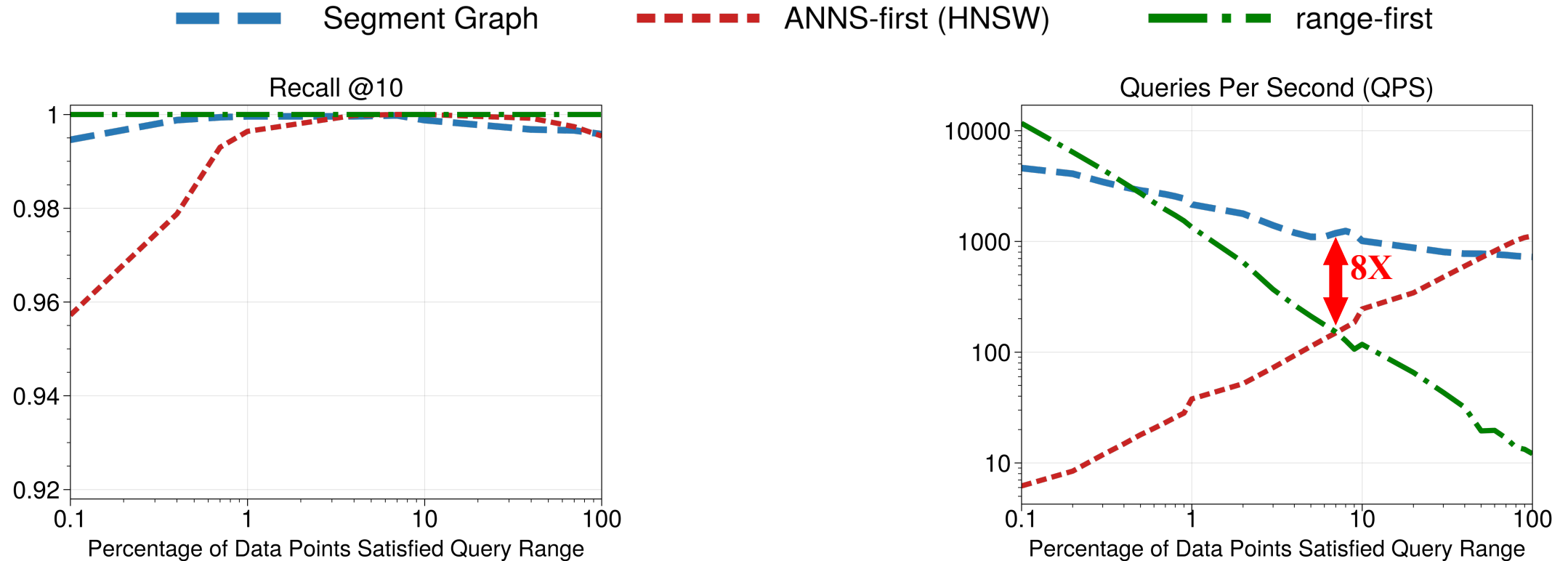


Index time of Segment Graph and HNSW is almost the same



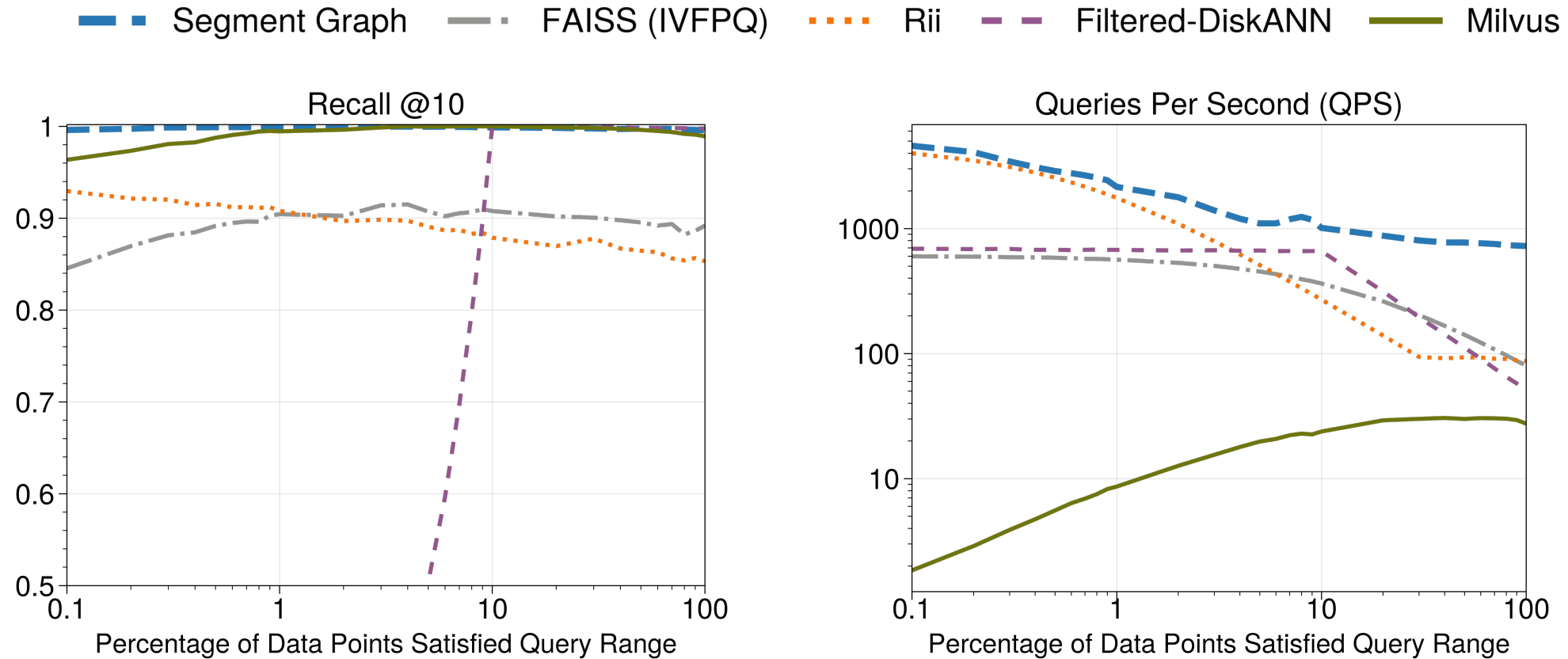
Segment Graph has at most 1.6x the avg. degree of HNSW

# Half-bounded Query Ranges



For query range size between 0.5% and 50% segment graph achieved the highest QPS and almost perfect recall

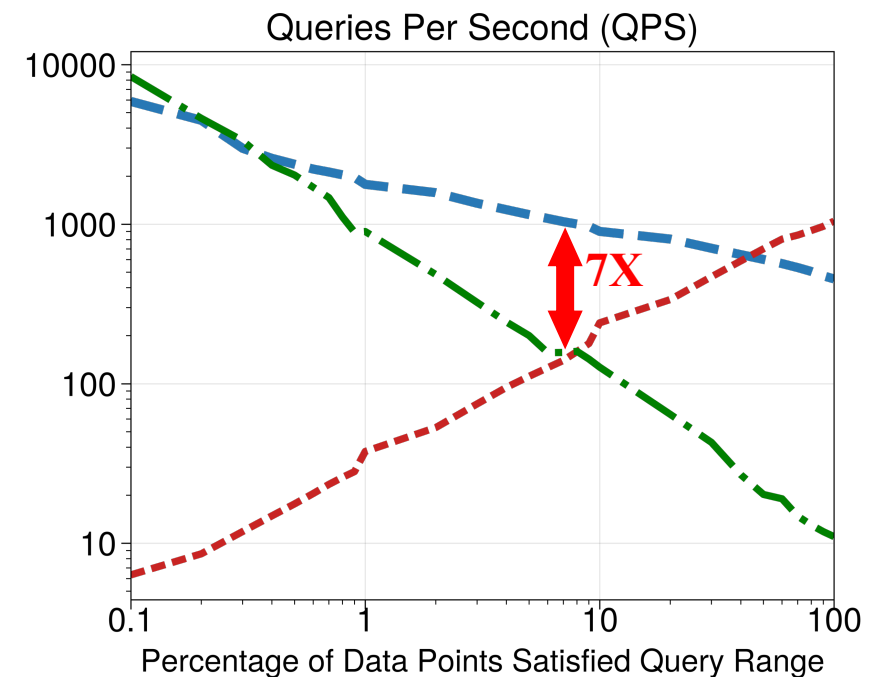
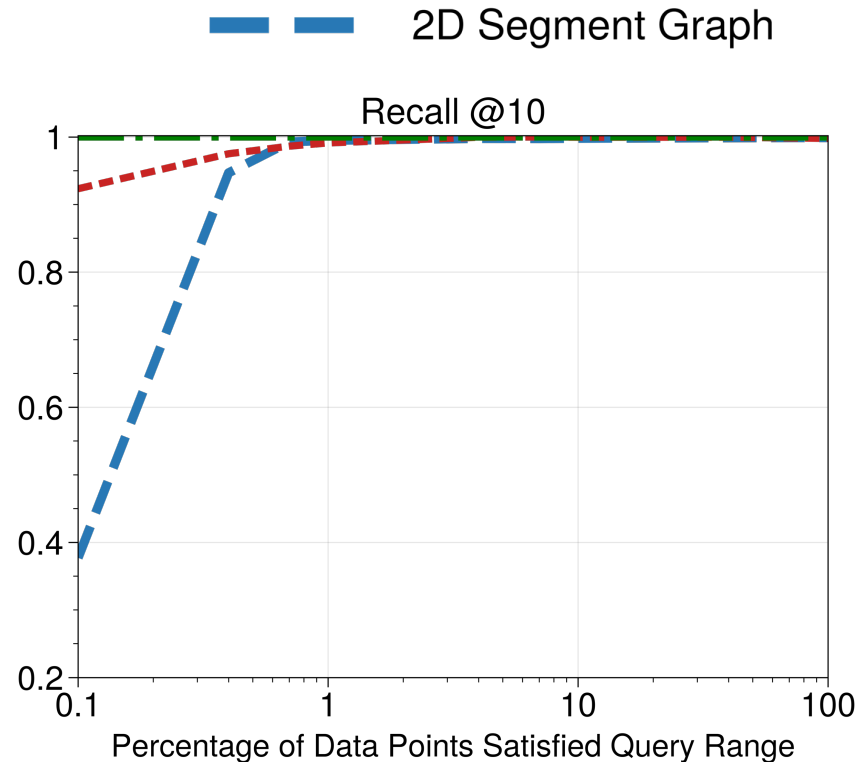
# Half-bounded Query Ranges – More Baselines



FAISS is a popular library for ANNS  
Filtered-DiskANN is graph-based

Rii is based on product quantization  
Milvus is a vector database

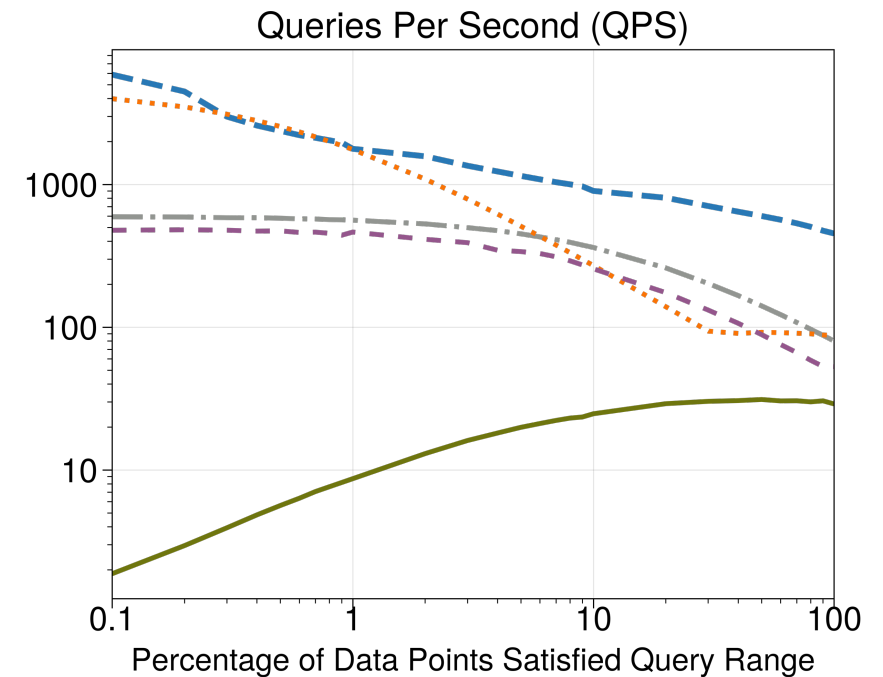
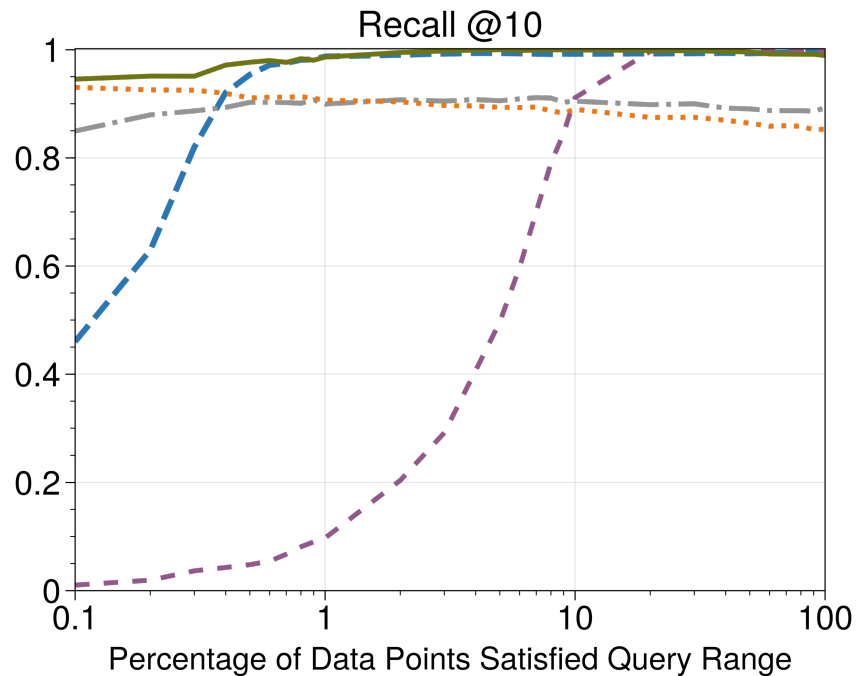
# Arbitrary Query Range



2D Segment Graph achieved high recall when query range size  $> 0.2\%$  while having high QPS

# Arbitrary Query Range – More Baselines

2D Segment Graph   FAISS (IVFPQ)   Rii   Filtered-DiskANN   Milvus



Our method outperforms other existing methods in when query range size  $> 0.5\%$



# Lemmas

---

Lemma 1. Under the same parameters  $M$  and  $K$ , the HNSW graph constructed after inserting nodes in  $Dx = \{v_1, v_2, \dots, v_x\}$  sequentially is the graph  $Gx$  reported by Algorithm 4, for each  $x \in [n]$ .

Lemma 2. Based on Definition 2, for an integer  $K$ ,  $kNN(vx, [i, x - 1], K)$  denotes the set of  $K$ -nearest neighbors of  $vx$  in  $D_{i,x-1}$  and  $kNN(vx, [i + 1, x - 1], K)$  denotes that in  $D_{i+1,x-1}$ . If  $vi \notin kNN(vx, [i, x - 1], K)$ , we have  $kNN(vx, [i, x - 1], K) = kNN(vx, [i + 1, x - 1], K)$ .

Lemma 3. Let  $i'$  be the minimum key of points in the  $K$ -nearest neighbors  $kNN(vx, [i, x - 1], K)$  in  $D_{i,x-1}$ , we have  $kNN(vx, [i, x - 1], K) = kNN(vx, [i + 1, x - 1], K) = \dots = kNN(vx, [i', x - 1], K)$ .

Lemma 4. On a segment graph  $G$  constructed with Algorithm 5 opting out Step 3, in the neighbor list of any node  $vi \in D$ , every triple  $(vj, b, e)$  must have  $b = \max\{i, j\}$  and  $e = n$ .

Lemma 5. The worst-case size of a 2DSegmentGraph is  $O(n^2M)$ . The worst-case 2DSegmentGraph construction time has two parts: 1)  $O(n^2)$  calls of online searches (Algorithm 7), and 2) indexing time  $O(n^2 * M^2)$  excluding the online search.

# Theorems

---

Theorem 6. Consider the segment graph  $G$ . For each edge from  $v_i$  to  $v_j$  in  $G$ ,  $\forall i, j \in [n]$ , i.e., there is a triple  $(v_j, b, e)$  in the neighbor list of  $v_i$  in  $G$ , we call  $[b, e]$  the range of edge  $(v_i, v_j)$ . For any  $1 \leq x \leq n$ , denote by  $G[x]$  the subgraph of the segment graph  $G$  containing all edges whose range has  $b \leq x \leq e$ ; let  $G_x$  be the  $x$ -th HNSW graphs produced by Alg. 4. We have  $G[x] = G_x$ , for all  $x \in [n]$ ; **in other words, the segment graph  $G$  produced by Alg. 5 is a lossless compression of the  $n$  HNSW graphs produced by Alg. 4.**

**Complexity Analysis.** Inserting a node to the segment graph creates no more than  $2M$  edges; the total number of edges in a segment graph is no more than  $2nM$ . In contrast, in an HNSW graph, the outdegree of each node is bounded by  $M$ , and the total number of edges in an HNSW graph is no more than  $nM$ . The space complexity of both structures is  $O(nM)$ .

# Theorems

---

Theorem 7. Under the assumption that the attribute values of points in  $D$  are independent, the expected number of rounds in Lines 4-9 of Algorithm 8 is  $O(n \log n)$ , and thus in the average-case, the index size of 2DSegmentGraph is  $O(nM \log n)$  and index time has two parts: 1)  $O(n \log n)$  calls of online searches (Algorithm 7), and 2) indexing time  $O(nM^2 \log n)$  excluding the online search.

Theorem 8. The 2D segment graph produced by Algorithm 8 is a lossless compression of  $n$  segment graphs  $G_1, G_2, \dots, G_n$  constructed by Algorithm 5 with Step 3 opted out under an assumption that the procedures SegmentANNSearch and 2DSegmentANNSearch return the exact  $K$ -nearest neighbors of the query point.

# Filtered Vector Search

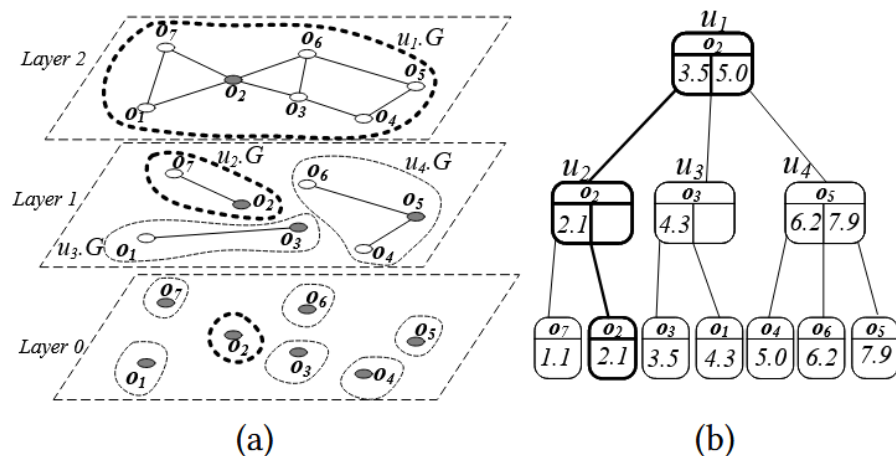
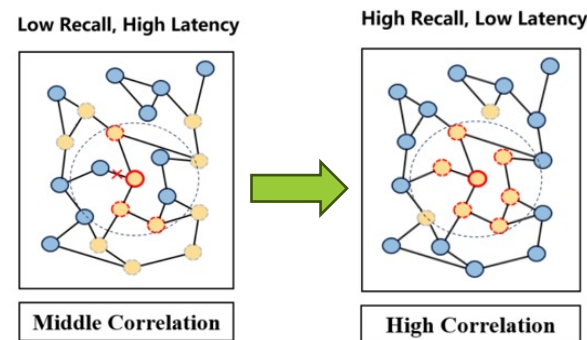


Fig. 3. The DIGRA index for object set  $O$  in Figure 2.

通过基于范围属性划分子图使得每个子图内节点都满足谓词条件



## DIGRA ( search-aware )

### ■ 核心思想：引入B树解决动态更新问题

- a) B树式动态属性管理：借鉴 B 树的分裂与合并机制，通过树结构的自动平衡，应对大规模数据的频繁实时增删。
- b) 嵌套式局部图索引：在树节点内部维护 NSW 图，利用“树定位区间、图定向检索”提升搜索效率。

# Filtered Vector Search

## ■ 三个设计维度

### □ 过滤谓词类型 (What to filter)

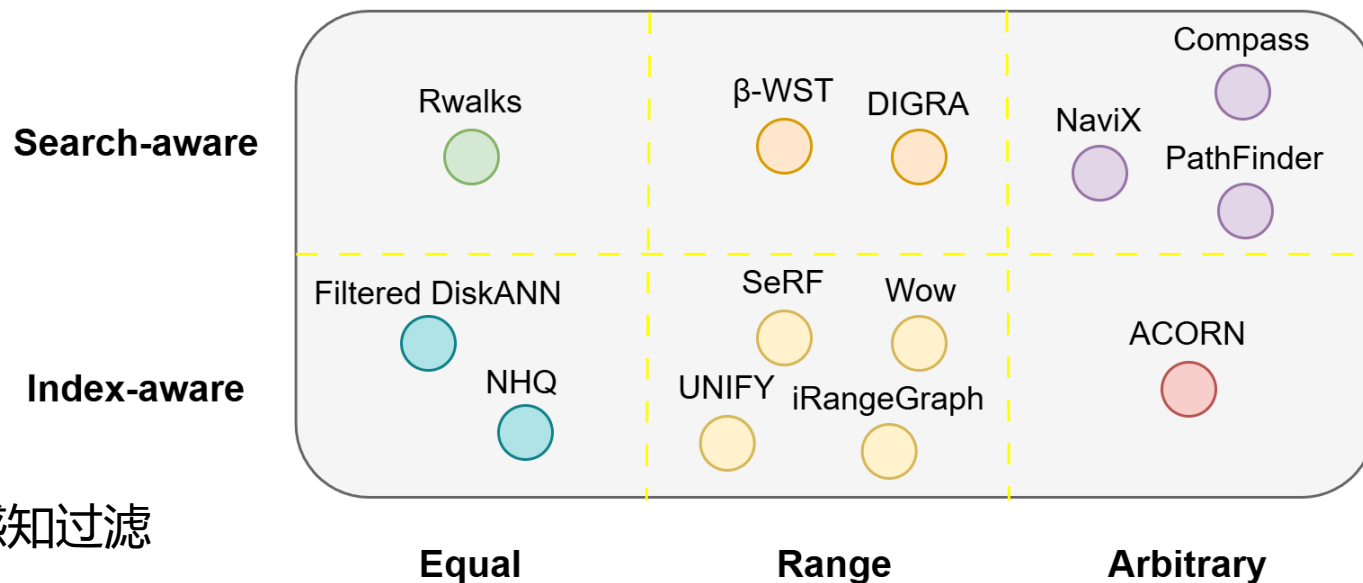
- 等值 (Equal) : 类别 / ID
- 范围 (Range) : 时间 / 数值区间
- 谓词无关 (Predicate-agnostic)

### □ 作用阶段 (Where to act)

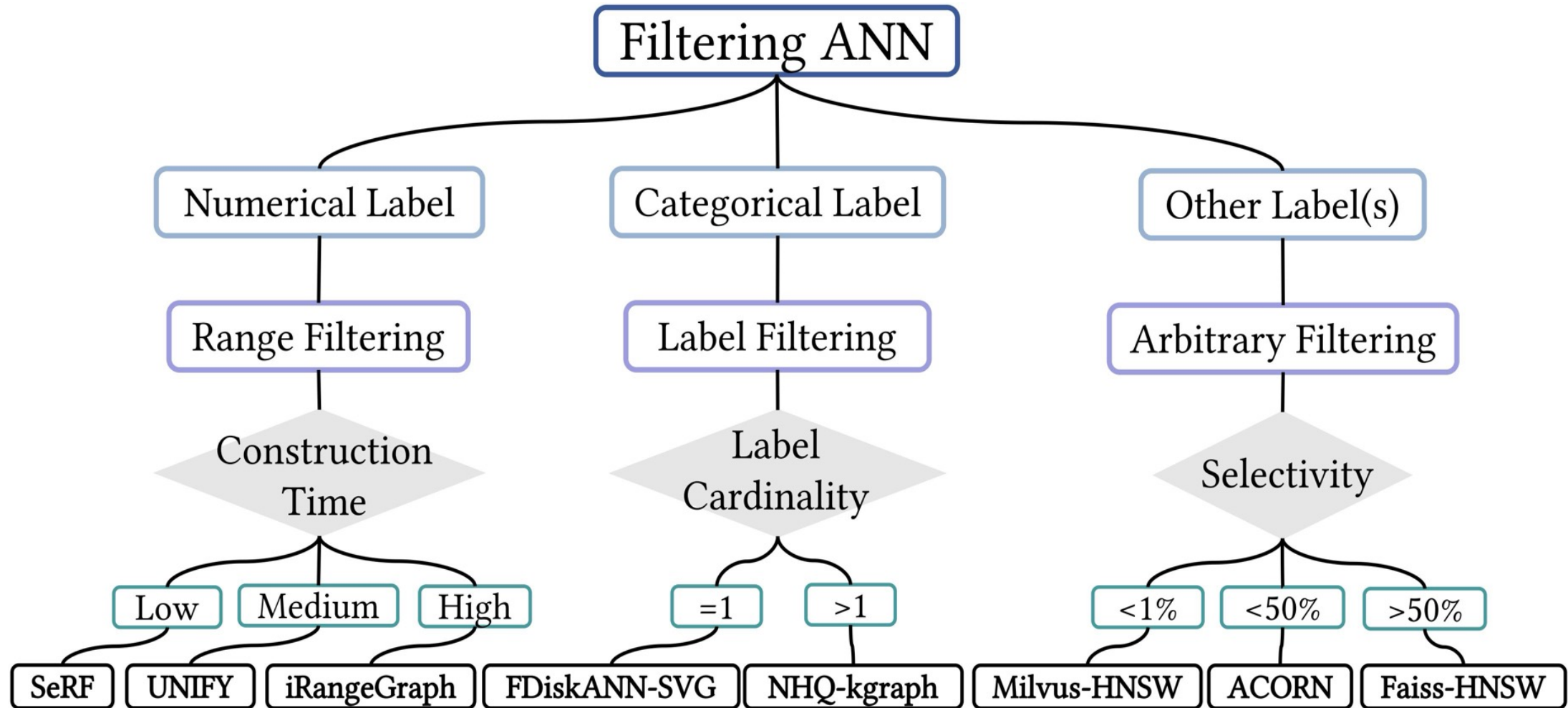
- 索引阶段 (Index-aware) : 构建时感知过滤
- 搜索阶段 (Search-aware) : 查询时处理过滤

## ■ 不同类型过滤的核心特点

- 等值过滤: 谓词条件离散、互斥; 满足条件节点稀疏, 遍历路径中断, 导致搜索“走死路”
- 范围过滤: 谓词条件连续、有序; 查询中可能的区间组合过多, 静态索引难以覆盖所有区间
- 谓词无关: 谓词条件不可知; 查询中不为特定谓词定制的索引难以保持通用性



## Algorithm Selection Guide



# Filtered Vector Search

| 索引名称                | 过滤类型        | 核心算法      | 更新支持 | 敏感性                    | 构建大小                   | 构建时间                   | 综合得分 |
|---------------------|-------------|-----------|------|------------------------|------------------------|------------------------|------|
| NHQ                 | Equal       | NSW(G)    | ✓    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 6    |
| Filtered-DiskANN 23 | Equal       | Vamana(G) | ✓    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 7    |
| ACORN 24            | Arbitrary   | HNSW(G)   | ✓    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 3    |
| SeRF 24             | Range       | HNSW(G)   | ✗    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 6    |
| iRangeGraph 24      | Range       | RNG(G)    | ✗    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 4    |
| UNIFY 24            | Range       | HSIG(G)   | ✓    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 6    |
| DIGRA 25            | Range       | HNSW(G)   | ✓    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 6    |
| RWalks 25           | Equal       | HNSW(G)   | ✓    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 7    |
| PathFinder 25       | Range+Equal | Vamana(G) | ✗    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 5    |
| NaviX 25            | Arbitrary   | HNSW(G)   | ✓    | <div><div></div></div> | <div><div></div></div> | <div><div></div></div> | 6    |

- Equal（等值过滤）方法整体最成熟，在构建规模、时间和综合评分上普遍占优。
- Range（范围过滤）方法能力最强，但设计复杂、代价更高，往往牺牲构建效率。
- Arbitrary（任意过滤）方法最通用，但在性能和稳定性上整体偏弱，综合评分通常不高。

- Filtered ANN 已从“外挂过滤”演进为索引与搜索协同设计的问题
- **不同类型索引trade-off清晰**
  - Equal: 工程最成熟, 效率与稳定性最优
  - Range: 表达能力最强, 但构建与维护代价高
  - Arbitrary: 通用性最高, 但性能与鲁棒性仍是瓶颈
- **图结构已被统一采用**, 研究创新从图本身转向过滤感知的搜索与调度机制



# Open Problems & Future Directions

01

## Arbitrary Filtering remains an open challenge

ACORN is the only method; IVF-based techniques may better support flexible subset search

02

## Hyperparameter tuning is impractical at scale

Performance highly sensitive to dataset size, distribution, selectivity, and dimensionality

03

## Limited integration with vector databases

Specialized indexes create barriers to adoption in general-purpose database systems

04

## Dynamic index updates underexplored

Only DSG supports updates; most methods require attribute-sorted insertion orders

05

## Attribute distribution analysis lacking

Uniform distribution assumption may not reflect real-world vector-attribute correlations

Thanks.